

* ER Model :- [Entity Relationship Diagrams]

⇒ Used for diagrammatic representation of Database design (High Level Database Design).

* Steps to Design DB :-

High Level
DB Design

- 1) Requirements :- what type of data/users/operations.
 - 2) Conceptual DB Design :- Identify Entity Sets [Tables] & Relationship sets [Draw ERD]
 - 3) Logical DB Design :- ERD to RDBMS tables [default 1NF]
 - 4) Schema Refinement :- Decompose RDBMS tables into higher NF (Normalization of DB) [Schema definition with Integrity Constraints]
 - 5) Physical DB Design :- Design Indexing.
 - 6) Security Design :- Grant Access to users based on Role of user.
- Low Level DB Design

* main Components of ER Diagram :-

- 1) Attributes
- 2) Entity Sets
- 3) Relationship Sets

1) Attributes :- ○

- Key Attribute =

- multivalued Attribute ○

- Derived Attribute ○ :- value of Attr. derived from other stored Attr.

Eg:-

DOB

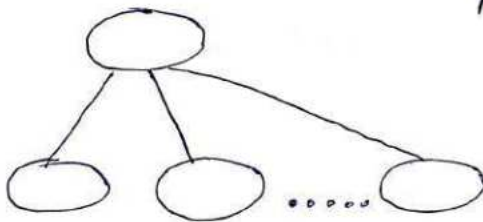
age

$$\text{age} = \text{Sysdate} - \text{DOB}$$

* Select sid, (sysdate - DOB) As age
FROM Stud;

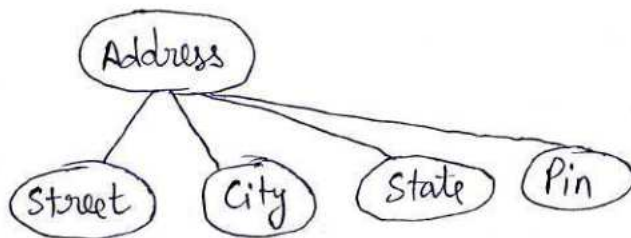
* Select sid, $\frac{\text{marks1} + \text{marks2} + \text{marks3}}{300} \times 100$ As percentage
FROM Stud;

- Composite Attribute :- Attr. which is represented as two or more Attr.

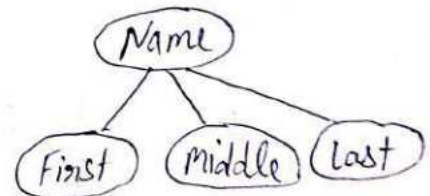


Eg:-

*)



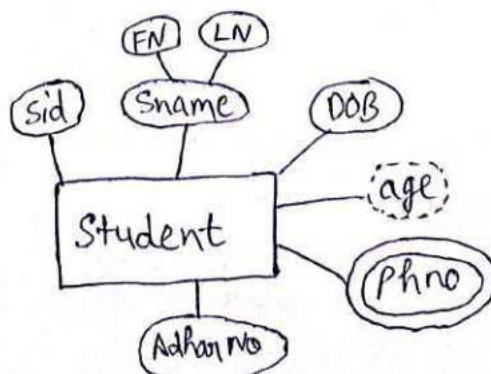
*)



2) Entity Sets :- Set of similar entities (objects are tuples).

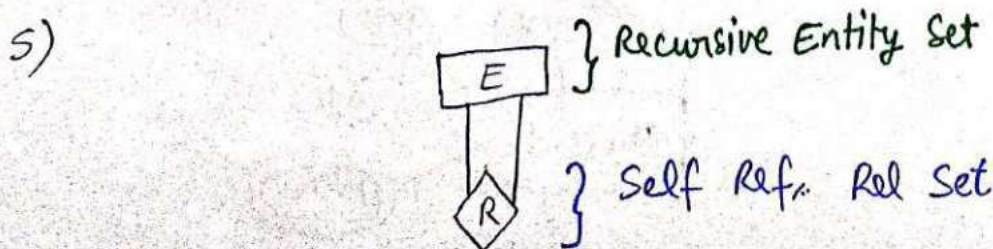
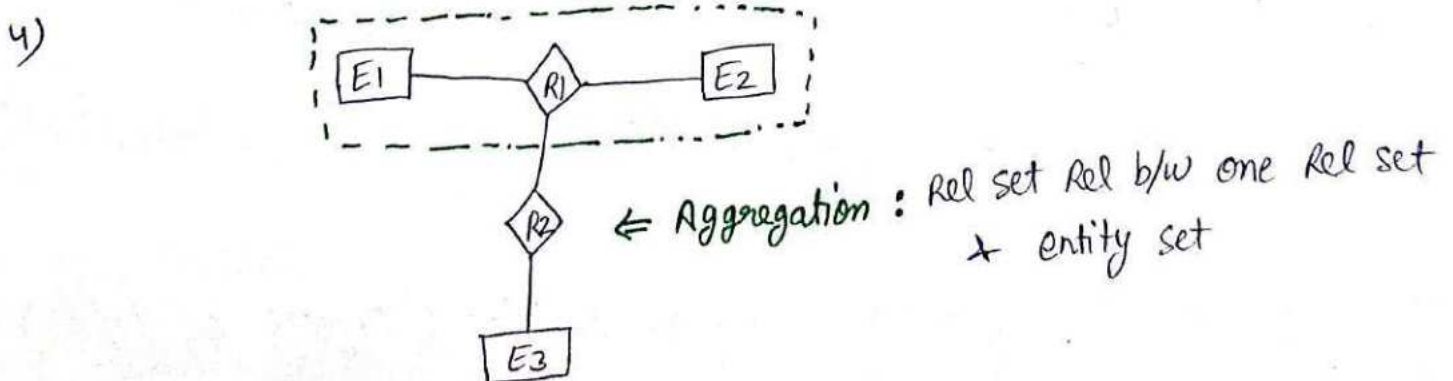
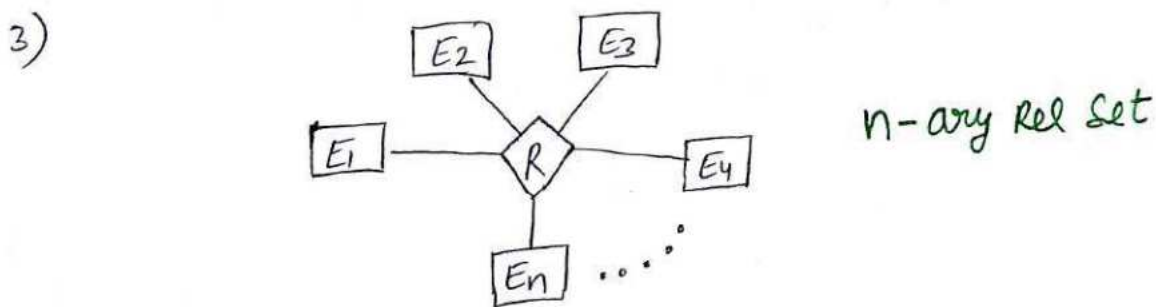
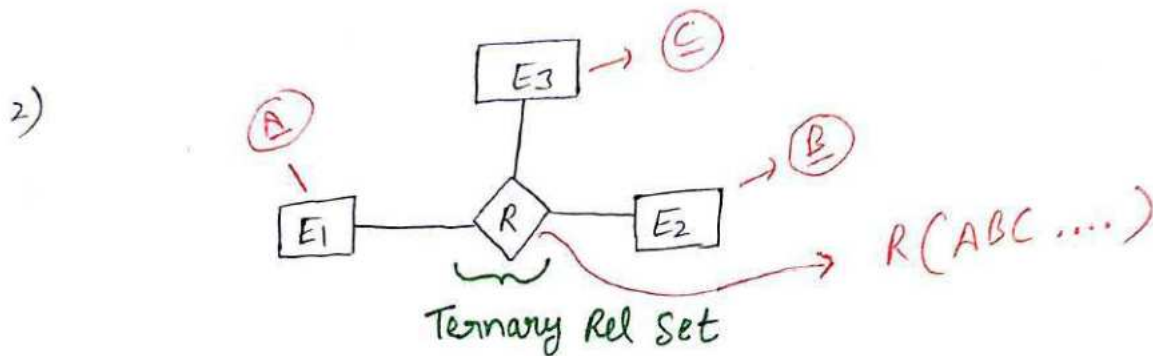
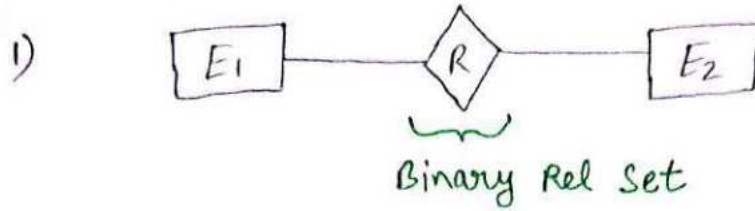
- Denoted by □

Eg:-

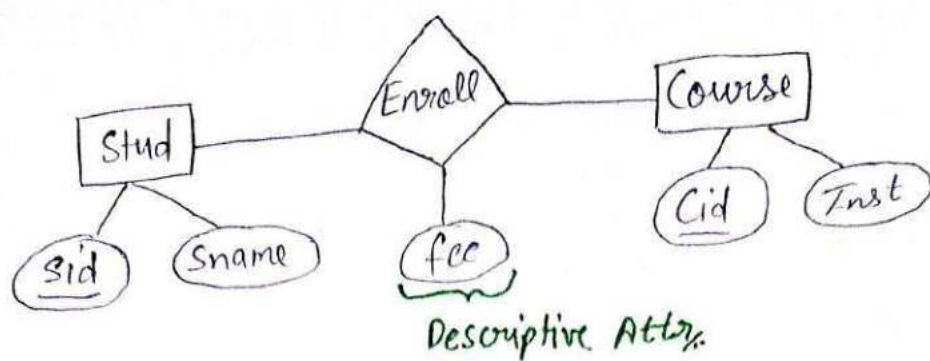


3) Relationship sets :- Used to relate two or more entity sets.

• Denoted by 



Eg8-



Stud(sid Sname) | Enroll(sid Cid fee) | Course(Cid Inst)

⇒ sid Cid → fee

S1 C1 4000

S2 C1 5000

S2 C2 4000

main IC of entity

Set :- Primary Key

main IC of

Rel Set :-

Foreign Key

⇒ Constraints of Relationship Set :-

① ⇒ Participation :-

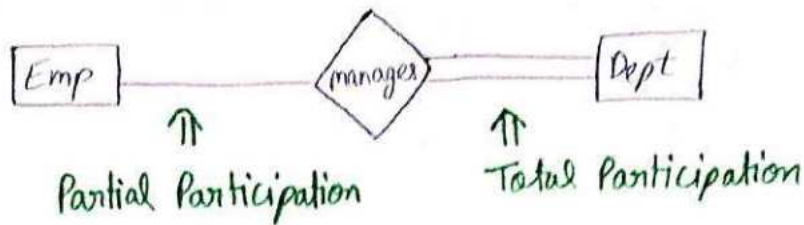
- If every entity of entity set must related with Relationship Set then total participation (must be 100% participation) otherwise Partial Participation (may or may not 100% participation).

TP :- [=====]

PP :- [—————]

⇒ Given specification :-

- Each dept must manage by one employee.
- Each emp can manage many dept.



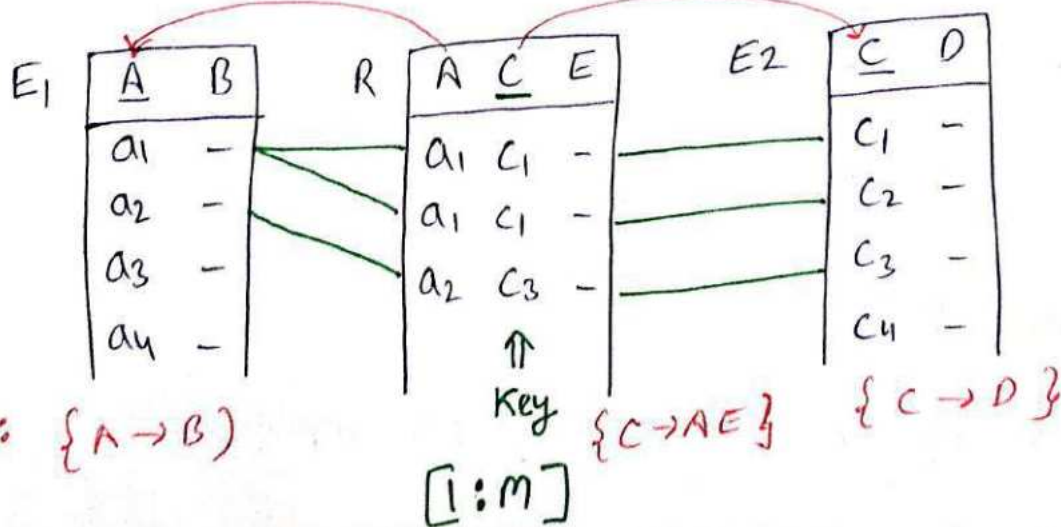
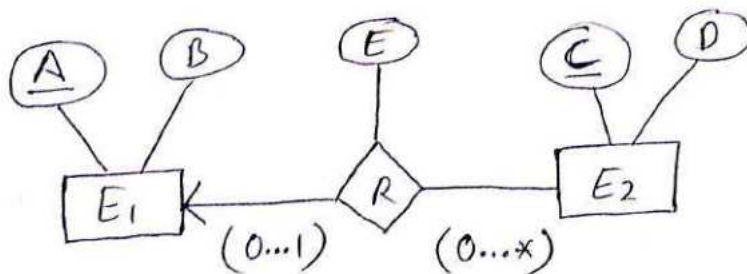
② ⇒ Mapping :- [Cardinality]

One :- [Almost one] ($\rightarrow$)
 many :- [0 or more] (---)

⇒ For Binary R. Set 4 possible mappings $[1:1, 1:m, m:1, m:m]$

:- CK of Rel Set is based on mapping

① 1:m mapping :-

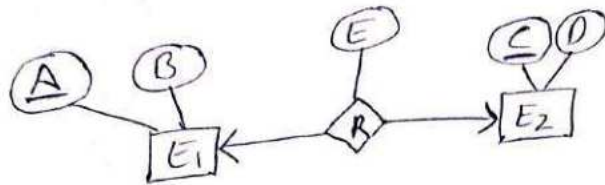


⇒ For 1:m mapping CK of $R(A, C, E)$ is $\{C\}$ which is also Key for right side entity set.

⇒ For m:m mapping Rel Set b/w E_1, E_2 , CK of Rel Set $R(A, C, E) \{ \underline{AC} \}$.

FD set : $\{A \rightarrow B\} \{AC \rightarrow E\} \{C \rightarrow D\}$

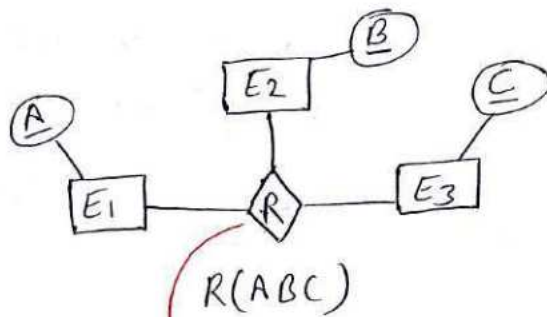
② 1:1 mapping :-



⇒ CK of $R(A, C, E)$ are $\{ \underline{A}, \underline{C} \}$ for 1:1 mapping.

FD set : $\{A \rightarrow B\} \{ \begin{matrix} A \rightarrow CE \\ C \rightarrow AE \end{matrix} \} \{C \rightarrow D\}$

*)



→ mappings

1:1:1
1:1:m
1:m:1
m:1:1
1:m:m
m:1:m
m:m:1
m:m:m

CK of $R(ABC)$

$\{ \underline{A}, \underline{B}, \underline{C} \}$
 $\{ \underline{C} \}$
 $\{ \underline{B} \}$
 $\{ \underline{A} \}$
 $\{ \underline{BC} \}$
 $\{ \underline{AC} \}$
 $\{ \underline{AB} \}$
 $\{ \underline{ABC} \}$

⇒ 8 possible mappings for Ternary R. Set.

Q) Draw ER Diagram for given specification

- Each Kid likes some candy &
- Each Candy likes by atmost one kid.

Soln



Some : One or more
[Atleast one]

atmost : [0 or 1]

Exactly one : [1]
(only one)

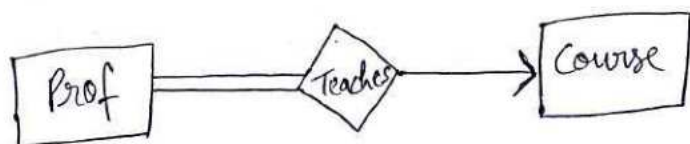
many : [0 or more]

- Mapping [1:m]
- TP at Kid side
- PP at Candy side

Q) Draw ER Diagram for given constraints

- Each professor teaches exactly one course

Soln



If other constraint is
not given :- default
⇒ many [0 or more]

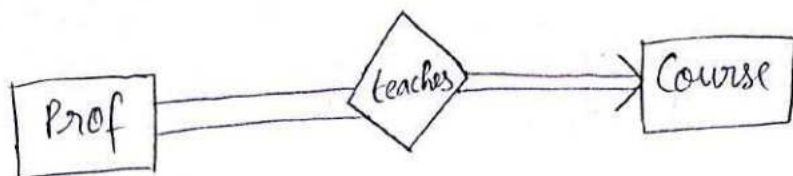
• mapping [m:1]

- Participation at Prof :- TP
- " " Course :- PP

Q) Draw ERD

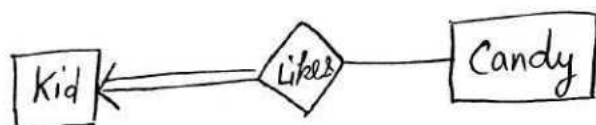
- Each Prof teaches exactly one course &
- Each course teaches by some Prof.

Soln



- mapping $[m:1]$
- Participation at Prof :- TP
- " " Course :- TP

Q)



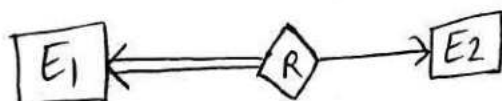
1) If 100 kids & 150 candies How many max tuples possible in likes relationship set?

- a) 100 ☒ b) 150 c) 5000 d) 25

Soln

Likes (Kid Cid)

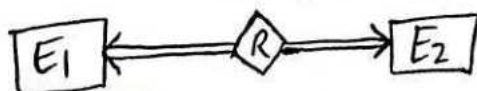
Q)



if n tuples in E_1 , m tuples in E_2 which is TRUE?

- ☒ a) $n \leq m$ b) $n \geq m$ c) $n = m$ d) None
TRUE

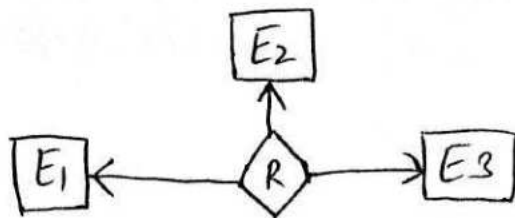
Q)



if n tuples in E_1 , m tuples in E_2 which is TRUE?

- a) $n \leq m$ b) $n \geq m$ ☒ c) $n = m$ d) None
TRUE

Q)



1) if 25, 15, 50 tuples in E_1, E_2, E_3 respectively.

How many max tuples possible in R ?

Soln Key $\Rightarrow R(\underline{A} \underline{B} \underline{C})$

$\left. \begin{array}{c} b_1 \\ b_2 \\ b_3 \\ b_4 \\ \vdots \\ b_{15} \end{array} \right\}$

Almost 15 tuples //

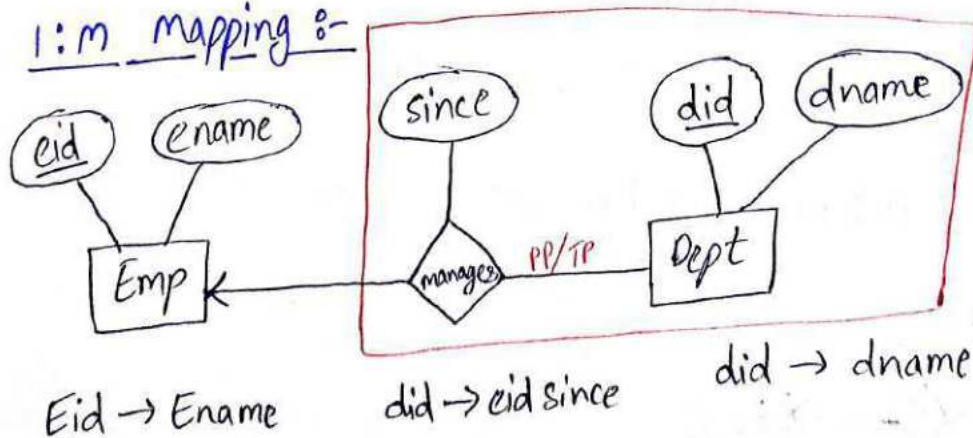
ii) mapping b/w E_1, E_2, E_3 is $m:m:m$ then max tuples possible in R ?

Soln $R(A B C)$

$= 25 \times 15 \times 50 = 18,750$ tuples.

* Design minimal RDBMS Rel for given ERD :-

a) 1:m mapping :-



Min 2 tables
&
1 FK

- If PP at dept side then FK eid should allowed NULL.
- If TP at dept side then FK eid is NOT NULL attribute.

Emp (<u>eid</u> ename)	m (eid <u>did</u> Since)	Dept (<u>did</u> dname)
e1 A	e1 d1 2020	d1 X
e2 B	e1 d2 2021	d2 X
e3 B	e2 d3 2020	d3 Y
e4 C		d4 Z
e5 A		

Emp (<u>eid</u> ename)	Dept-m (<u>did</u> dname <u>eid</u> ^{FK} Since)
e1 A	d1 X e1 2020
e2 B	d2 X e1 2021
e3 B	d3 Y e2 2020
e4 C	d4 Z NULL NULL
e5 A	

{ eid → ename }
BCNF

{ did → dname eid Since }
BCNF

⇒ 1:m rel merges with left side entity set :-

Emp-m (eid ename <u>did</u> Since)	dept (<u>did</u> dname)
e1 A	d1 -
e1 A	d2 -
e2 B	d3 -
	d4 -

not BCNF

{ eid → ename, did → eid Since }

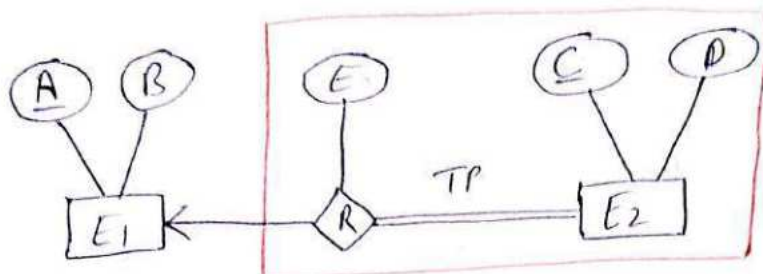
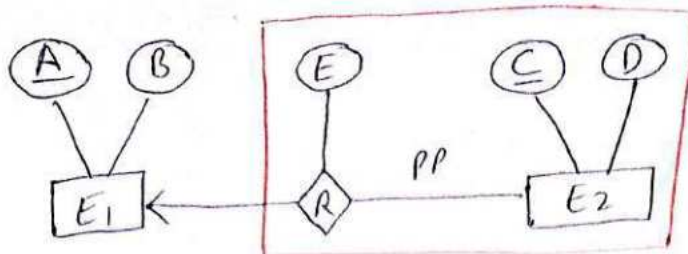
did : PK (NOT BCNF)

• Disadvantages :-

① Emp's who are not managers [PP entities] not possible to insert.

② No BCNF [Data Redundancy]

* 1:M



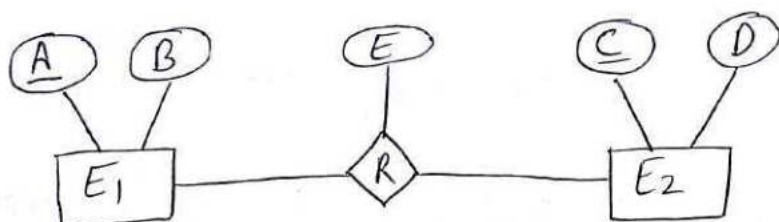
$A \rightarrow B$ $C \rightarrow AE$ $C \rightarrow D$

min 2 tables & 1 FK //

$E_1(\underline{A} B)$
 $A \rightarrow B$

$E_2 R(\underline{C} D (\underline{A} E))$
 $C \rightarrow DAE$

* m:m mapping :-

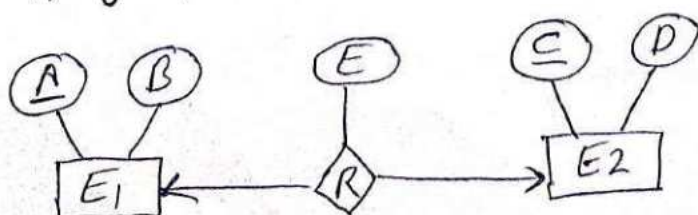


$A \rightarrow B$ $AC \rightarrow E$ $C \rightarrow D$

$E_1(\underline{A} B)$ $R(\underline{A} C E)$ $E_2(\underline{C} D)$
 $A \rightarrow B$ $AC \rightarrow E$ $C \rightarrow D$

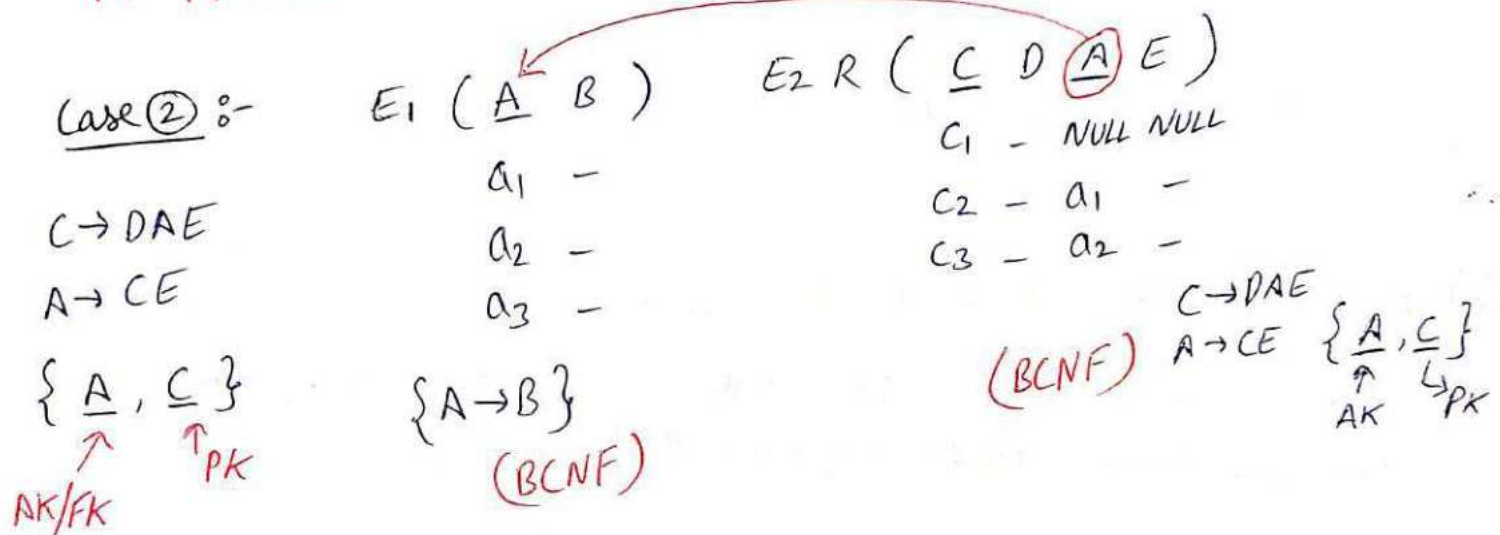
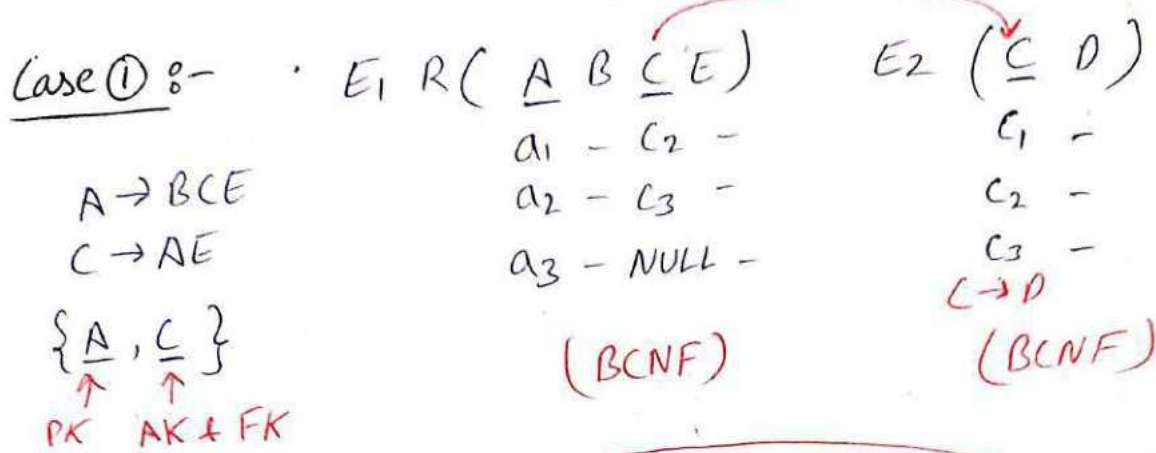
$\Rightarrow$ min 3 tables & 2 FK'S //

* 1:1 Mapping b/w E_1 & E_2 with PP at both ends.



$A \rightarrow B$ $A \rightarrow CE$ $C \rightarrow D$
 $C \rightarrow AE$

Min 2 tables
& 1 FK //



$\Rightarrow E_1 \text{ R } E_2$

$\left\{ \frac{A}{PK}, \frac{B}{AK} \right\}$

<u>A</u>	B	<u>C</u>	D	E
a₃	-	NULL	NULL	NULL
NULL	NULL	c₁	-	NULL
a ₁	-	c ₂	-	-
a ₂	-	c ₃	-	-

$$\{ A \rightarrow BCE, C \rightarrow DAE \}$$

$\{ \underline{A}, \underline{C} \}$: Cand Keys

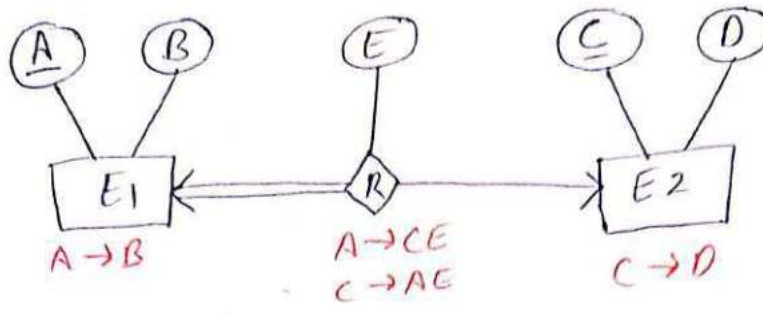
→ Allowed null becoz of PP at E_1

allowed null becoz of
PP at E2

(WRONG Design)

* 1:1 mapping with atleast one side TP.

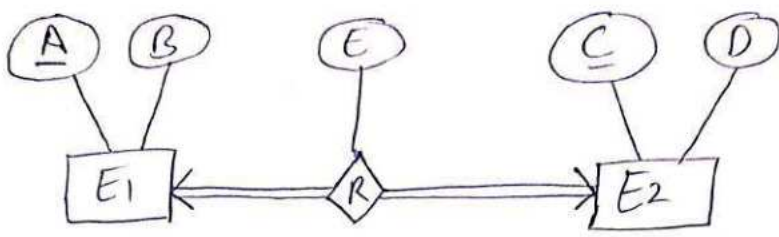
may not efficient



E1 R E2

A	B	C	D	E
AK		PK		

$A \rightarrow BCE$
 $C \rightarrow DAE$



E1 R E2

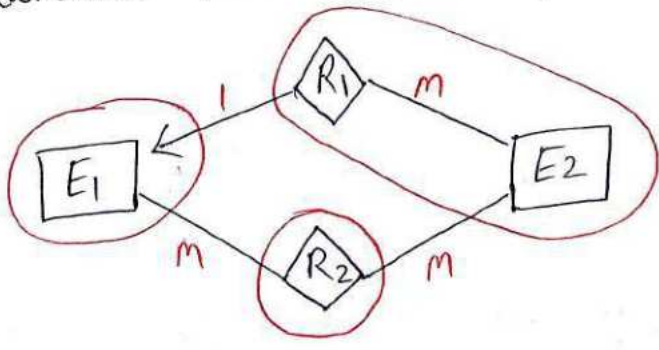
A	B	C	D	E
PK		UNIQUE NOT NULL		

$A \rightarrow BCE$
 $C \rightarrow DAE \{ \underline{A}, \underline{C} \}$

=> "min 1 Table & No FK's"

Q) E1 E2 entity sets & R1 R2 Relationship sets Related b/w E1 E2 with 1:m, m:m mappings respectively. How many min relational tables required?

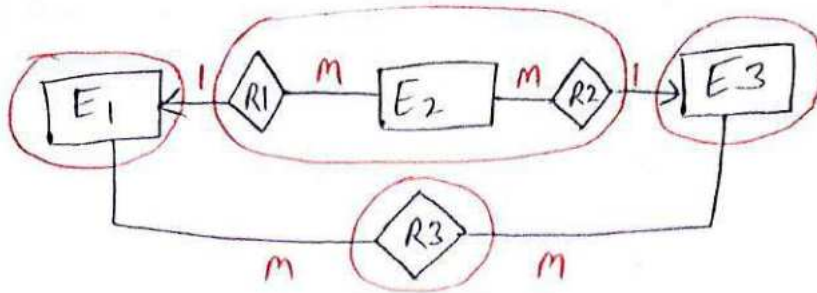
Soln



Ans = 3

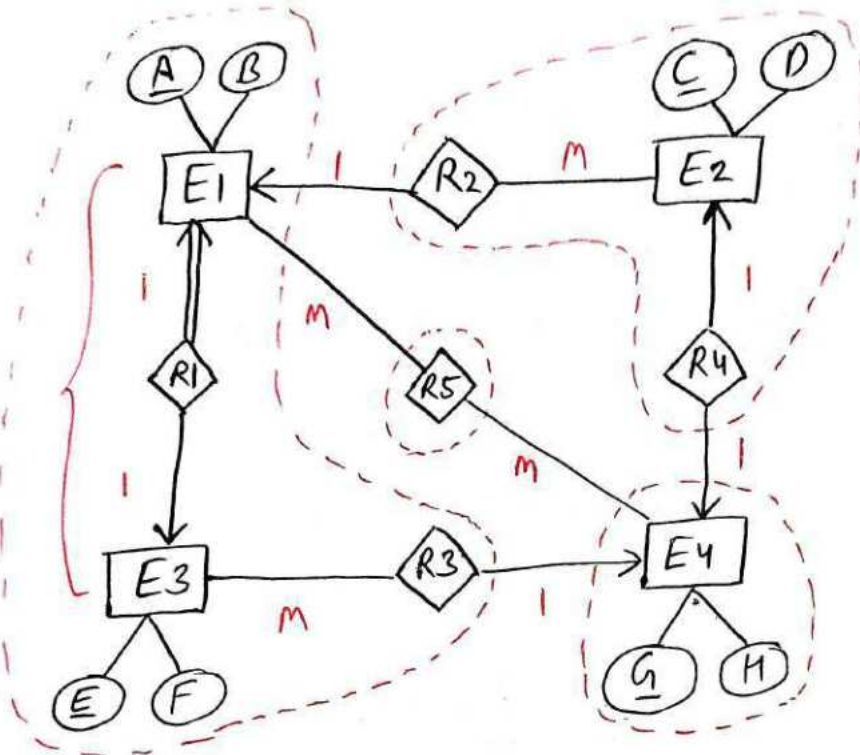
Q) E1 E2 E3 entity sets R1 Relationship b/w E1 E2 with 1:m, R2 Rel b/w E2 E3 with m:1, R3 Rel b/w E1 E3 with m:m. How many min relational tables required?

Soln



Ans = 4 //

Q)



Soln

Ans = 4 tables & 5 FK'S

⇒ E1, R1, E3, R3 ($\frac{A}{AK}, \frac{B}{PK}, \frac{E}{PK}, F, \underline{G}$)
 FK of m:1

$\left\{ \frac{A}{AK}, \frac{E}{PK} \right\}$

⇒ E2, R2, R4 ($\frac{C}{PK}, D, \underline{A}, \underline{G}$)
 FK of 1:m

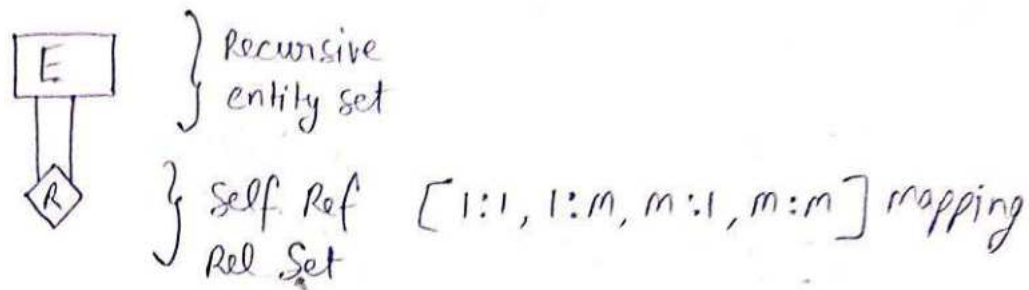
$\left\{ \frac{C}{PK}, \frac{G}{AK} \right\}$

⇒ E4 ($\underline{G}, H$)

⇒ R5 ($\underline{A}, \underline{G}$)

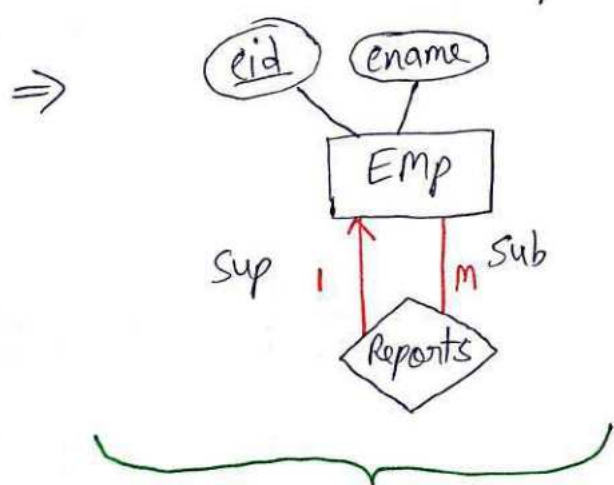
* Self Referential Relationship Set :-

⇒ Entity of entity set E is related to some other ordered pair entity of same entity set E .



Eg:- ⇒ Emp entity set, Relationship set related b/w supervisors, subordinates where sup, sub are employees.

- i) Each supervisor supervises many subordinates,
Each subordinate reports to one supervisor.



min 1 table & 1 FK

(1:m)

Emp(<u>eid</u> ename)		Rep(Sup Sub)	
e1	A	e1	e2
e2	B	e1	e3
e3	C	e2	e4
e4	D		

{ Sub } → Key

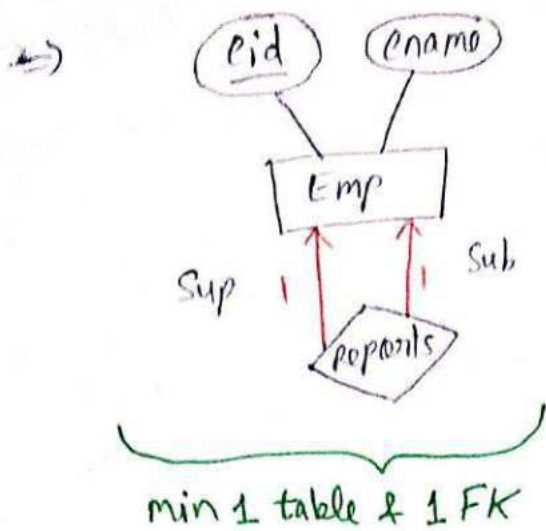
Emp-Rep

<u>eid</u>	ename	Sup
e1	A	NULL
e2	B	e1
e3	C	e1
e4	D	e2

{ eid, Sup }
PK AK

⇒ ordered pair of entities of Emp entity set are related.

- ii) Each supervisor supervises one subordinate,
Each subordinate reports to one supervisor.



Emp (<u>eid</u> ename)	Rep (<u>Sup</u> <u>Sub</u>)
	(1:1)
e1 A	e1 e2
e2 B	e2 e3
e3 C	e3 e4
e4 D	

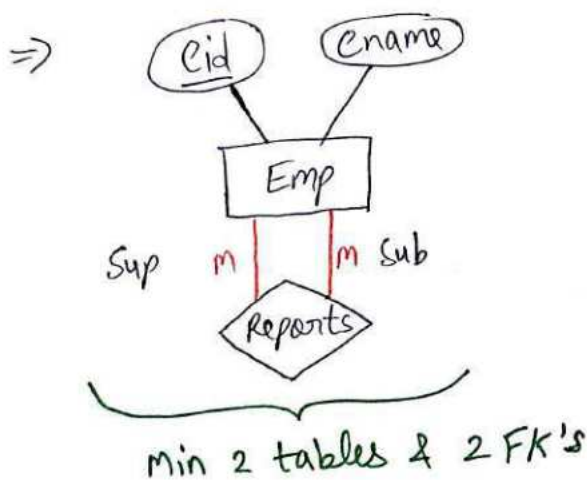
↑ key ↑ key
 { Sup, Sub }

Emp-Rel

<u>eid</u>	ename	<u>Sub</u> ^{FK}
e1	A	e2
e2	B	e3
e3	C	e4
e4	D	NULL

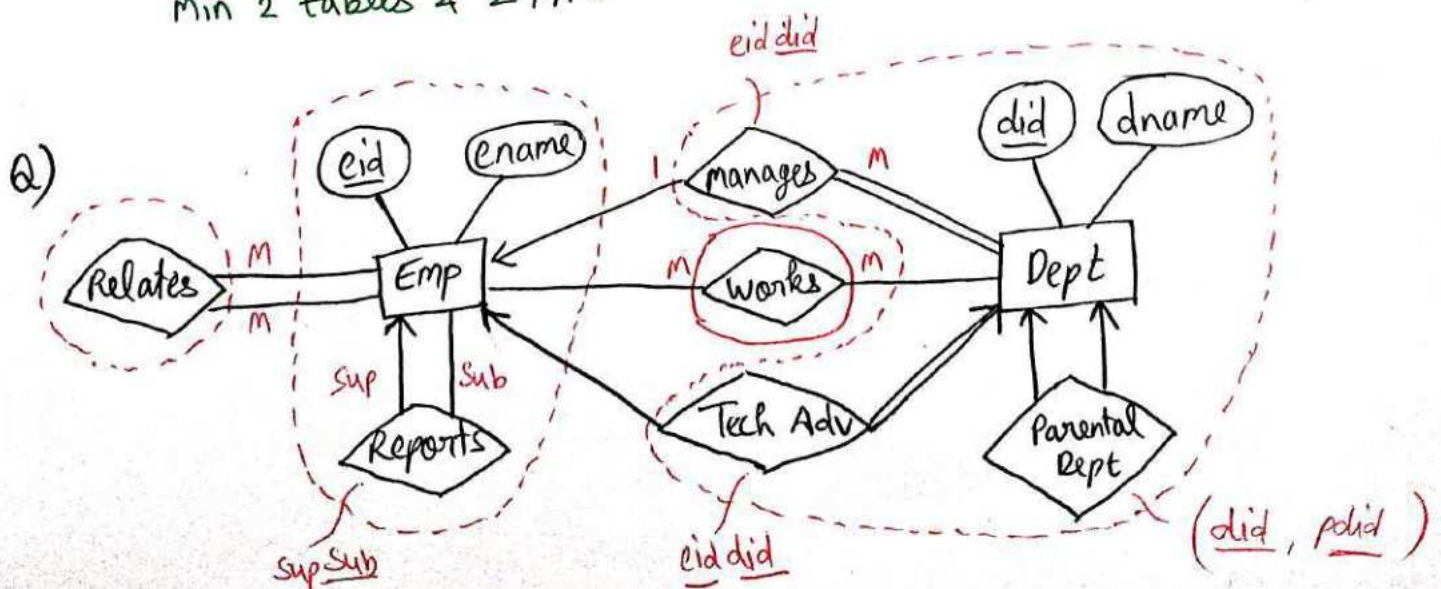
{ eid PK, Sub AK }

iii) Each supervisor supervises many subordinates,
 Each subordinate reports to many supervisors.



Emp (<u>eid</u> ename)	Rep (<u>Sup</u> <u>Sub</u>)
	(m:m)
e1 A	e1 e2
e2 B	e1 e3
e3 C	e4 e3
e4 D	

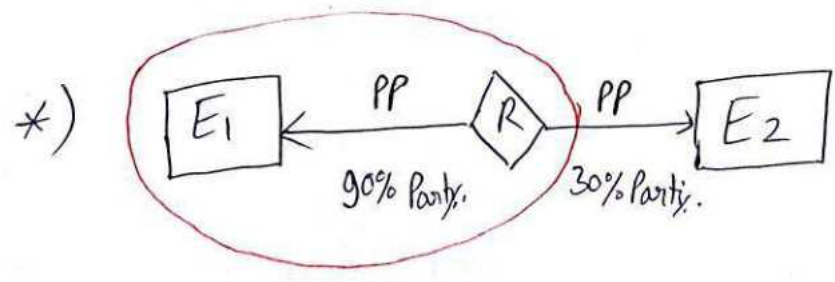
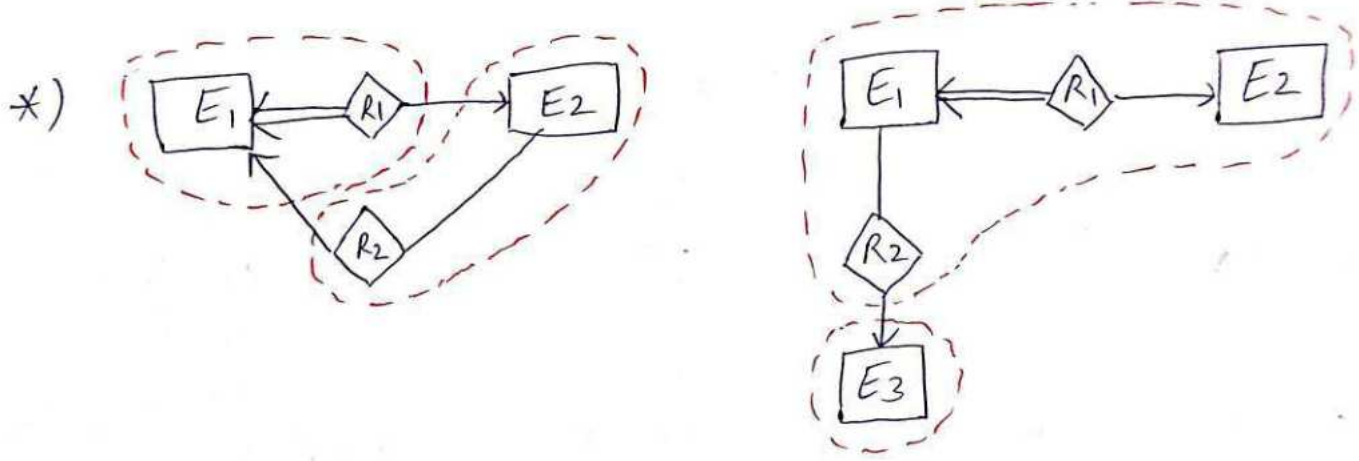
{ Sup Sub }



Soln

- EmpRep (eid, ename, Sup)
FK(1:m)
- Relates (eid1 eid2)
- DeptManages (did, dname, m_eid, Tech_Adv_eid, P_did)
TechAdv
Parental Dept
FK(1:m) FK(1:1) FK(1:1)
- Works (eid did)

⇒ min 4 tables & 4 FK'S

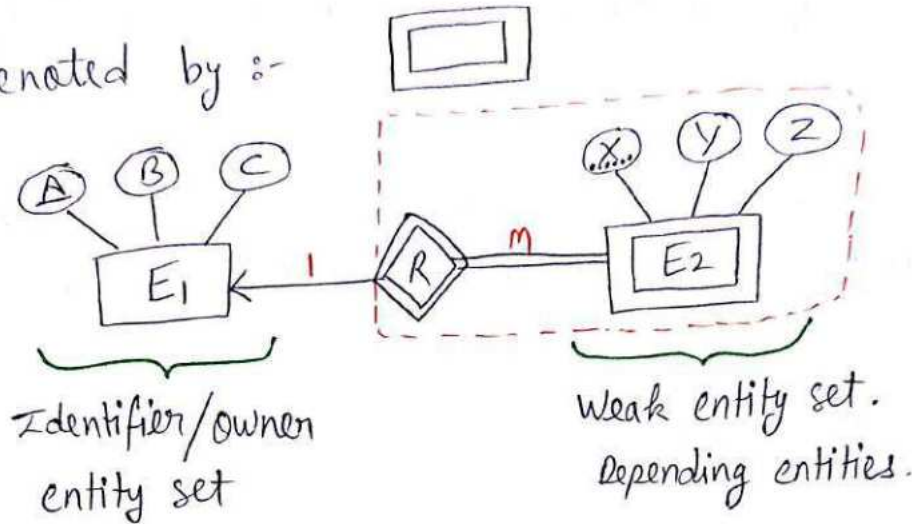


* Weak Entity Set :-

⇒ Entity set with no Key

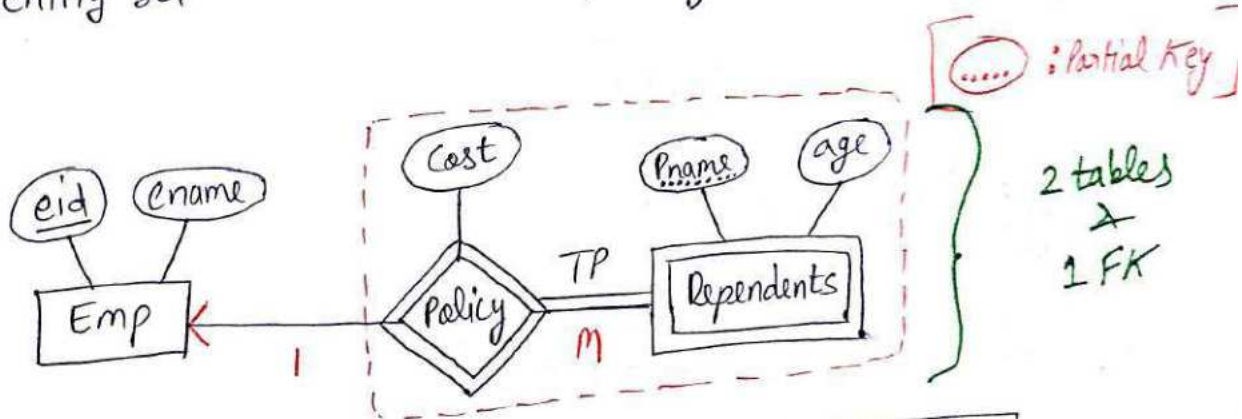
⇒ Attributes of weak entity set not sufficient to differentiate entity uniquely.

⇒ Denoted by :-



$E_1(\underline{ABC}) \xrightarrow{FK} E_2R(\underline{XYZ})$
RDBMS Design

Eg :-



<u>eid</u>	ename	<u>eid</u>	<u>Pname</u>	Cost	<u>Pname</u>	age
e1	A	e1	P1	-	P1	10
e2	B	e2	P1	-	P1	12
e3	B	e2	P2	-	P2	13
e4	C	e3	P2	-	P2	13

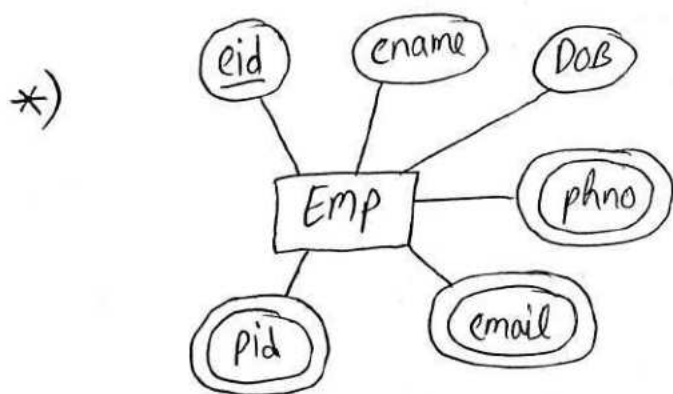
eid Pname
1:M

(depending)

RDBMS Design { Emp(eid ename) Rep-Pol (eid Pname age Cost) }
FK

NOTE :-

- ① Participation at weak entity set side must be Total.
- ② mapping b/w Identifier set & weak entity set must be 1:M.
No other mappings possible.
- ③ weak entities uniquely identified by combining partial Key & identifier key.
- ④ weak entity set & multi valued Atty. allowed to represent in ER diagrams not allowed in RDBMS tables.



⇒ FD set { $eid \rightarrow ename\ DOB$ }

⇒ Emp ($eid\ ename\ DOB\ phno\ email\ pid$)

{ $eid \rightarrow ename\ DOB$ } CK: { $eid\ phno\ email\ pid$ }

not in 2NF

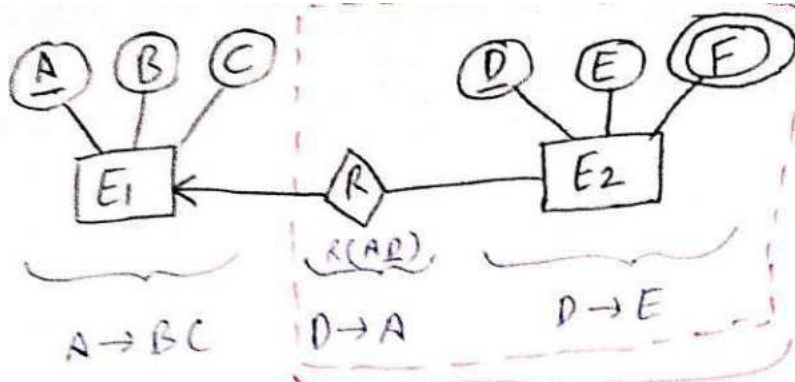
1NF

⇒ Decomposition into higher NF :-

2NF { Emp1 (eid $ename\ DOB$) Emp2 ($eid\ phno\ email\ pid$)
3NF { $eid \rightarrow ename\ DOB$ } { No Non-trivial FD's }
BCNF

MVD ⇒ $eid \rightarrow phno$
 $eid \rightarrow email$
 $eid \rightarrow pid$

Q) ①



How many min rel required which satisfy 3NF?

Scn

$A \rightarrow BC$

$R(AD)$

$D \rightarrow A$

$D \rightarrow E$

1:m Rel set Combine with right side entity set.

$\Rightarrow E_1(\underline{A}BC)$ $E_2R(D E F \underline{A})$

$A \rightarrow BC$
 $\underline{A} : CK$
 BCNF

$D \rightarrow EA$
 not 2NF $\underline{D} : CK$

1NF ✓
 2NF X

$\Rightarrow E_1(ABC)$ $D^+ = \{DEA\}$

$A \rightarrow BC$

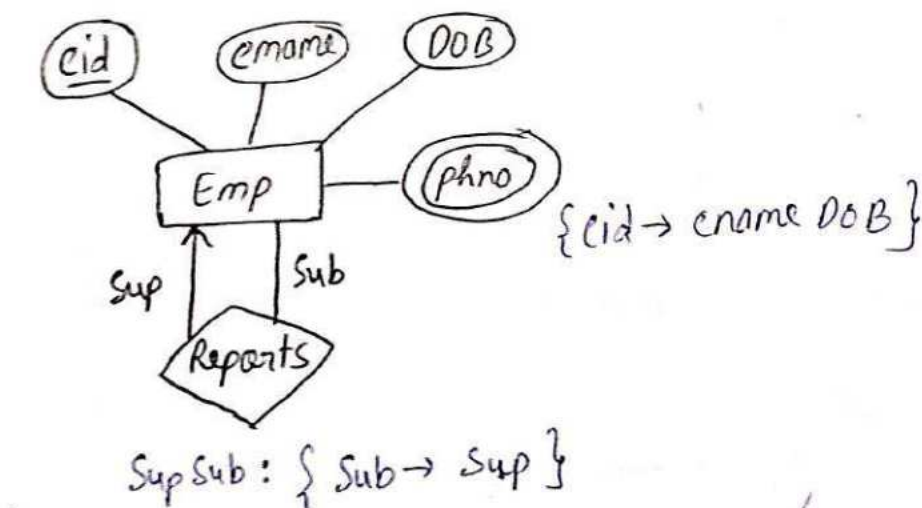
$E_{21}R(\underline{D}EA)$ $E_{22}(DF)$

$D \rightarrow EA$ No non-trivial FD's

2NF ✓
 3NF ✓
 BCNF ✓

Ans $\Rightarrow 3$

②



$\Rightarrow Emp_R(\underline{cid} \text{ ename DOB phno } \underline{Sup})$

$[cid \text{ phno}]$ $cid \rightarrow ename DOB Sup$

PK (PD)

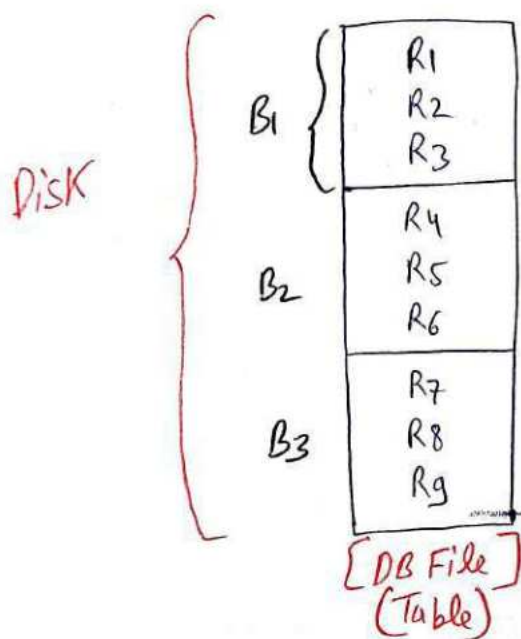
Foreign Key Reference is wrong.

⇒ Emp1 Rep (eid [←] ename DOB Sup) Emp2 (eid phno)
 eid → ename DOB Sup

Ans = 2 //

* File Organization & Indexing :-

- ⇒ DB is collection of files (tables)
- ⇒ File is collection of blocks (pages)
- ⇒ Block is collection of records

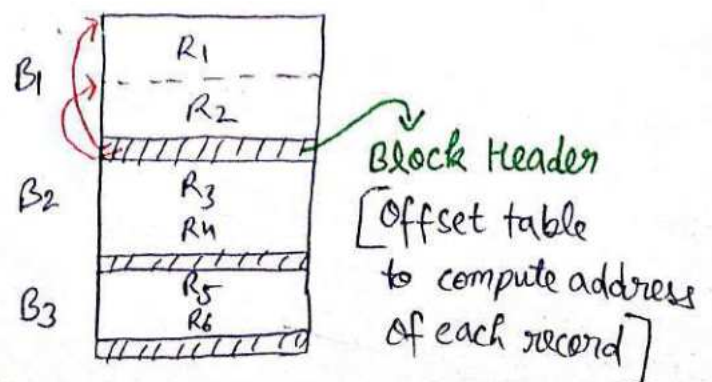


⇒ Data access from Disk to main memory is block by block.

* Records in DB File :-

① Fixed Length Records :-

Create table R
 (A char(100),
 B char(100),
 C char(200)
); (400 bytes record size)



② Variable Length Records :-

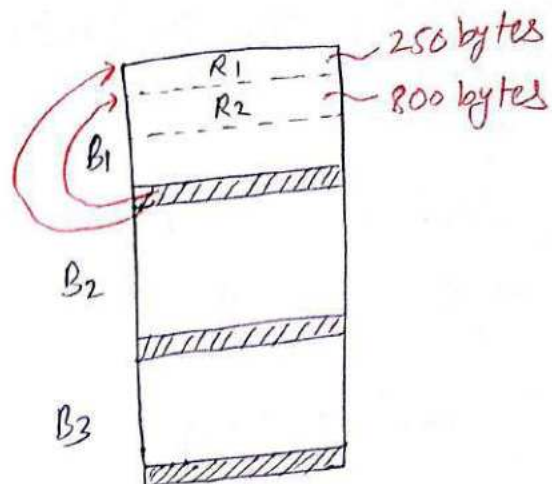
Create table S

(D Char(100),

E Char(100),

F text

); [Records of 'S' are variable length]

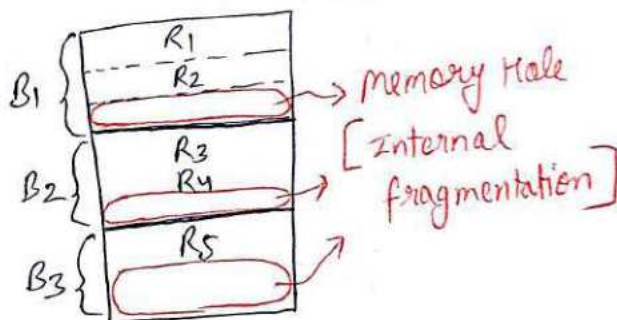


* Records Organization in DB File :-

① Unspanned Org :- [Preferred with Fixed length DB File]

⇒ Complete record must allocate in one block.

Eg:- Block : 1000 bytes
Record : 400 bytes

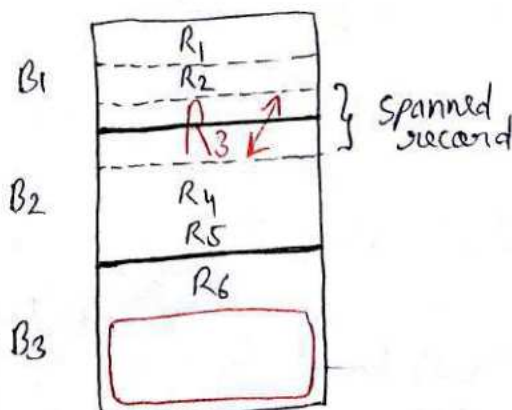


⇒ Less access cost & easy to organize.
⇒ more possible internal fragmentations.

② Spanned Org :- [Preferred with variable length DB File]

⇒ Record allowed to span in more than one block.

Eg:- Block : 1000 bytes
Record : 400 bytes



⇒ Less internal fragmentations.

⇒ more I/O cost may required to access spanned records.

NOTE :-

* Unspanned Org is preferred to organize DB File fixed length records.

* Spanned Org is preferred to organize DB File variable length records.

Eg:- Records 30, 80, 40, 65, 50, 60, 20 bytes respectively.
If unspanned org used to store records in DB file with block size : 100 bytes. How much wastage of Disk space?

Soln

B ₁	R ₁ [30] ↗ 70
B ₂	R ₂ [80] ↗ 20
B ₃	R ₃ [40] ↗ 60
B ₄	R ₄ [65] ↗ 35
B ₅	R ₅ [50] ↗ 50
B ₆	R ₆ [60] R ₇ [20] 20

% of memory wastage

$$= \frac{70 + 20 + 60 + 35 + 50 + 20}{600} \times 100$$

$$= \frac{255}{6} = 42.5 \%$$

* Block Factor of DB File :-

⇒ maximum possible records which can store in one block.

Block size : B bytes

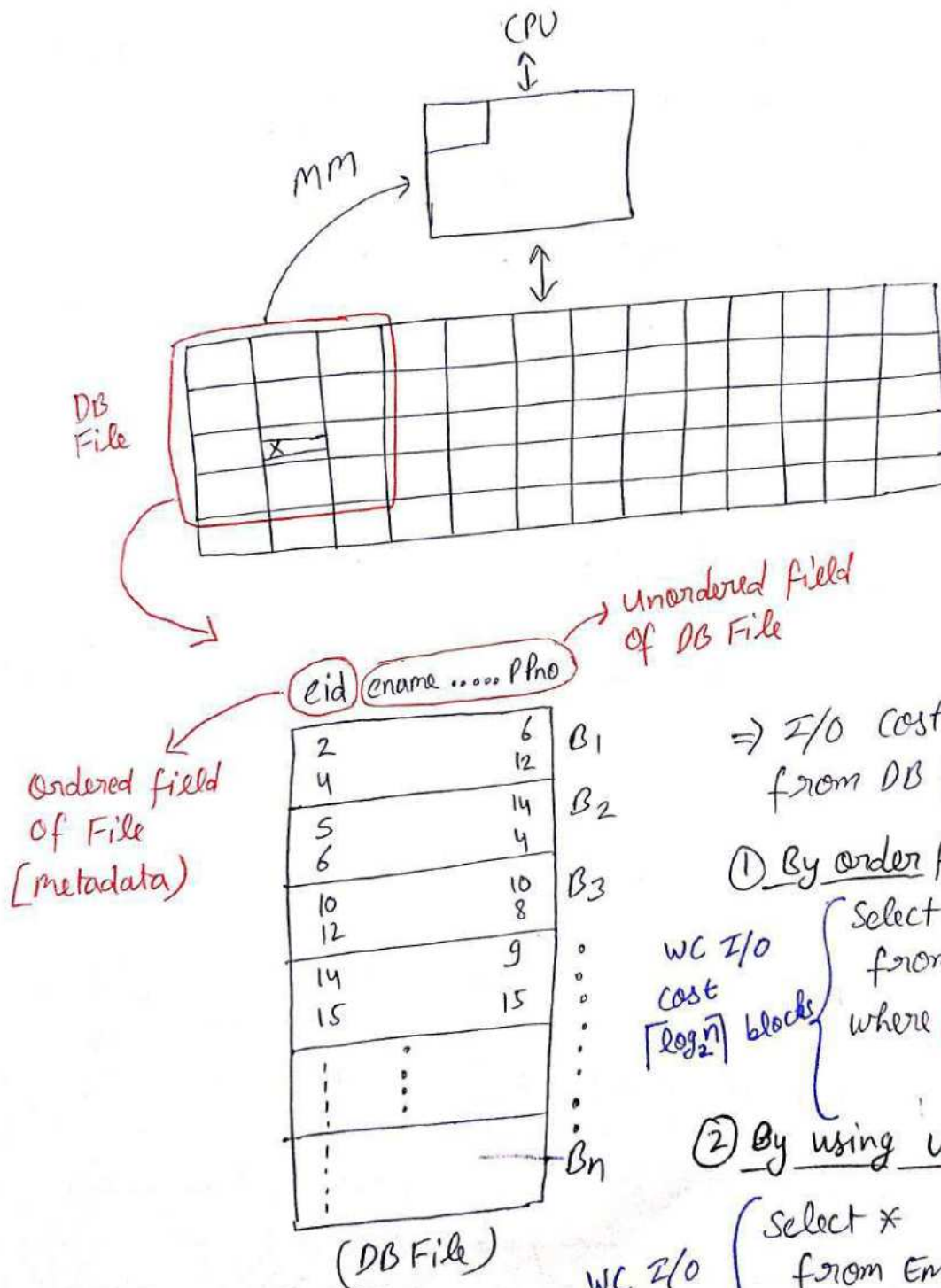
Record size : R bytes (Fixed length)

Block Header : H bytes

$$B_f \text{ of DB} = \left\lfloor \frac{B-H}{R} \right\rfloor \text{ records/block} \quad // \text{ unspanned org is used}$$

⇒ Access cost [I/O cost] :-

- Number of Disk blocks required to transfer from Disk (DB File) to main memory in order to access required data.



⇒ I/O cost to access record from DB file without Index.

① By order field :-

WC I/O cost $\left\lceil \log_2 n \right\rceil$ blocks

Select *
from Emp
where eid = x ;
ordered field

[Binary Access]

② By using unordered field :-

WC I/O cost n blocks

Select *
from Emp
where ppno = y ;
unordered field.

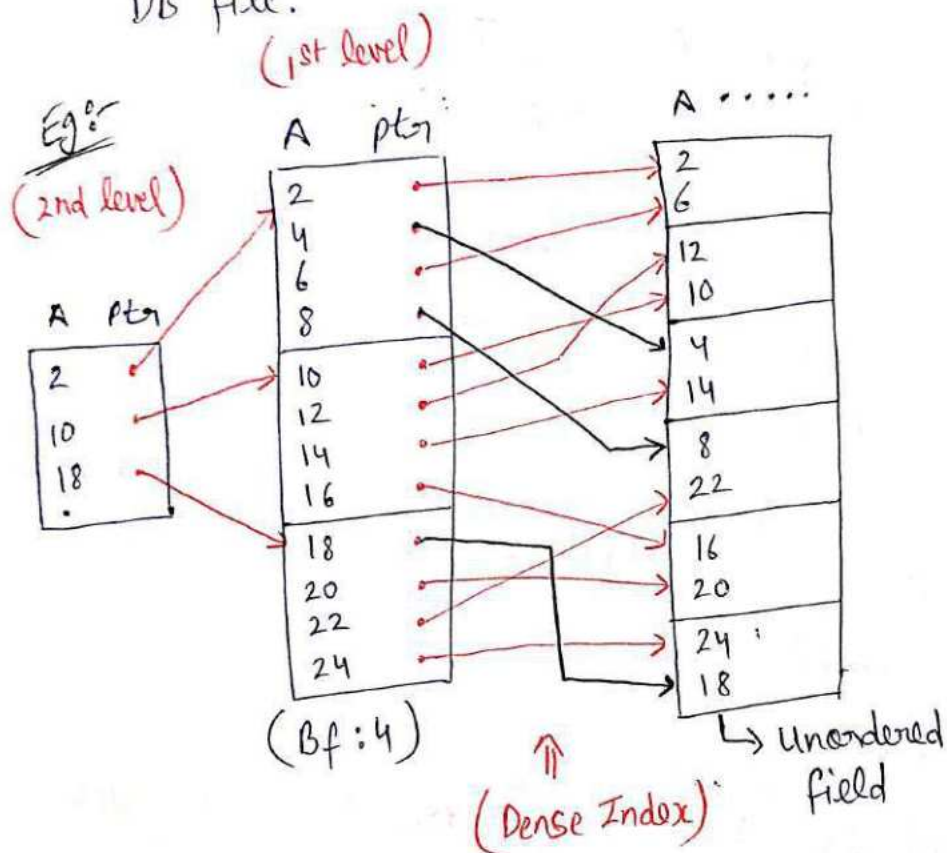
[Sequential Blocks Access]

$$\text{Block factor of Index file} = \left\lfloor \frac{B-H}{K+P} \right\rfloor \text{ entries/block}$$

⇒ Categories of Index :-

① Dense Index :- For each record of DB File there exists entry in index file.

⇒ Dense index possible over ordered field or unordered field of DB file.



⇒ more index entries

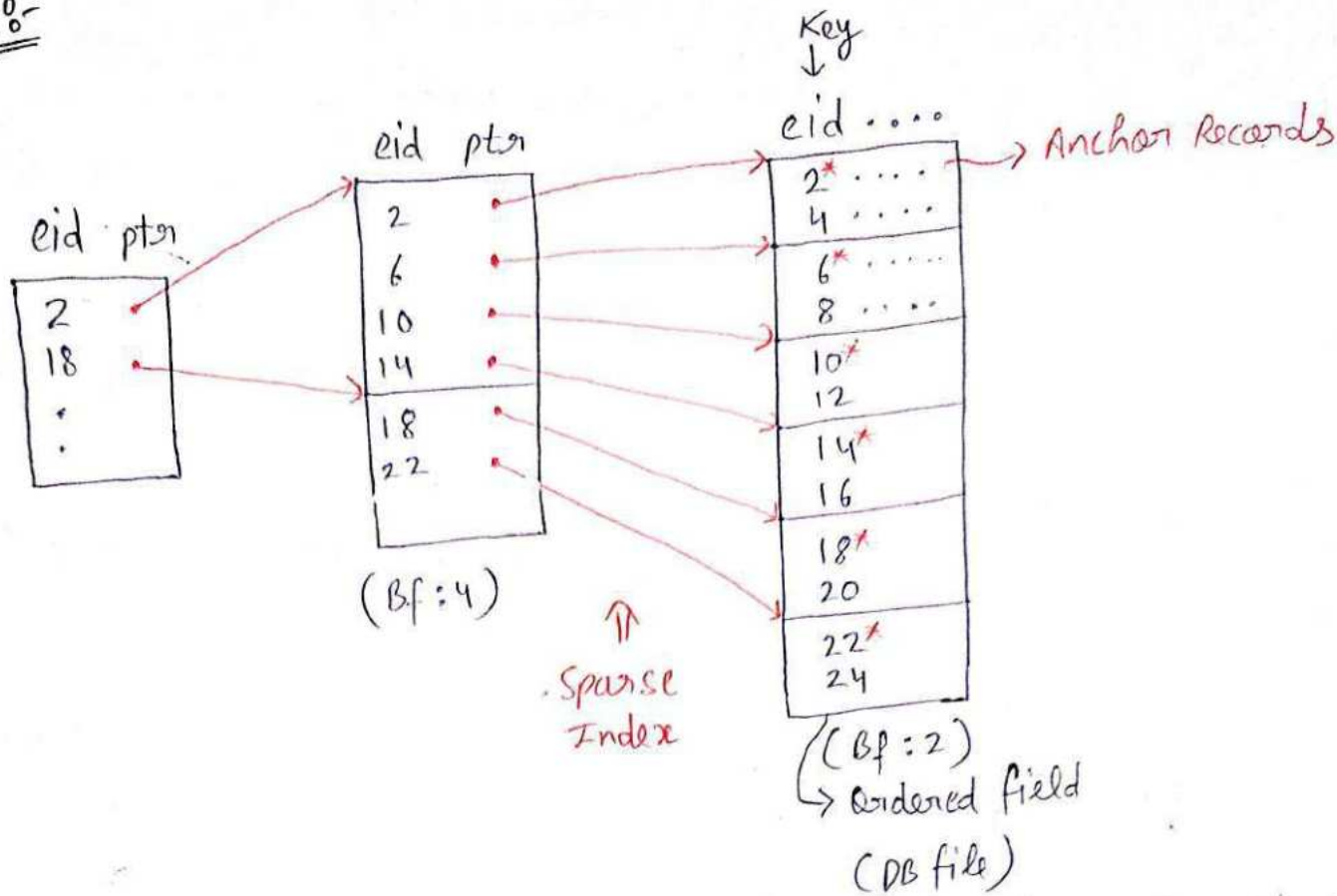
⇒ If dense index used $\left\{ \begin{array}{l} \text{\# of entries} \\ \text{in Index} \end{array} \right\} = \left\{ \begin{array}{l} \text{\# of records of} \\ \text{the DB file} \end{array} \right\}$

② Sparse Index :- For set of records there exists entry in index file

⇒ Less index entries

⇒ If sparse index over Non-Key $\left\{ \begin{array}{l} \text{\# of index} \\ \text{entries} \end{array} \right\} < \left\{ \begin{array}{l} \text{\# of records} \\ \text{of DB file} \end{array} \right\}$

Eg:-



Q) DB consists 1 lakh records with record size 110 bytes stored in DB File with block size 1024 bytes, Search key which is Cand. Key of the file is 9 bytes, Pointer size used for index is 10 bytes.

① I/O cost to access record without index

- by using ordered field
- by using unordered field

② If dense index build over search key (a) number of dense index blocks.

b) I/O cost to access record using 1st level index.

c) If multi level index used with dense at 1st level How many Index total index blocks? Levels of index? I/O cost to access record?

③ If sparse index used over search key

a) # of sparse blocks?

b) I/O cost to access record using 1st level.

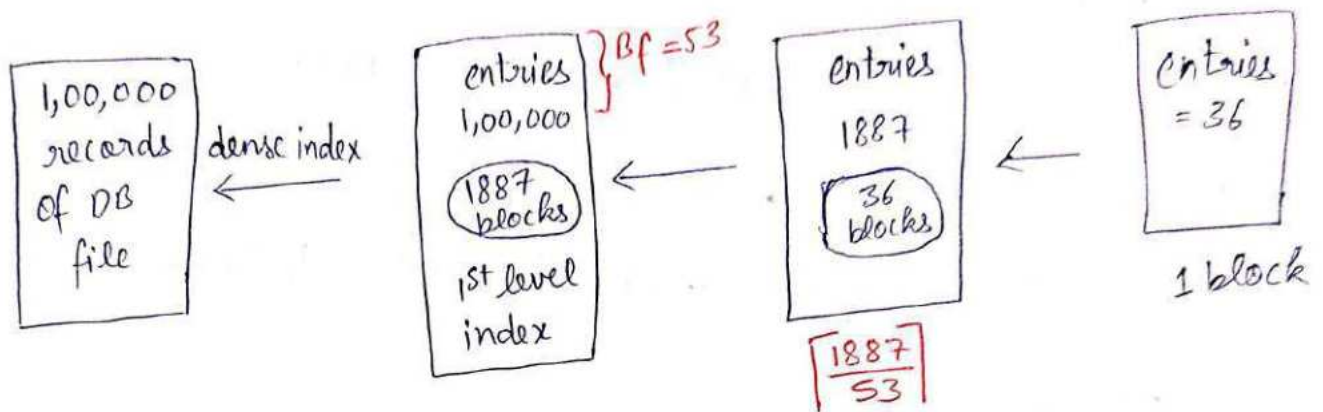
c) multi level index with sparse index

1) Index blocks $\underline{210+4+1 = 215}$

2) Levels of index $\underline{3}$.

3) I/O cost to access record $\underline{K+1 = 3+1 = 4 \text{ blocks}}$

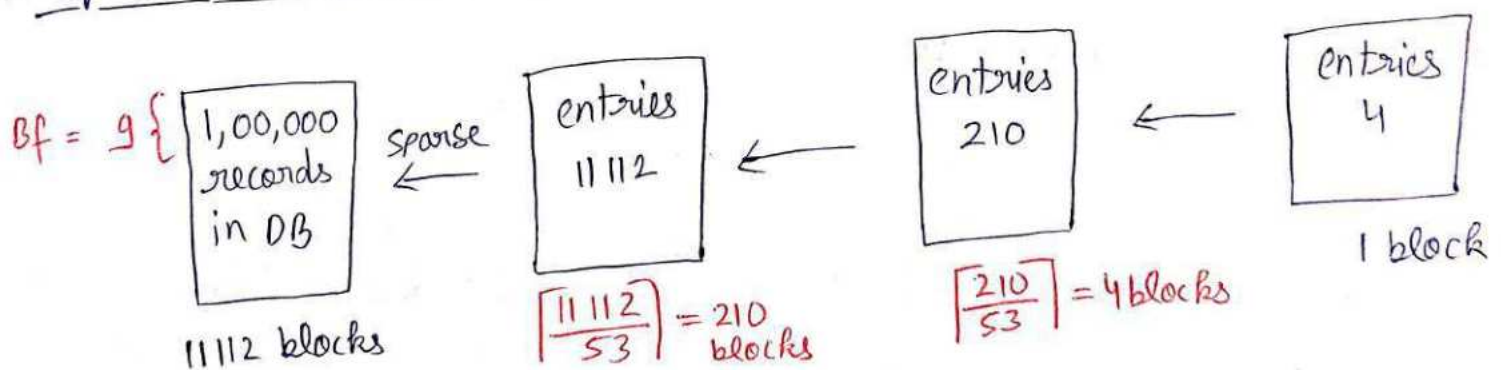
Soln :- * Dense Index at 1st level :-



$$\Rightarrow \text{Bf of index} = \left\lfloor \frac{B-H}{K+P} \right\rfloor = \left\lfloor \frac{1024-0}{9+10} \right\rfloor = 53 \text{ entries/block}$$

$$\Rightarrow \# \text{ of 1st level (Dense blocks)} = \left\lceil \frac{\# \text{ of entries}}{\text{Bf}} \right\rceil = \left\lceil \frac{1,00,000}{53} \right\rceil = 1887 \text{ blocks}$$

* Sparse Index at 1st level :-



$$\Rightarrow \text{Bf of DB} = \left\lfloor \frac{B-H}{R} \right\rfloor = \left\lfloor \frac{1024-0}{110} \right\rfloor = 9 \text{ records/block}$$

$$\Rightarrow \# \text{ of DB blocks} = \left\lceil \frac{1,00,000}{9} \right\rceil = 11,112 \text{ blocks}$$

$$\Rightarrow \# \text{ of sparse index blocks} = \left\lceil \frac{11,112}{53} \right\rceil = 210 \text{ blocks}$$

① a) $\lceil \log_2 n \rceil = \lceil \log_2(11112) \rceil = 14 \text{ blocks}$

b) $n: 11, 112 \text{ blocks.}$

$$\left\{ \begin{array}{c} \log_2 n \\ \text{on} \\ n \end{array} \right\} \quad \left\{ \lceil \log_2 m \rceil + 1 \right\} \quad \left\{ K+1 \right\}$$

at DB File at 1st level multi level Index

② a) 1887 blocks

b) $\lceil \log_2(1887) \rceil + 1 = 12 \text{ blocks}$

c) • Index blocks $\Rightarrow 1887 + 36 + 1 = 1924$

• Levels = 3

• I/O cost to access record $\Rightarrow K+1 = 4 \text{ blocks}$

③ a) 210 blocks

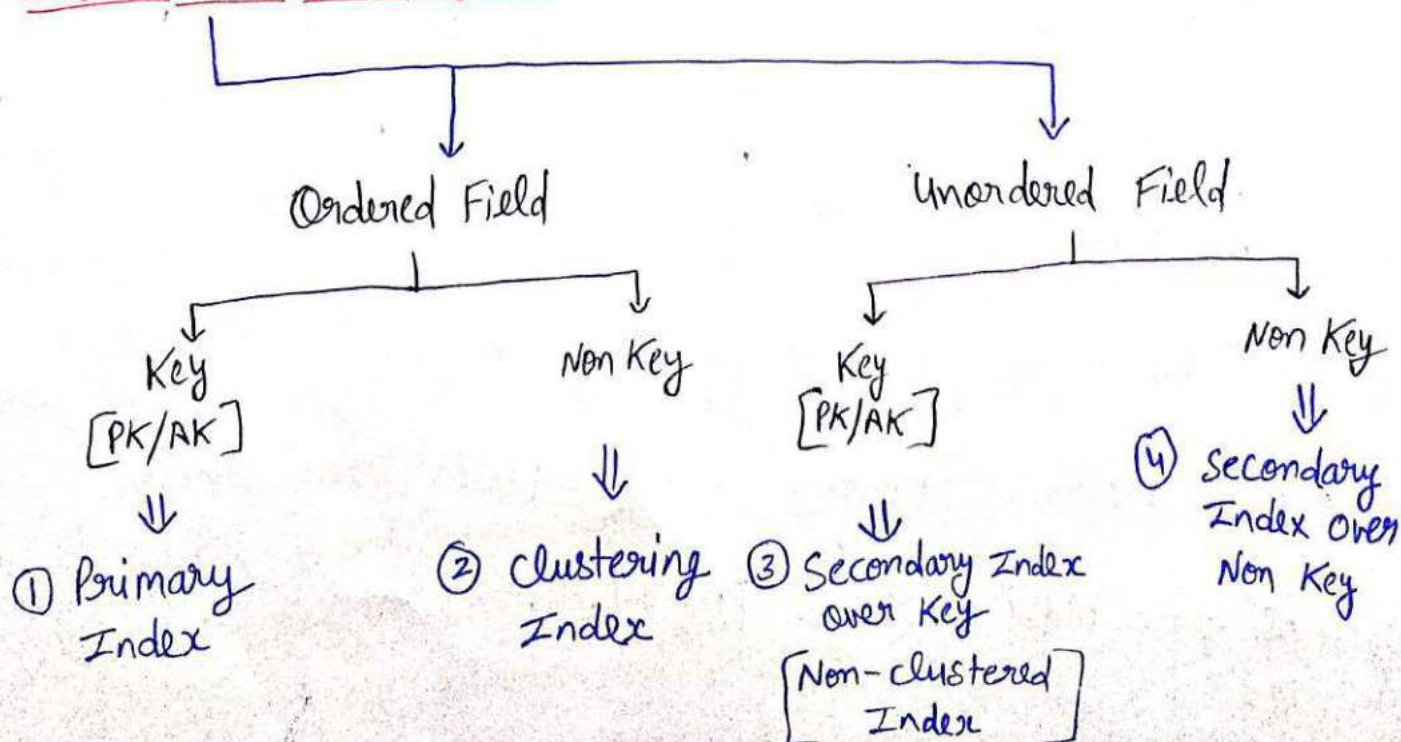
b) I/O cost $\Rightarrow \lceil \log_2 m \rceil + 1 = 9 \text{ blocks}$

c) 1) Index blocks = $210 + 4 + 1 = 215$

2) Levels of index = 3

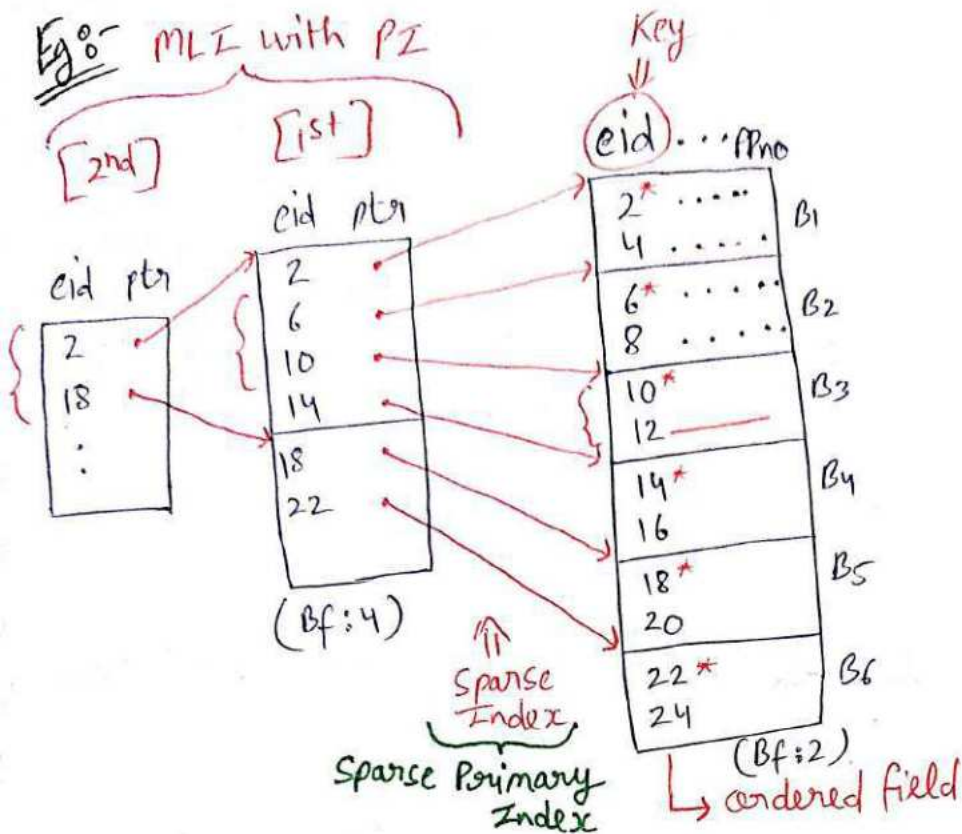
3) I/O cost to access record = $K+1 = 3+1 = 4 \text{ blocks}$

* Types of Indexing *



① Primary Index :- [ordered Field + Key]

Eg:- MLZ with PZ



⇒ I/O cost to access record by using

i) Only 1st level PZ :- $\lceil \log_2 m \rceil + 1$

ii) MLZ with PZ :- $K+1$ blocks

Select *
from Emp
where eid = 12;
PZ with MLZ exists

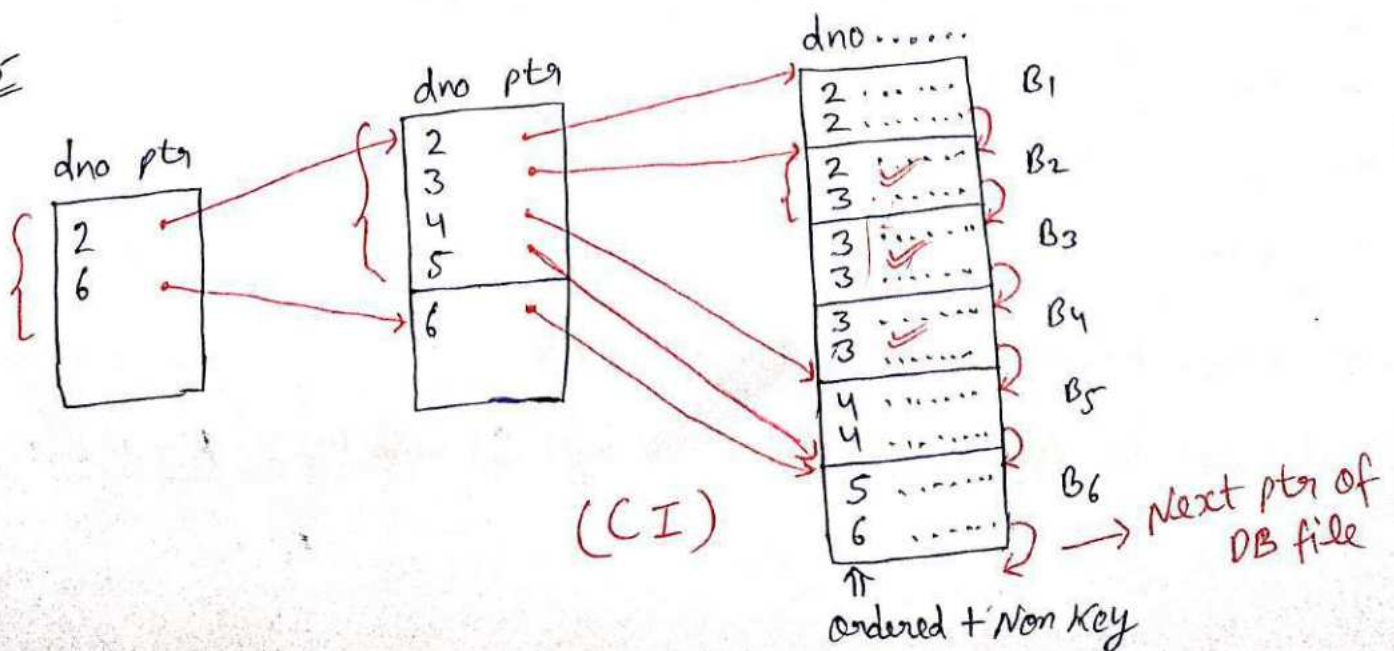
⇒ Primary index can be either dense or sparse but sparse PZ is preferred.

⇒ For any DB relation atmost 1 Primary index is possible. because search key must be ordered field.

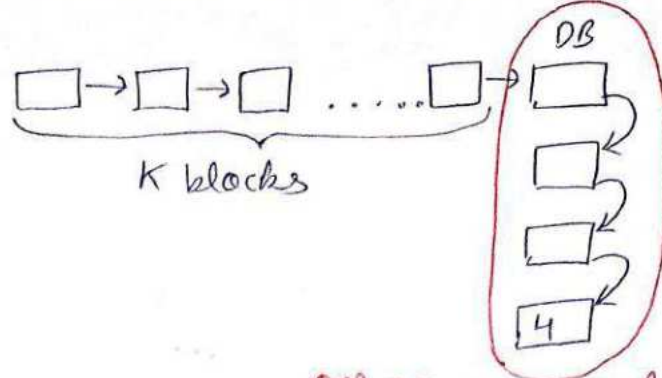
② Clustering Index :- [ordered Field + Non Key]

⇒ Search Key is ordered field and non key.

Eg:-



⇒ Select *
FROM Emp
where dno = 3;
MLI with CI



⇒ I/O cost to access cluster of records with MLI using CI

$\left\{ K + \text{one or more DB blocks until 1st record of next cluster} \right\}$

⇒ CI mostly sparse index... (Dense CI may occur if each cluster has one record).

⇒ At most one CI is possible for any DB table, because search key must be ordered field.

⇒ Either PI or CI is possible for any DB table but not both.

③ Secondary Index :-

⇒ Search key used for index :-

Unordered and (Key or Non-Key)

⇒ SI is alternative index even prime index or clustering index already exists.

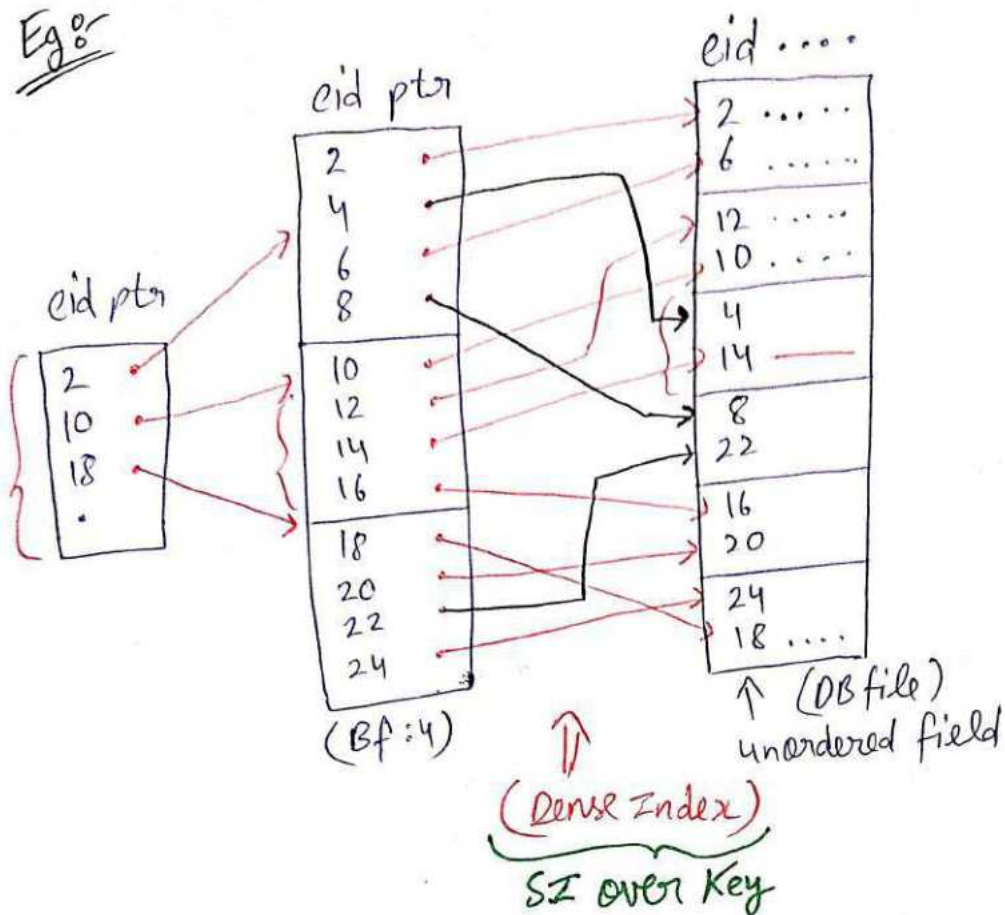
⇒ many SI's possible for DB files

① SI over Key :- [Non-clustering Index]

⇒ SI over Key is always Dense Index

⇒ I/O cost to access record by using SI with MLI $K+1$ blocks.

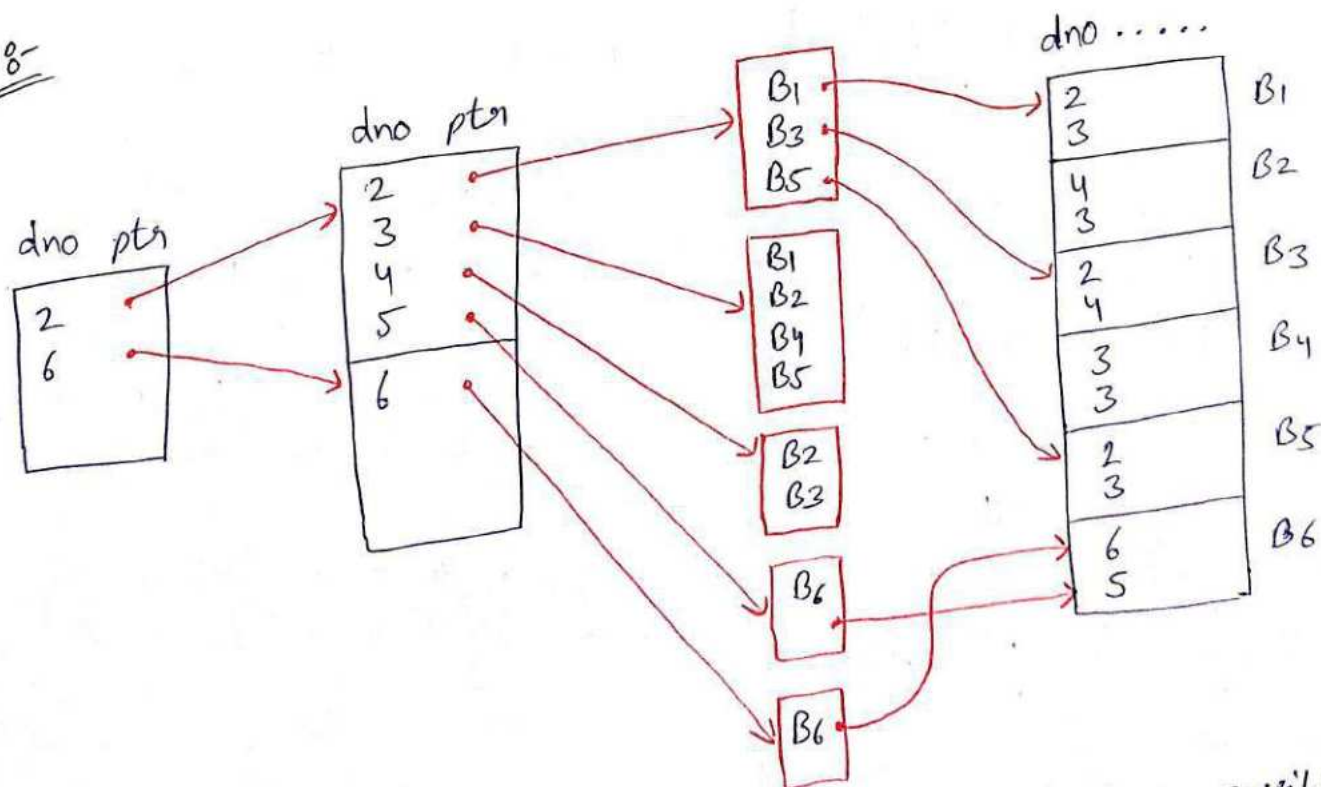
Eg:-



Dynaminc MLZ
 Standard DS
 B Tree } Index
 B+ Tree }

(ii) SZ over Non Key :-

Eg:-



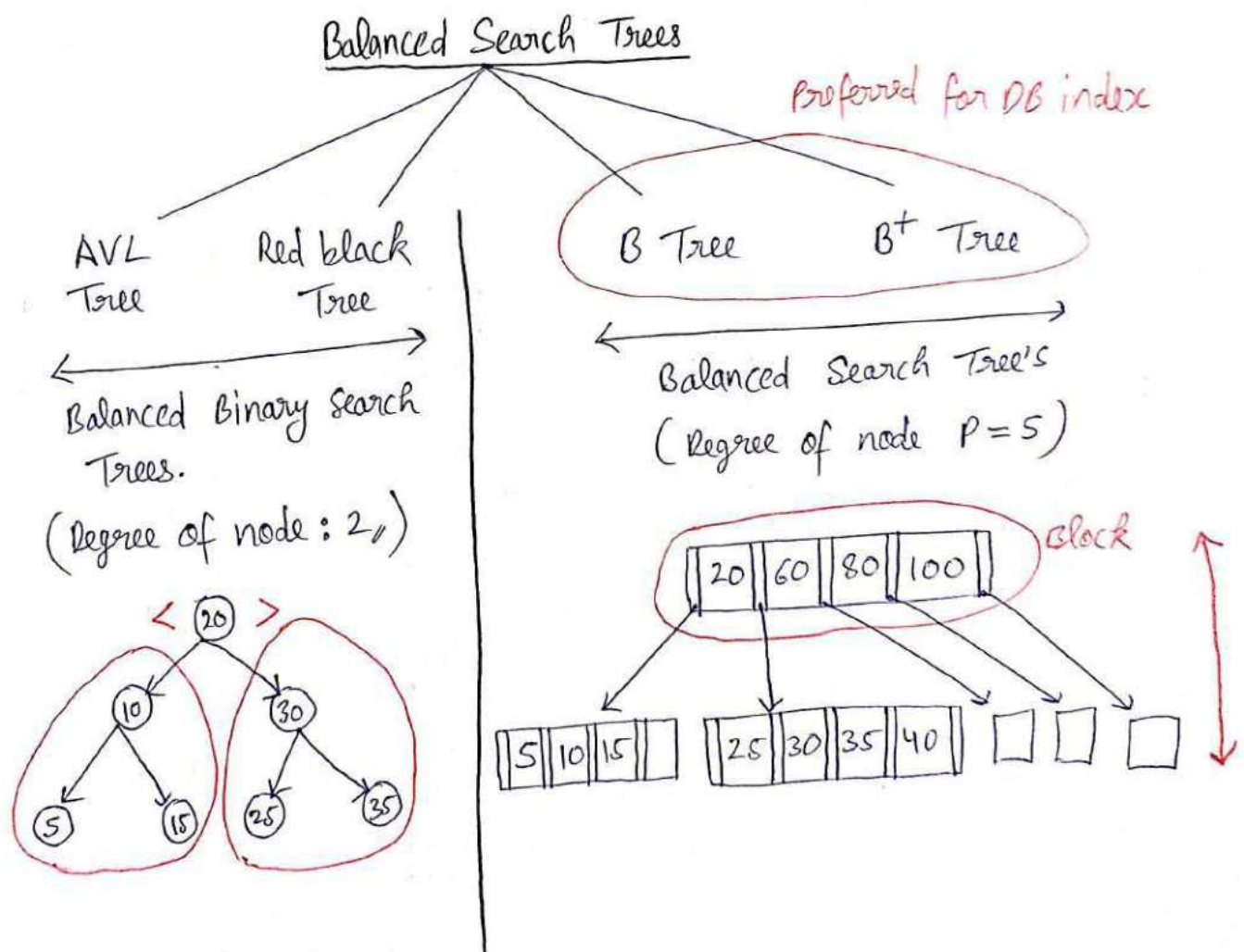
⇒ SZ over non key is mostly sparse index but dense may possible if each non Key value of the search Key is one record.

* Dynamic MLI :-

⇒ B/B⁺ Tree Index [Balanced Search Tree]

⇒ Balanced Search Tree [Height Restricted Search Tree] :-

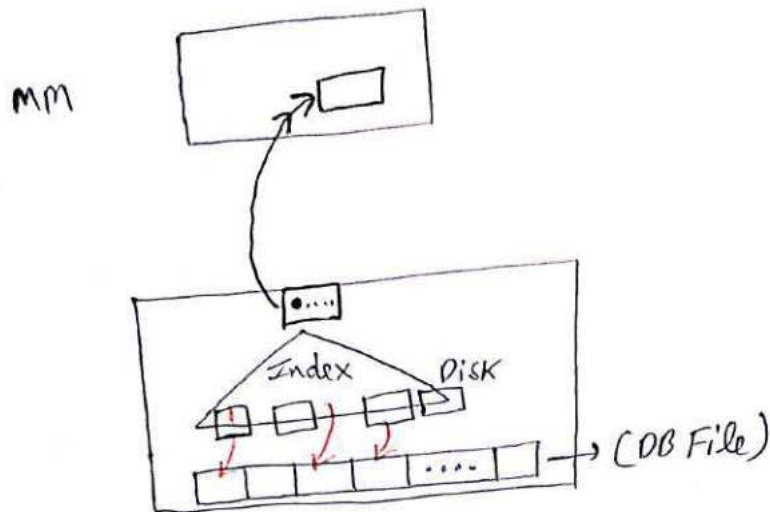
- * Max height of search tree for n distinct keys should not exceed $O(\log n)$
- * WC search cost to search key over balanced search of n distinct Key : $O(\log n)$
- * Best DS for search application.



Q) why B/B⁺ Tree is used for DB index rather than Balanced Binary Search Tree?

Soln

DB File in Disk & Index file also must be in Disk & Data access from Disk block by block.

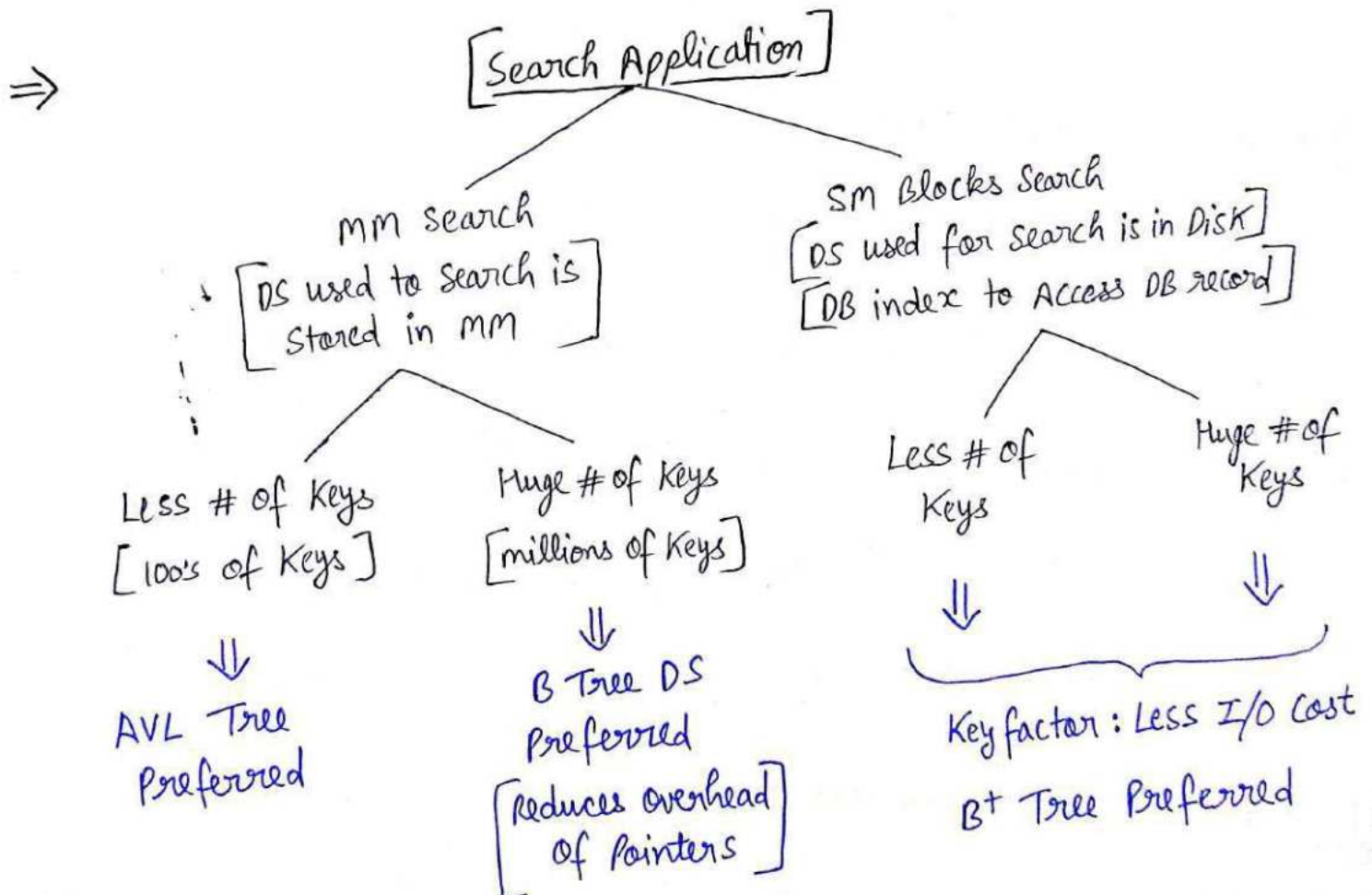


⇒ If AVL Tree used DB Index

- more levels of index
- more I/O cost

⇒ If B/B⁺ Tree used DB Index

- Less levels of Index
- Less I/O cost

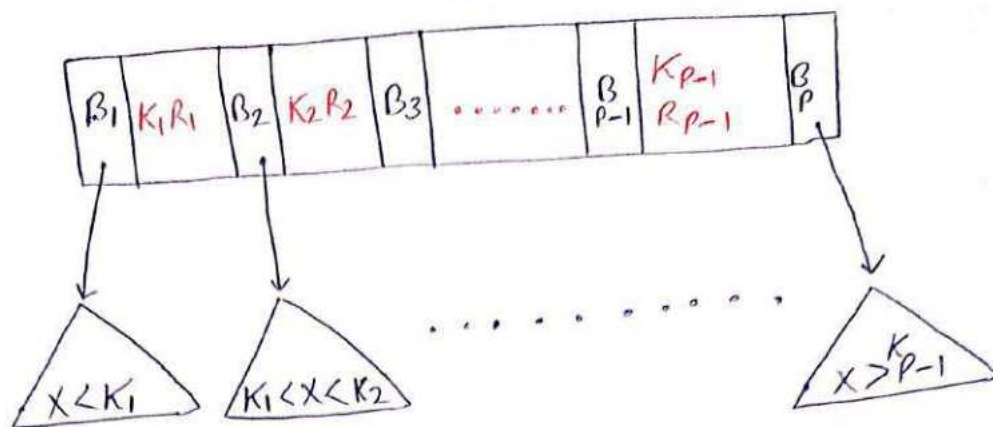


⇒ B Tree Definition :-

Order p :- max possible tree ptrs [block ptrs] which can store in B Tree node.

degree of node
order
fanout
All are same

a) Node Structure :- $\{ p \text{ block ptrs, } (p-1) \text{ Keys, } (p-1) \text{ Records ptrs} \}$



b) Root node can be with

min :- 2 block ptrs & 1 Key

max :- p block ptrs & $(p-1)$ Keys.

c) Internal nodes except Root

min :- $\lceil \frac{p}{2} \rceil$ block ptrs & $(\lceil \frac{p}{2} \rceil - 1)$ Keys

max :- p block ptrs & $(p-1)$ Keys.

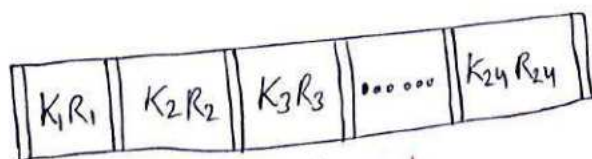
d) All leaf nodes must be at same level.

min :- $\lceil \frac{p}{2} \rceil - 1$ Keys } for each leaf node

max :- $p-1$ Keys }

Balancing
Conditions
of B Tree

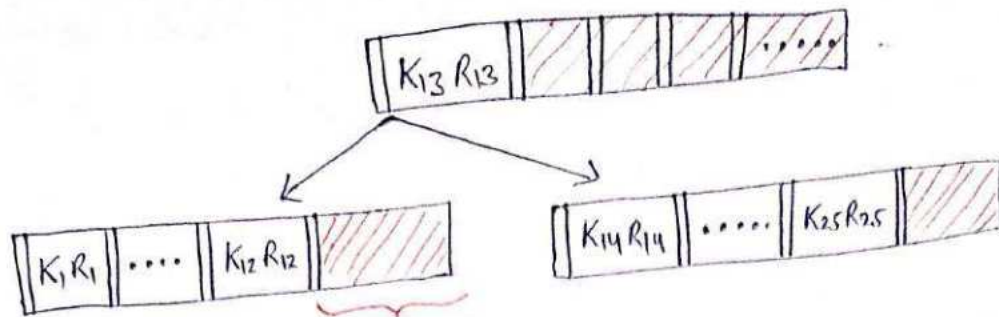
Eg:- $p=25$ [max block ptrs]



Insert: $[K_{25}R_{25}]$

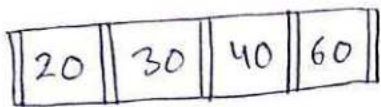
↓ Node
split

$K_1 < K_2 < \dots < \underbrace{K_{13}}_{\text{Root node}} < \dots < K_{25}$
Left child Right child



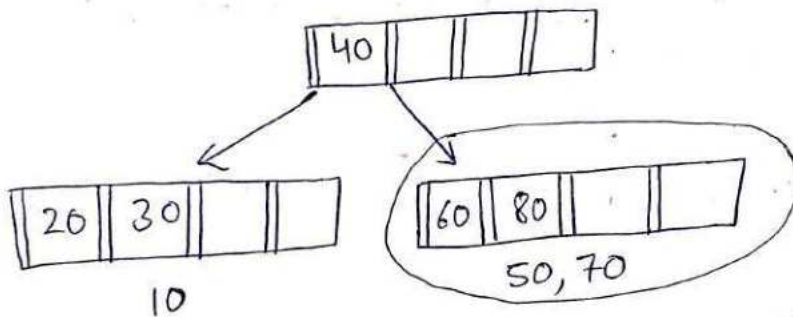
Q) Construct B Tree with order $p: 5$ [max possible tree ptrs per node] and sequence of Keys:-
 40, 60, 20, 30, 80, 50, 10, 70, 90, 15, 5, 25, 95, 85, 75, 45, 35,
 55, 65, 38, 48, 68, 78.

Soln



Insert: 80 [Node Split]

20 30 40 60 80

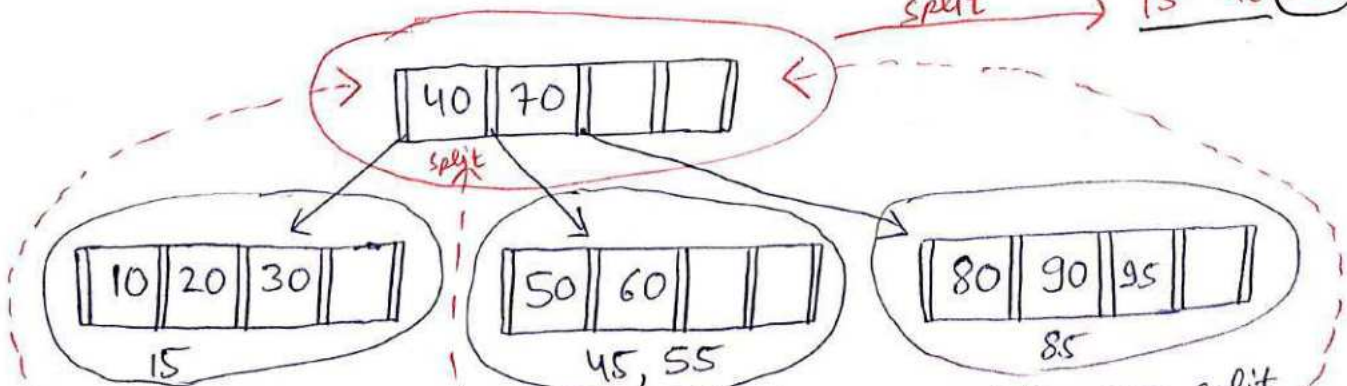


Ins: 90 [split Node]

50 60 70 80 90

split

15 40 55 70 85



Ins: 5 split

5 10 15 20 30

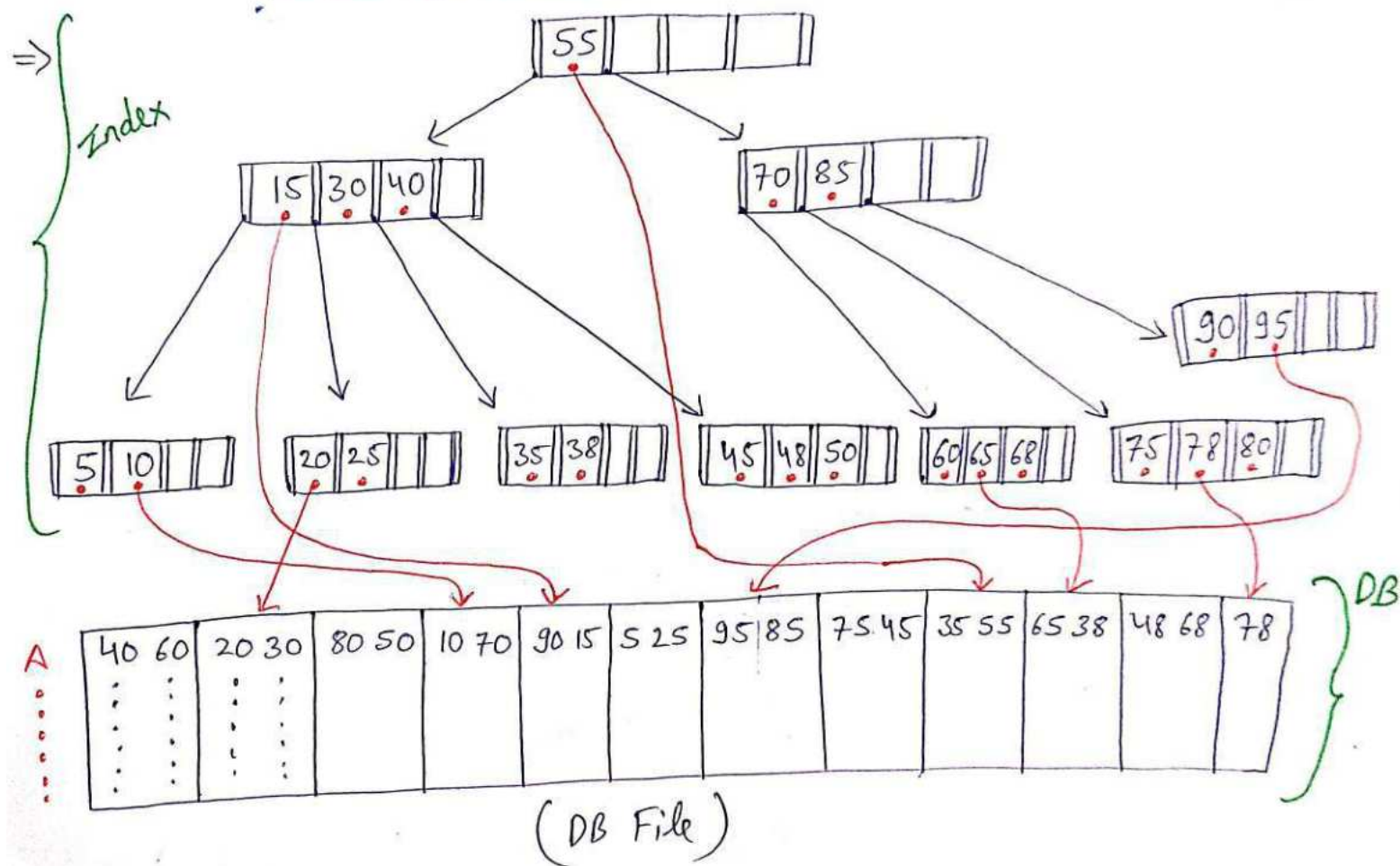
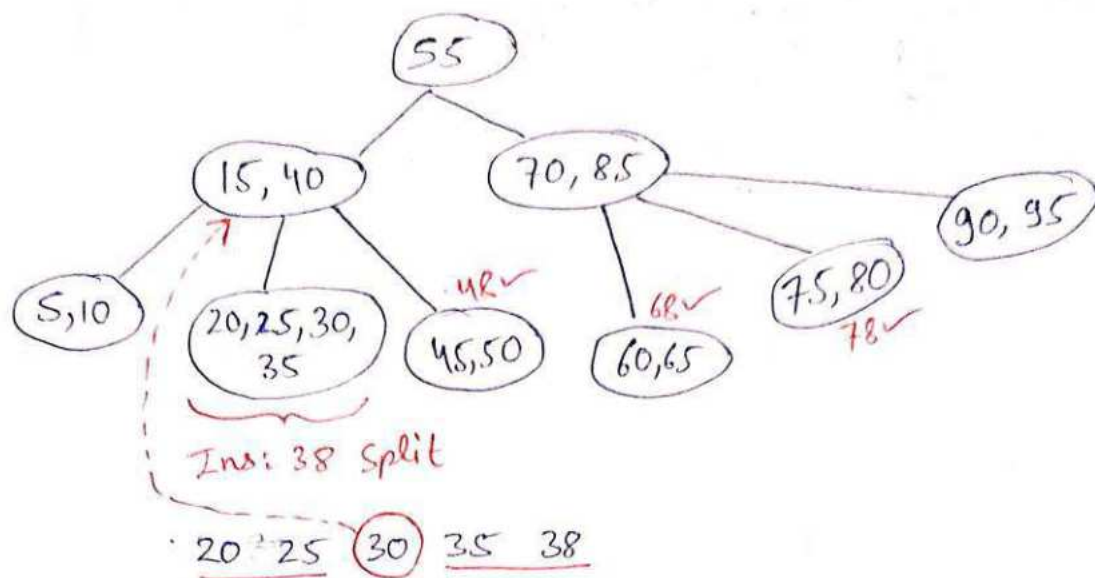
25, 35

Ins: 65 split

45 50 55 60 65

Ins: 75 split

75 80 85 90 95



⇒ Advantage :-

* B Tree index best suitable for Random Access of any one record.

Eg:-
 Select *
 From R
 where (A = 38); Any one

• WC I/O cost = (K+1) blocks

• BC I/O cost = 2 blocks

⇒ Disadvantage:-

* B Tree index not best suitable for Sequential Access of range of records.

Eg:- Select *
FROM R

where $A \geq 45$ and $A \leq 65$ Sequential Access

X records

• Estimated $\approx X(K+1)$ blocks
(WC)
 $\approx O(X \cdot K)$

NOTE:-

- * Search key used for index is ordered field then B/B⁺ tree index is sparse index, number of keys used for index is equal to number of blocks of DB file.
- * Search key used for index is unordered field then B/B⁺ tree index is Dense index, number of keys used for index is equal to number of records of DB file.

Q) Initially empty B tree, order P: 5 [max block ptrs] if Keys inserted in sequence of 1, 2, 3, 4, 5, 6, 7,

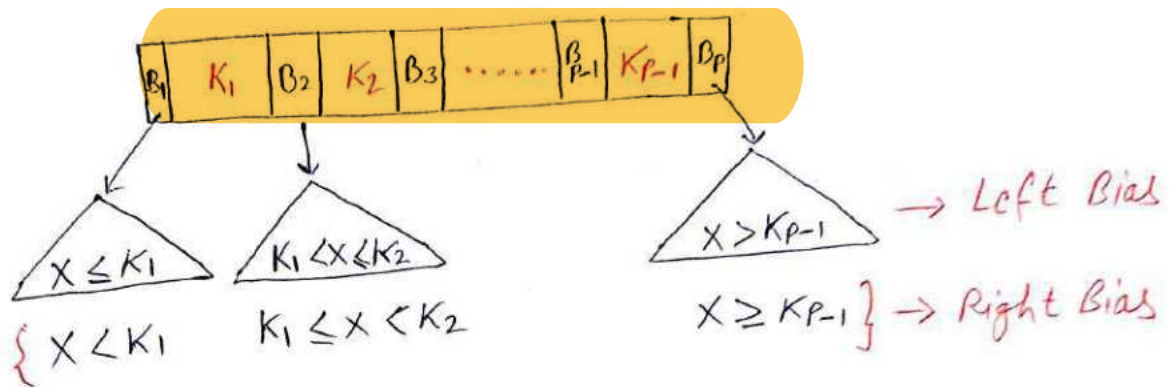
- i) which is the First Key insertion leads to level-3 B Tree?
- ii) which is the First Key insertion leads to level-4 B Tree?

⇒ B⁺ Tree :-

Order P: max possible pointers per node.

* Node Structure :-

a) Internal node :- [P block ptrs, (P-1) Keys]



b) Leaf node :- Set of (Key, R_p) pairs & one block pointer.
 (P-1) [R_p + K], 1 B_p



* Root :- min :- 2 bp & 1 Key max :- P bp's & (P-1) Keys

* Other Internal nodes :-

min :- $\lceil \frac{P}{2} \rceil$ bp's, $\lceil \frac{P}{2} \rceil - 1$ Keys

max :- P bp's & (P-1) Keys

* Leaf nodes :-

min :- $\lceil \frac{P}{2} \rceil - 1$ Keys, $\lceil \frac{P}{2} \rceil - 1$ R_p's

max :- (P-1) Keys, (P-1) R_p

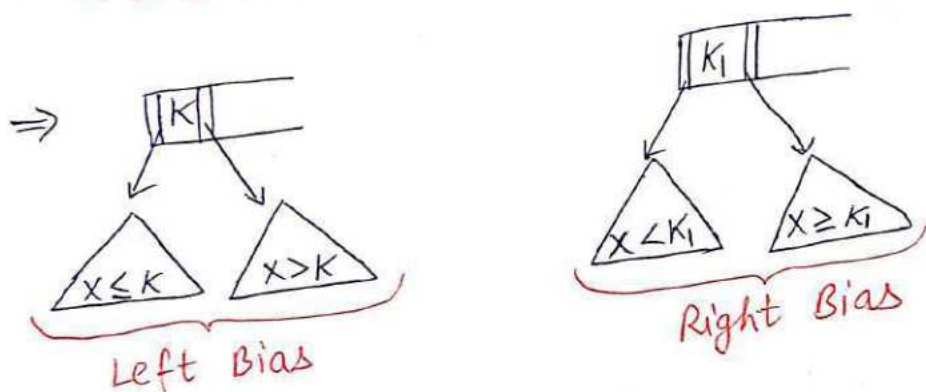
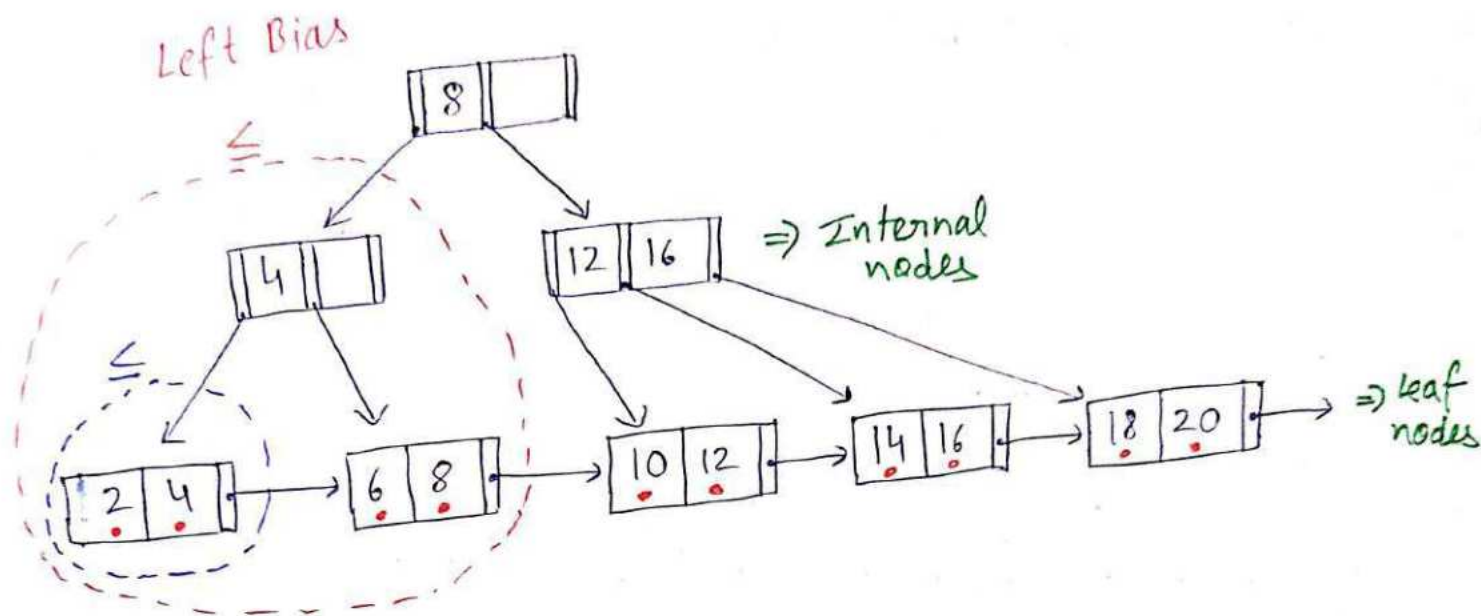
with 1 block pointer

⇒ all leaf nodes must be at same level.

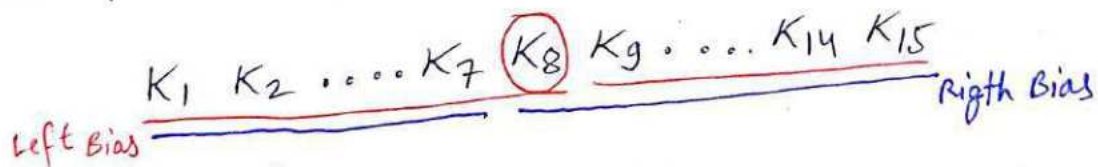
Node
Filling
Factor

[Balancing
Conditions]

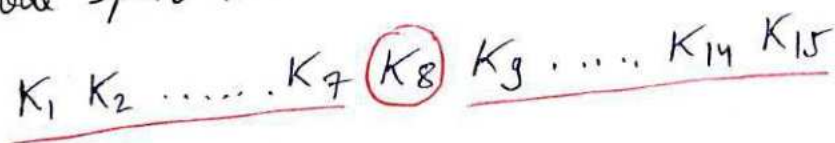
- ⇒ All Keys must be at leaf level
- ⇒ Every leaf node pointed to next leaf level.
- ⇒ All record pointers only at leaf level.
- ⇒ Node structure of Internal node & leaf node are not same.



⇒ Leaf Node Split :-

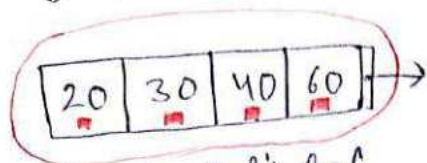


⇒ Internal Node Split :-



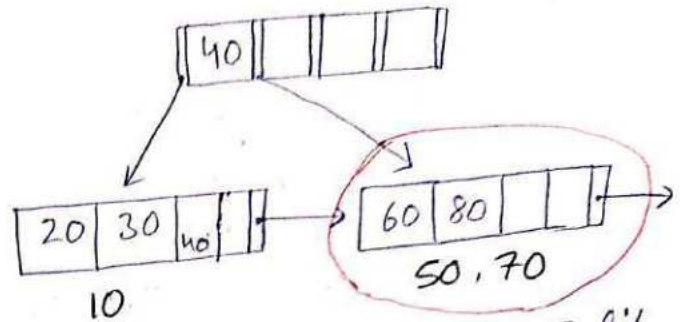
Q) Construct a B^+ tree with order: 5 [max ptrs per node] sequence of keys:- 40, 60, 20, 30, 80, 50, 10, 70, 90, 15, 5, 25, 95, 85, 75, 45, 35, 55, 65, 38, 48, 68, 78.

Capm $\Rightarrow$ using left bias.



Ins: 80 Split leaf

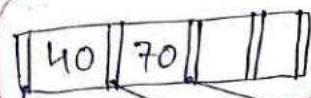
20 30 40 60 80



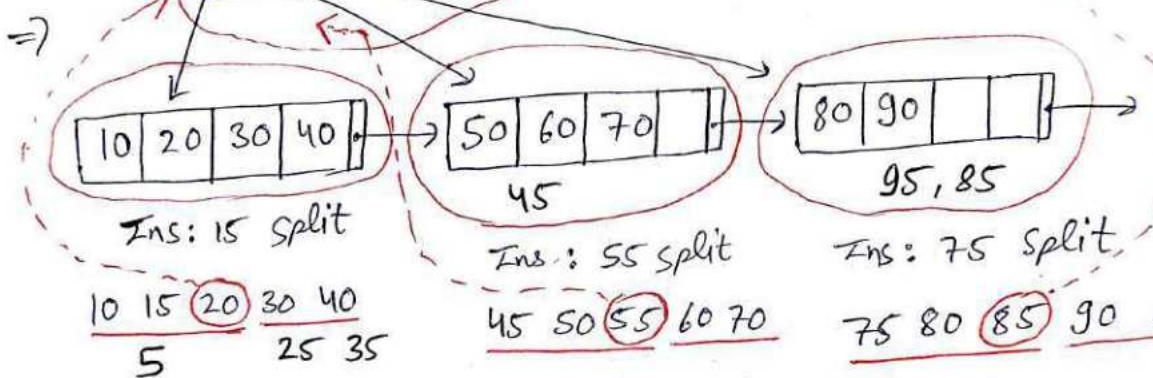
Ins: 90 Split

50 60 70 80 90

Ins: 55 Split



20 40 55 70 85



Ins: 15 Split

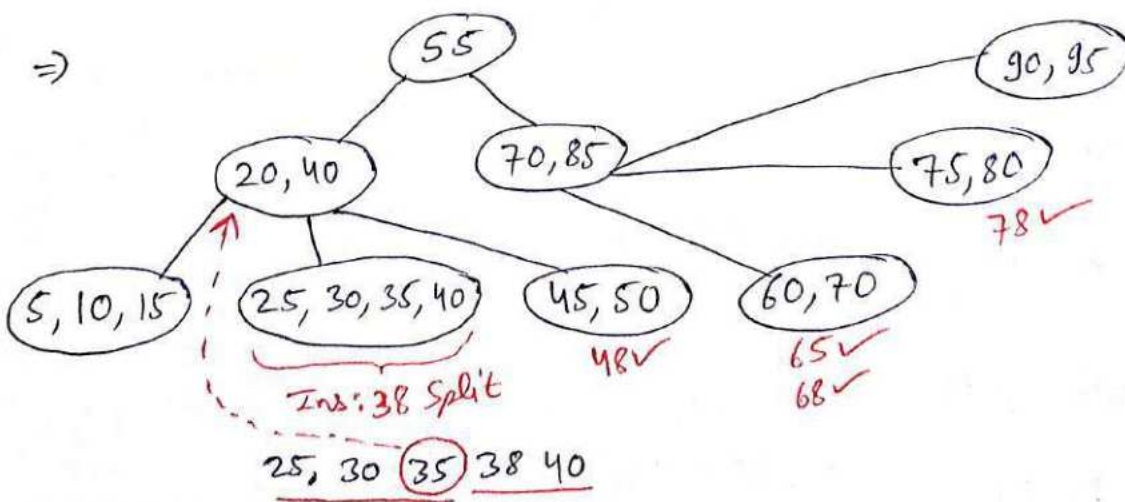
10 15 20 30 40
5 25 35

Ins: 55 Split

45 50 55 60 70

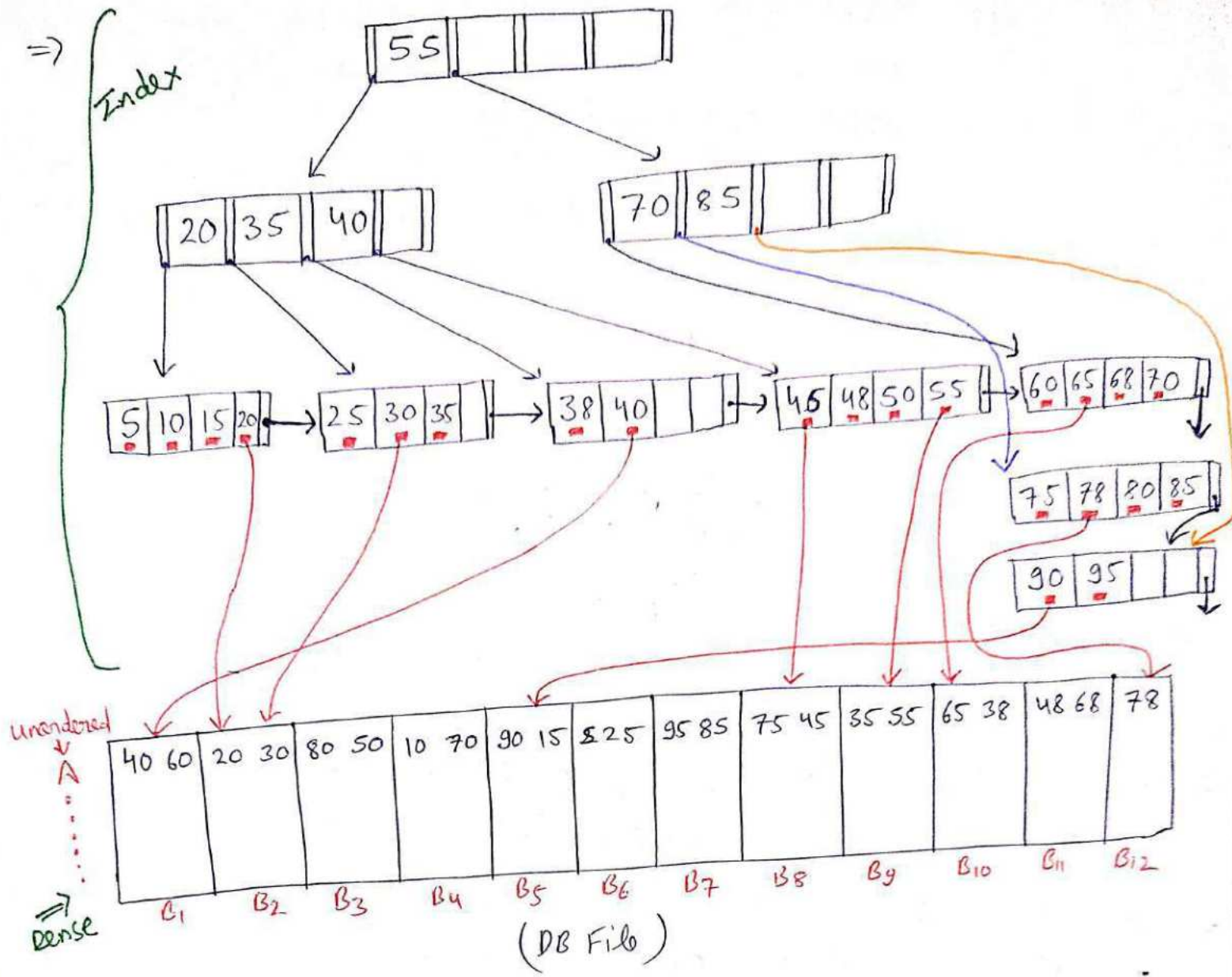
Ins: 75 Split

75 80 85 90 95



Ins: 38 Split

25, 30 35 38 40



⇒ Advantage :-

① B⁺ tree index best suitable for Random access of any one record with I/O cost to access record (K+1) blocks.

Eg:- Select *
From R
where A=55;

• I/O cost to Access record :- (K+1) blocks.

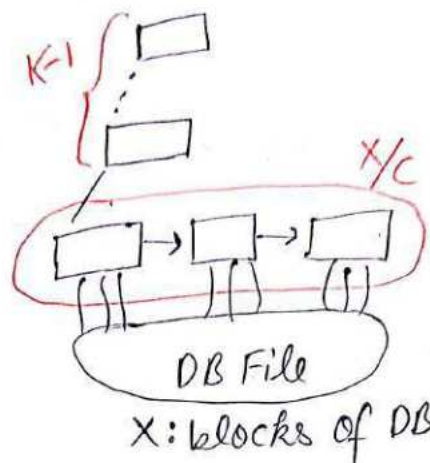
② B⁺ tree index is also best suitable for Sequential access of range of records.

Eg:- Select *
FROM R

where $A \geq 45$ and $A \leq 85$;

sequential access of range of records

I/O cost $\Rightarrow (K-1) + X/C + \frac{X}{L} \approx O(K+X)$
 $\xrightarrow{\text{records}}$



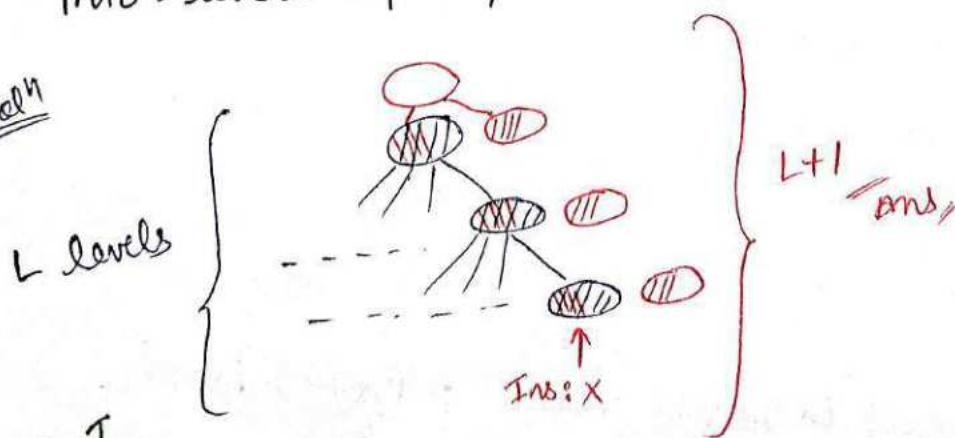
NOTE:-

$\Rightarrow$ For left biasing of B^+ tree, keys of internal nodes are maximum key of each leaf node except last leaf node.

$\Rightarrow$ For right biasing of B^+ tree, keys of internal nodes are minimum key of each leaf node except first leaf node.

Q) How many max new nodes required to create to insert 1 key into 'L' levels of B/B⁺ tree?

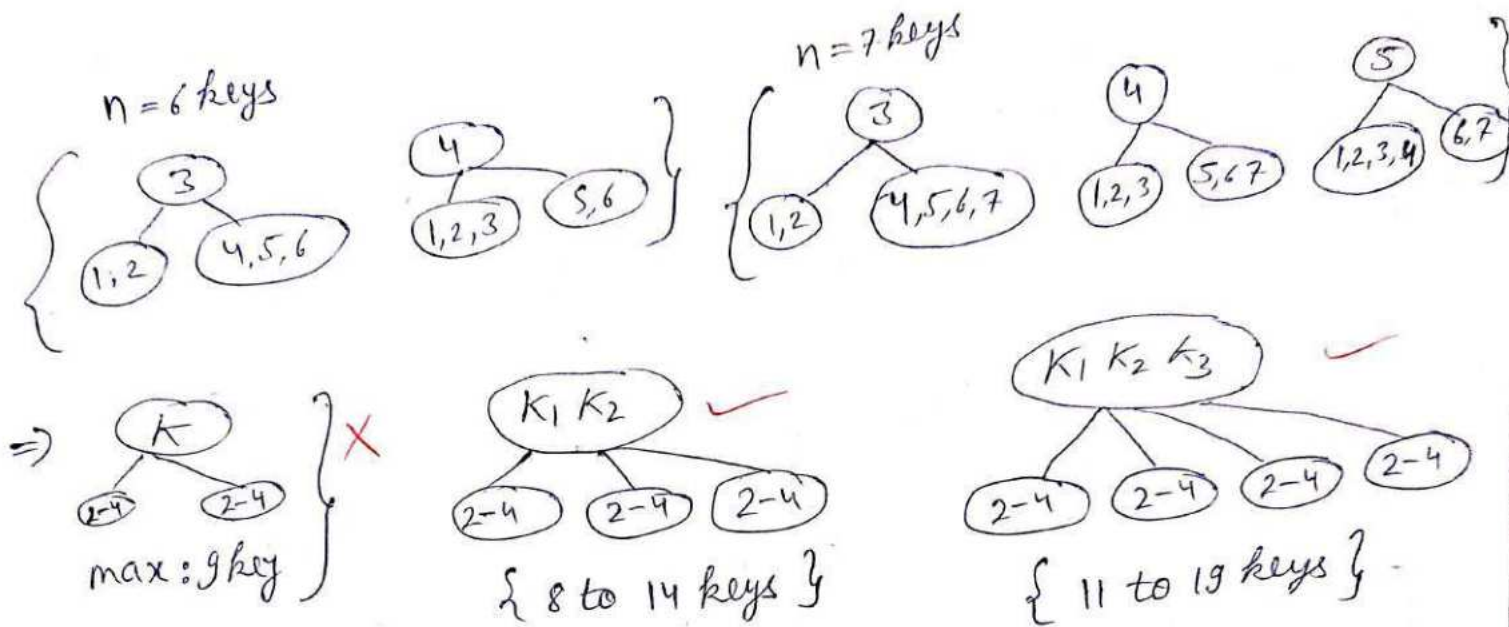
Soln



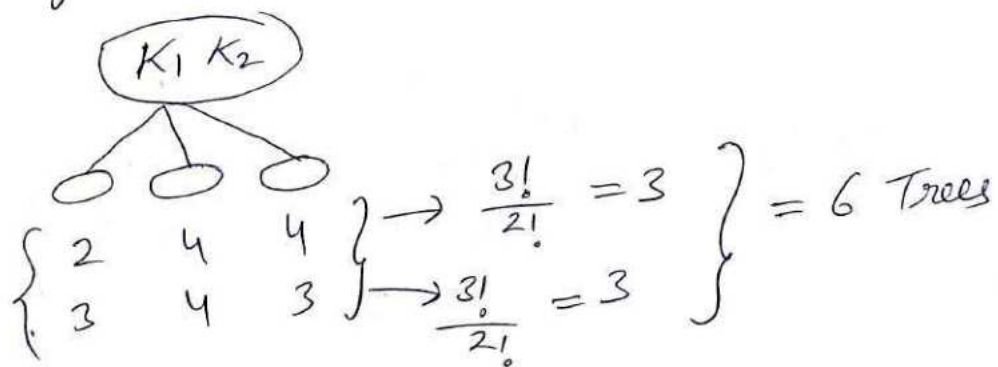
Q) $M-1$ Find number of possible B/B⁺ tree for given n keys and order P given? $\Rightarrow$

① How many possible B Trees for 12 distinct keys with order p is 5 (max bp's per node) ?

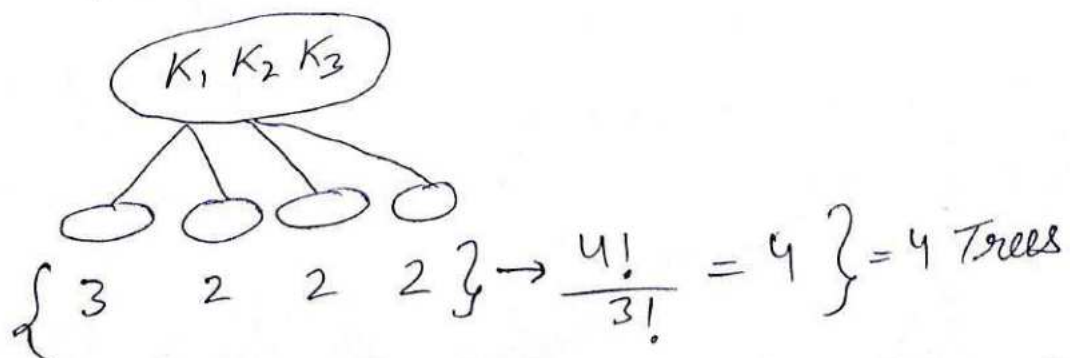
Soln Except Root $\min: \lceil \frac{p}{2} \rceil - 1 = 2$ Keys
Root min 1 key



Case ① : 2 Keys in Root



Case ② : 3 Keys in Root

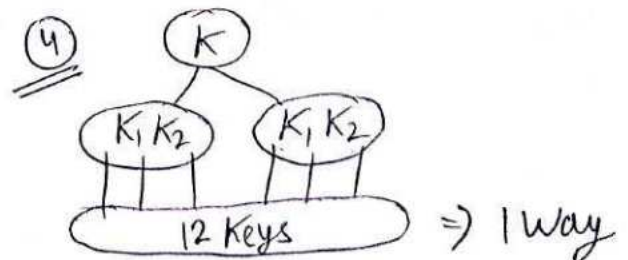
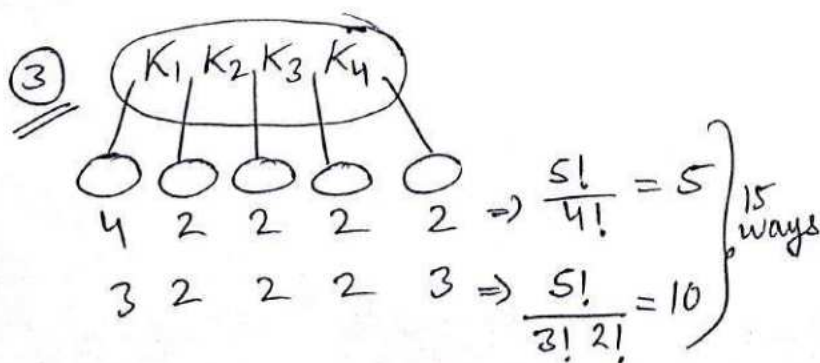
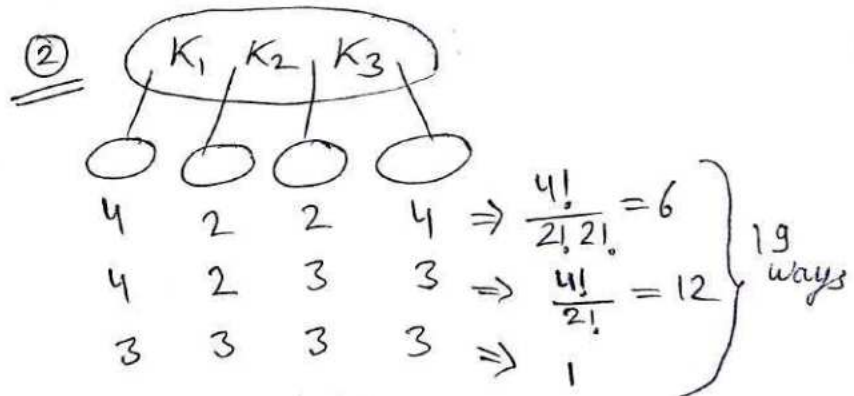
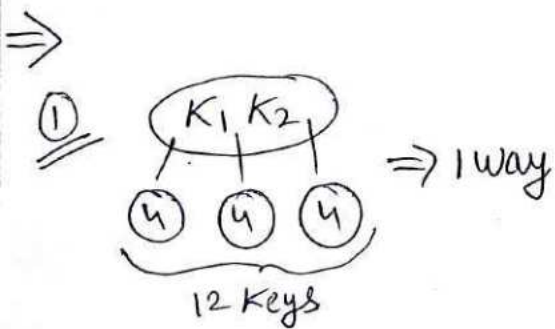
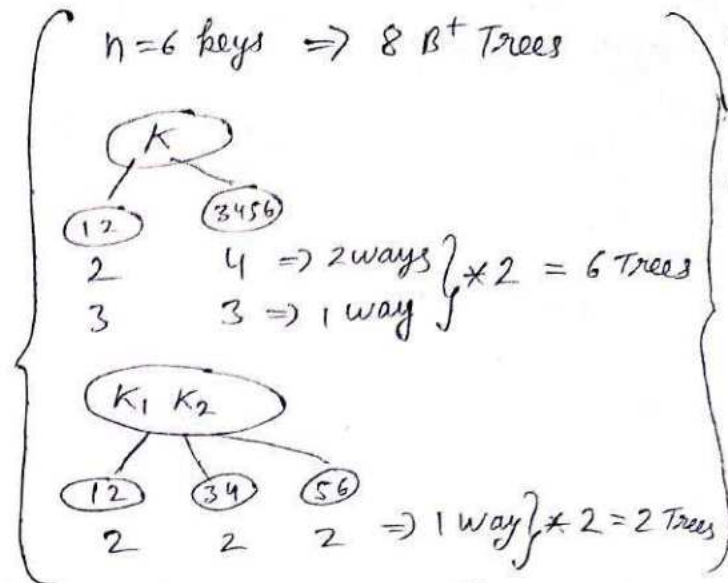
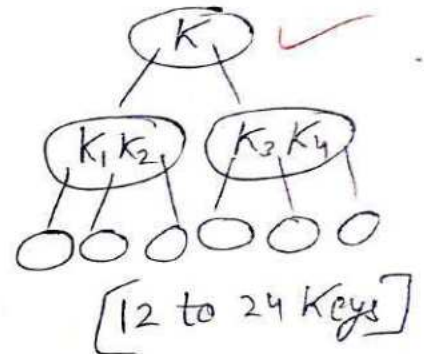
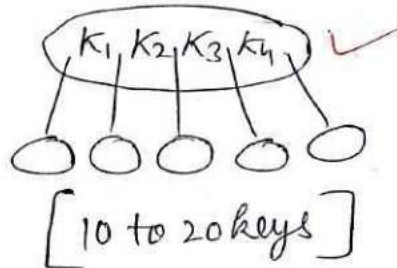
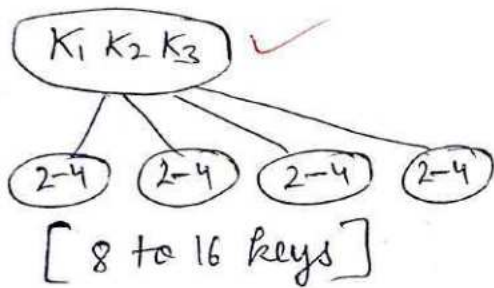
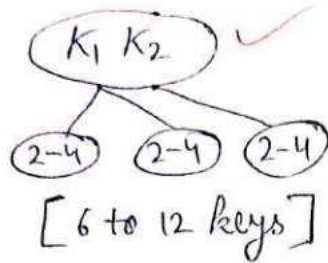
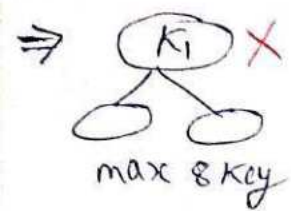


Ans $\Rightarrow$ 10 B Trees //

② How many B⁺ Trees possible for 12 distinct keys & order P:5 (max ptrs per node).

Soln

Root $\Rightarrow$ 1 to 4 keys
Other $\Rightarrow$ 2 to 4 keys



$\Rightarrow 1 + 19 + 15 + 1 \Rightarrow 36 \times 2 = 72$ B⁺ Trees // ans.

$\uparrow$
for left bias tree
&
right bias tree

M-II $\Rightarrow$ Compute best possible order of B/B⁺ Tree Node :-

Q) Block size : 2048 bytes } = B/B⁺ Tree node size

Search key : 12 bytes

Block ptr : 15 bytes

Record ptr : 10 bytes

1) Order P: max possible block pointer in B Tree node,
 what is best possible order of B Tree node?

Soln

Block :

B Tree
 node

$$P \times B_p + (P-1)[K + R_p] \leq \text{Block Size}$$

$$P \times 15 + (P-1)[12 + 10] \leq 2048$$

order P:

[max Bp/node]

$$37P \leq 2070$$

$$P = \left\lfloor \frac{2070}{37} \right\rfloor = 55 // \text{Ans}$$

ii) Order P: max pointer in B⁺ tree node
 what is best possible order of B⁺ tree

a) Internal Node?

Soln

P : max ptrs

$\downarrow$
 P bp's

$$P \times B_p + (P-1) \text{Key} \leq \text{Block Size}$$

$$P \times 15 + (P-1) 12 \leq 2048$$

$$27P \leq 2060$$

$$P = \left\lfloor \frac{2060}{27} \right\rfloor$$

$$= 76 // \text{Ans}$$

b) Leaf Node?

Soln

P : max ptrs

(P-1) R_p 1 B_p

$$(P-1)[K + R_p] + 1 B_p \leq \text{Block Size}$$

$$(P-1)[12 + 10] + 15 \leq 2048$$

$$22P \leq 2055$$

$$P = \left\lfloor \frac{2055}{22} \right\rfloor$$

$$= 93 // \text{Ans}$$

NOTE:

$\Rightarrow$

If $\text{Size}(B_p) = \text{Size}(R_p)$ then

$$\left\{ \begin{array}{l} B^+ \text{ Tree internal} \\ \text{node order}(P) \end{array} \right\} = \left\{ \begin{array}{l} B^+ \text{ Tree of leaf} \\ \text{node order}(P) \end{array} \right\}$$

Q) Block size = 1024 bytes

Search Key = 7 bytes

$B_p = R_p = 9$ bytes

Order: P , max possible keys in B/B^+ tree node.

what is best possible order of $\left[P = \# \text{ of max pointer} = P+1 \right]$

i) B Tree node?

Soln

P : max keys per node

$$\boxed{(P+1)B_p + P[K + R_p]} \leq \text{Block Size}$$

$$(P+1)9 + P[7+9] \leq 1024$$

$$25P \leq 1015$$

$$P = \left\lfloor \frac{1015}{25} \right\rfloor = 40 // \text{ans}$$

ii) B^+ Tree Node?

Soln

For internal node

$$\boxed{(P+1)B_p + P * \text{Keys}} \leq \text{Block size}$$

$$(P+1)9 + P * 7 \leq 1024$$

$$16P \leq 1015$$

$$P = \left\lfloor \frac{1015}{16} \right\rfloor = 63 // \text{ans.}$$

For Leaf Node:-

$$\boxed{P[K + R_p] + 1 B_p} \leq \text{Block size}$$

$$\text{Ans} = 63 //$$

NOTE:-

$\Rightarrow$ If block size of B Tree Node = block size of B⁺ Tree Node

Then ① Order P of B Tree < Order P of B⁺ Tree

② # of B Tree nodes for n distinct keys > # of B⁺ Tree nodes for n distinct keys

③ # of levels of B Tree index for n distinct keys (I/O cost) > # of levels of B⁺ tree index for n distinct keys (I/O cost)

[B⁺ Tree preferred for DB index]

$\Rightarrow$ If order P of B Tree = Order P of B⁺ Tree then

① # of B Tree nodes for n distinct keys $\leq$ # of B⁺ Tree nodes for n distinct keys

② # of levels of B Tree index for n distinct keys $\leq$ # of levels of B⁺ Tree index for n distinct keys.

M-III

$\Rightarrow$ min/max possible keys in B/B⁺ Tree for given L levels (height) and order P.

Q1) How many max possible keys in B/B⁺ Tree of level 5 and order P: 7 (max bp's per node).

max 7bp's, 6 Keys

Soln

level	max nodes	max bp's	max keys
1	1	7	6
2	7	7*7	7*6
3	49	49*7	49*6
4	343	343*7	343*6
5	2401	2401*7	2401*6

$\rightarrow$ 2801 max nodes in B/B⁺ Tree

$\rightarrow$ 16,806 max keys in B Tree

$\rightarrow$ 14,406 max keys in B⁺ Tree

Q2) How many min possible keys in B/B⁺ Tree of level 5 and order P: 7 (max bps per node)?

Soln

Root $\Rightarrow$ min 2 bps & 1 Key

Other $\Rightarrow$ min $\lceil \frac{P}{2} \rceil = 4$ plus 1 3 keys

Level	min nodes	min bps	min Keys
(Root) 1	1	2	1
2	2	2 x 4	2 x 3
3	8	8 x 4	8 x 3
4	32	32 x 4	32 x 3
5	128	—	128 x 3

171 min nodes in B/B⁺ Tree

min 511 keys in B Tree

min 384 keys in B⁺ Tree

$$\left\{ \begin{array}{l} \text{\# of Keys in internal nodes} \\ \text{of B/B}^+ \text{ Tree} \end{array} \right\} = \left\{ \begin{array}{l} \text{\# of leaf nodes} - 1 \end{array} \right\}$$

Q3) Order P: b/w 1 to 2P keys for Root
b/w P to 2P keys for other nodes.

How many max/min keys in B/B⁺ Tree of order P=4 & levels 4?

Soln

max	Root	other
bp	9	9
Keys	2P:8	8:2P

min	Root	Other
bp	2	5
Keys	1	P:4

Level	max nodes	max bp	max Keys	min nodes	min bp	min Keys
(Root) 1	1	9	8	1	2	1
2	9	9×9	9×8	2	2×5	2×4
3	81	81×9	81×8	10	10×5	10×4
4	729	729×9	729×8	50	50×5	50×4

820 Nodes in B/B⁺ Tree
 max 6560 keys in B Tree
 5832 max keys in B⁺ Tree
 63 Nodes in B/B⁺ Tree
 249 min keys in B Tree
 200 min keys in B⁺ Tree

* Bulk Loading B⁺ Tree :-

⇒ [Bottom-up Approach]

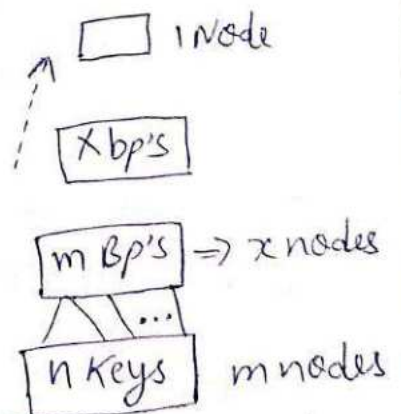
- 1) Sort keys which are used for B⁺ Tree construction.
- 2) Design Leaf Nodes :- Distribute given n distinct keys in leaf nodes. // # of leaf nodes in result
- 3) Design Internal Nodes :-

If m nodes at level l then m block ptrs required l-1 level.

Distribute m Bp's into nodes.

// # of nodes at level (l-1) in result

• Repeat until Root



Q1) # of keys given find min/max levels
min/max nodes of B⁺ Tree?

Q2) # of leaf nodes given find min/max internal nodes of B⁺ Tree?

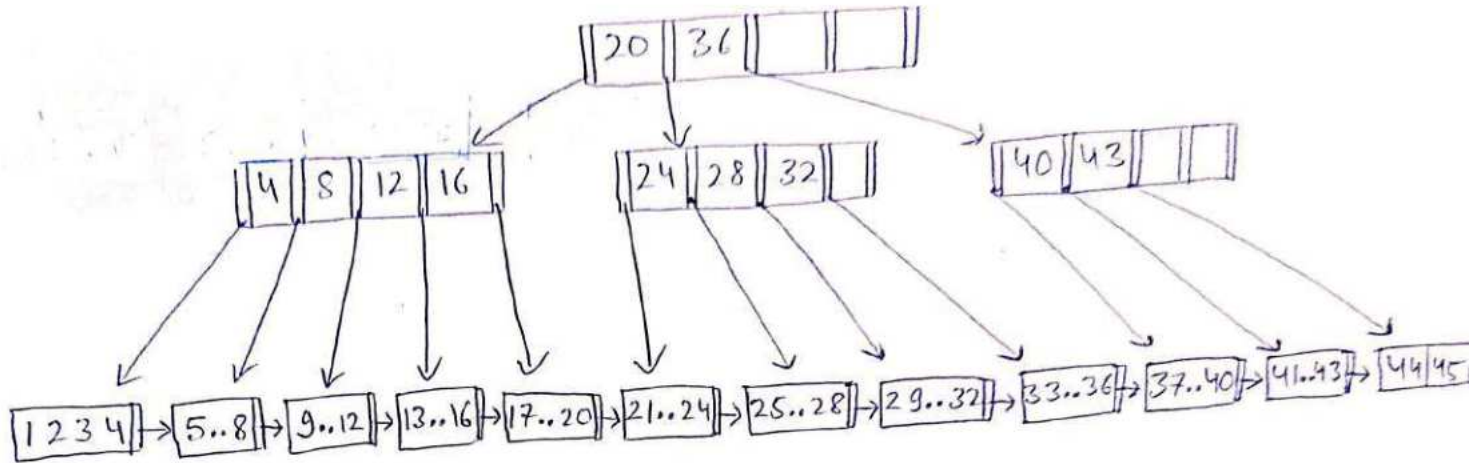
Use Bulk loading B⁺ Tree.

Q) Given keys : 1, 2, 3, 4, ..., 45

Order $P: 5$ (max ptrs per node), How many min levels of B^+ Tree (min nodes)?

Soln
min levels: Nodes filled as most as possible.

max: 5 ptrs & 4 keys / node



$\Rightarrow 3 \text{ bp's} = 1 \text{ node}$

$12 \text{ bp's} \Rightarrow \left\lceil \frac{12}{5} \right\rceil = 3 \text{ nodes}$
 $\hookrightarrow \text{max ptrs per node}$

$\left\lceil \frac{45}{4} \right\rceil = 12 \text{ nodes}$
 $\hookrightarrow \text{max keys per node}$

$\Rightarrow \text{min levels: } 3$

$\Rightarrow \text{min nodes: } 12 + 1 + 3$
 $= 16 \text{ nodes} // \text{ans}$

Q) Given keys : 1, 2, 3, ..., 45, order $P: 5$ (max ptrs per node). How many max levels of B^+ Tree? (max nodes).

Soln

min	Root	other
Bp	2	$\left\lceil \frac{P}{2} \right\rceil = 3$
keys	1	2

Internal nodes

2 bp's	1 node
7 bp's	$\left\lceil \frac{7}{3} \right\rceil = 2 \text{ nodes}$
22 bp's	$\left\lceil \frac{22}{3} \right\rceil = 7 \text{ nodes}$

leaf nodes: 45 keys $\left\lceil \frac{45}{2} \right\rceil = 22 \text{ nodes}$
 $\hookrightarrow \text{min keys/node}$

$\Rightarrow \text{max: } 4 \text{ levels}$

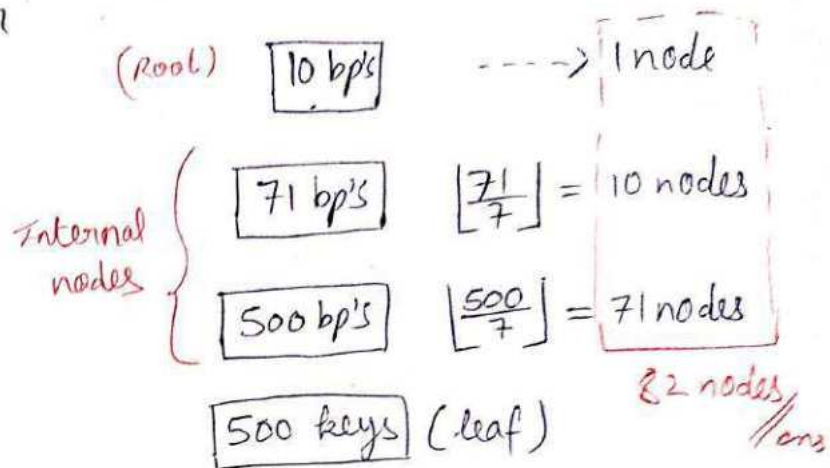
$\Rightarrow \text{max: } 32 \text{ nodes} // \text{ans.}$

Q) Order P: b/w 1 to 2P keys for root
b/w p to 2P keys for other nodes.

i) How many max internal nodes of B⁺ Tree with order P: 6
& 500 leaf nodes?

Soln max: min filling factor

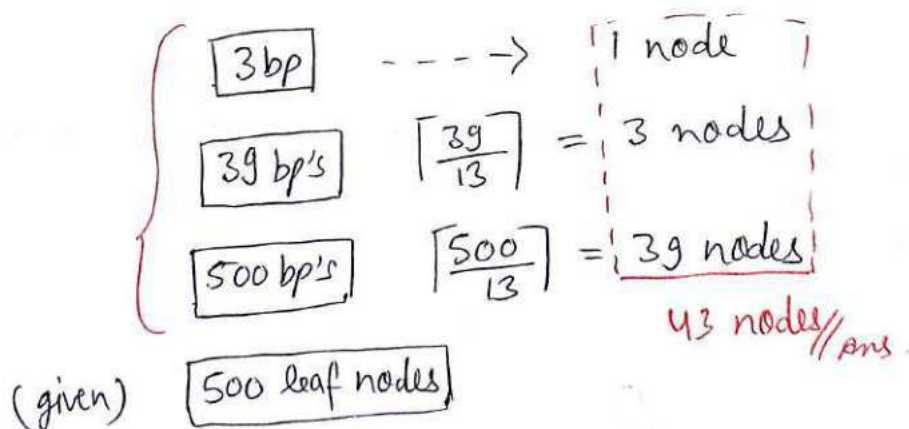
min	Root	Other
Bp	2	7
Keys	1	P=6



ii) How many min possible internal nodes of B⁺ Tree?

Soln min: max filling factor

max: 2P = 12 keys & 13 ptrs per node



Q) DB file consist 20,000 records and size of record: 500 bytes,
block size: 2048 bytes, search key: 20 bytes, pointer: 10 bytes.
If dense B⁺ Tree index build over DB file with nodes filled
as most as possible, How many levels of index required?

Soln

Order P:

$$P \times bp + (P-1)K \leq \text{block size}$$

$$P \times 10 + (P-1) 20 \leq 2048$$

$$30P \leq 2068$$

$$P = \left\lfloor \frac{2068}{30} \right\rfloor = 68$$

(max 68 p's
67 key per node)

5 bp's $\longrightarrow$ 1 node

299 bp's $\left\lceil \frac{299}{68} \right\rceil = 5 \text{ nodes}$

Leaf nodes

20000 keys $\left\lceil \frac{20000}{67} \right\rceil = 299 \text{ nodes}$

$\updownarrow$ dense
20,000 records } DB File

Ans = 3 levels //

Q) How many min levels of B Tree for 1000 keys with order P:15 (max bp's per node).

Soln min levels: Nodes fill as most as possible

level	max nodes	max bp	max keys
1	1	15	14
2	15	15 * 15	15 * 14
3	225	225 * 15	225 * 14

224 max keys for 2 levels
3374 max keys for 3 levels

$\Rightarrow$ min 3 levels for 1000 given keys //

Q) How many max levels of BTree for 1000 keys with $P:15$ (max bp's per node).

Solⁿ

max levels : Nodes fill as least as possible

$$\Rightarrow \min: \left\lceil \frac{P}{2} \right\rceil = 8 \text{ bp's, } 7 \text{ keys per node}$$

level	min nodes	min bp	min keys
1	1	2	1
2	2	2×8	2×7
3	16	16×8	16×7
4	128	128×8	128×7

min 1023 keys required for 4 levels

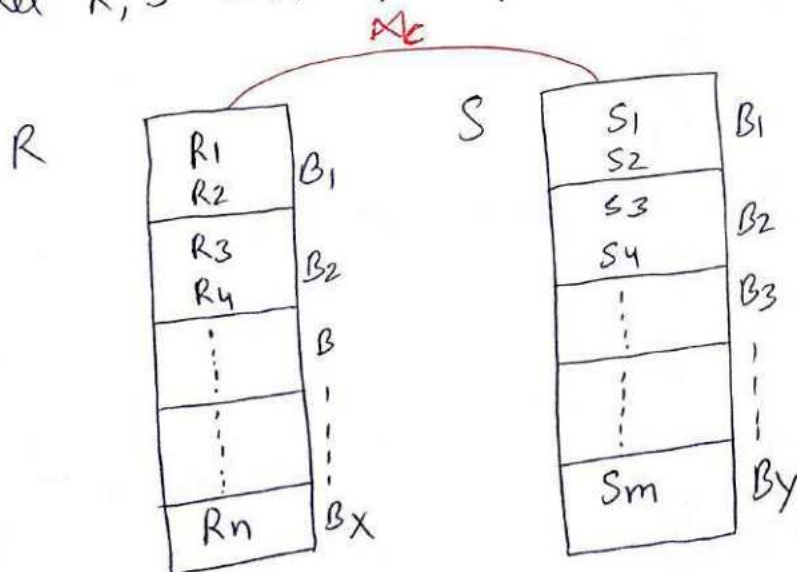
min 127 keys for 3 levels

$\Rightarrow$ max 3 levels possible for 1000 keys //

* Join Algorithm :-

- ① Nested Loop Join
- ② Block Nested Loop Join

$\Rightarrow$ Rel R, S with n, m tuples occupied x, y blocks respectively.



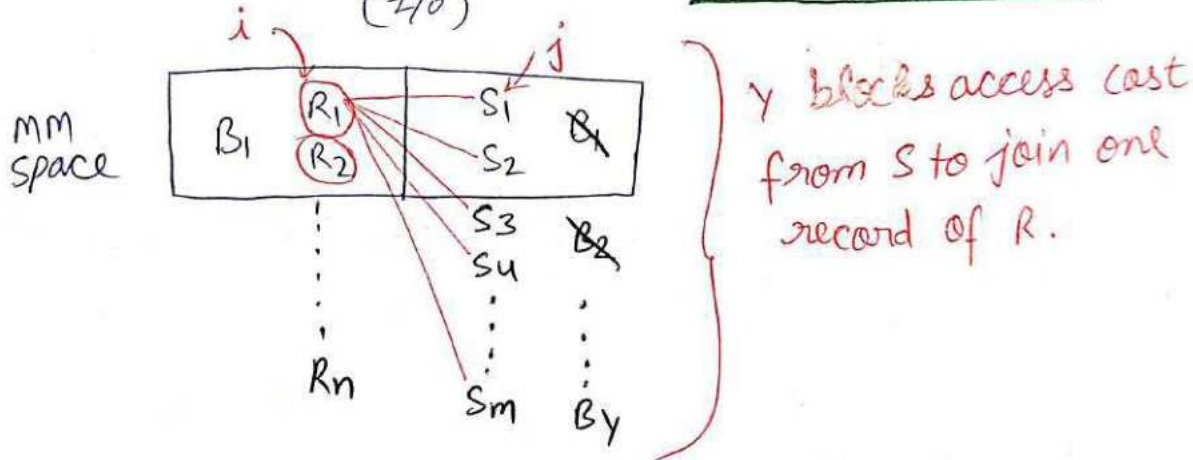
(Loop var : record number)

$\Rightarrow R \bowtie S \Rightarrow \text{for}(i=1; i \leq n; i++)$
 $\{ \text{for}(j=1; j \leq m; j++)$
 $\{ \text{Join}(R_i, S_j)$
 $\}$
 $\}$

$\Rightarrow$ Limited main memory space :-

Only two blocks of mm allocated to store R, S records at any time to perform Join:

① $R \bowtie S$ Access cost : $\{X + n \times Y\}$ blocks (I/O)



② $S \bowtie R$ Access cost : $(Y + m \times X)$ blocks

② Block Nested Loop Join :-

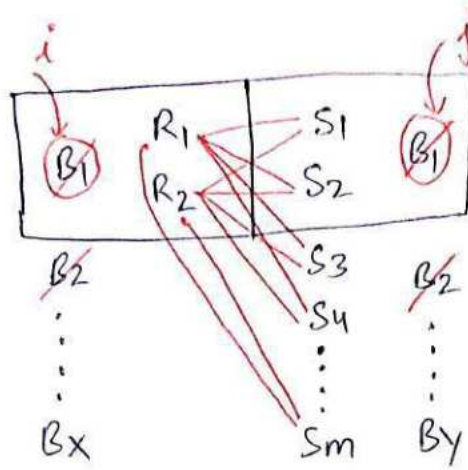
(Loop var : Block number of R, S)

$\Rightarrow R \bowtie S \Rightarrow \text{for}(i=1; i \leq X; i++)$
 $\{ \text{for}(j=1; j \leq Y; j++)$
 $\{ \text{Join}[\text{all records of } B_i \text{ block of } R, \text{ records of } B_j \text{ blocks of } S]$
 $\}$
 $\}$

⇒ MM Space Restriction :-

only two blocks of mm allocated to store R, S records at any time to perform join :-

① $R \bowtie S$ Access cost : $(X + X * Y)$ blocks



Y blocks access cost from S to join all records of one block of R

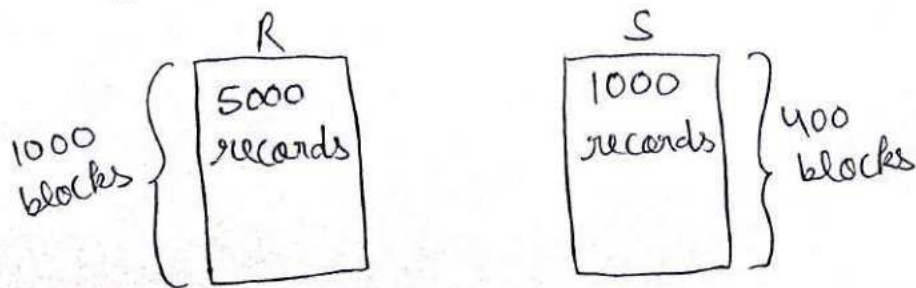
② $S \bowtie R$ Access cost : $(Y + Y * X)$ blocks

⇒ Block Nested Loop join algo. reduces I/O cost to perform join compared to nested loop join if mm space allocated for join is limited.

Q) Rel R with 5000 records occupied in 1000 blocks, Rel S with 1000 records occupied in 400 blocks. If mm space allocated for join is only two blocks to store R & S at any time.

- what is access cost to join by using nested loop join & most appropriate order of relations can be used for join?
- what is access cost to join by using block nested loop join & most appropriate order of relations can be used for join?

Soln



i) Nested Loop Join:-

- $R \bowtie S \Rightarrow 1000 + 5000 \times 400$
 $= 20,01,000$ blocks access cost

- $S \bowtie R$ $\Rightarrow 400 + 1000 \times 1000$
 $= 10,00,400$ blocks access cost

Ans $\Rightarrow 10,00,400$ ($S \bowtie R$)

ii) Block Nested Loop Join:-

- $R \bowtie S \Rightarrow 1000 + 1000 \times 400$
 $= 4,01,000$

- $S \bowtie R$ $\Rightarrow 400 + 400 \times 1000$
 $= 4,00,400$

Ans $\Rightarrow 4,00,400$ ($S \bowtie R$)

Q) How many min mm blocks required to join R & S with X & Y blocks respectively. Most appropriate order can be used for join by using nested loop join s.t. no block of R & S is retrieved more than once from Disk to MM.

Soln

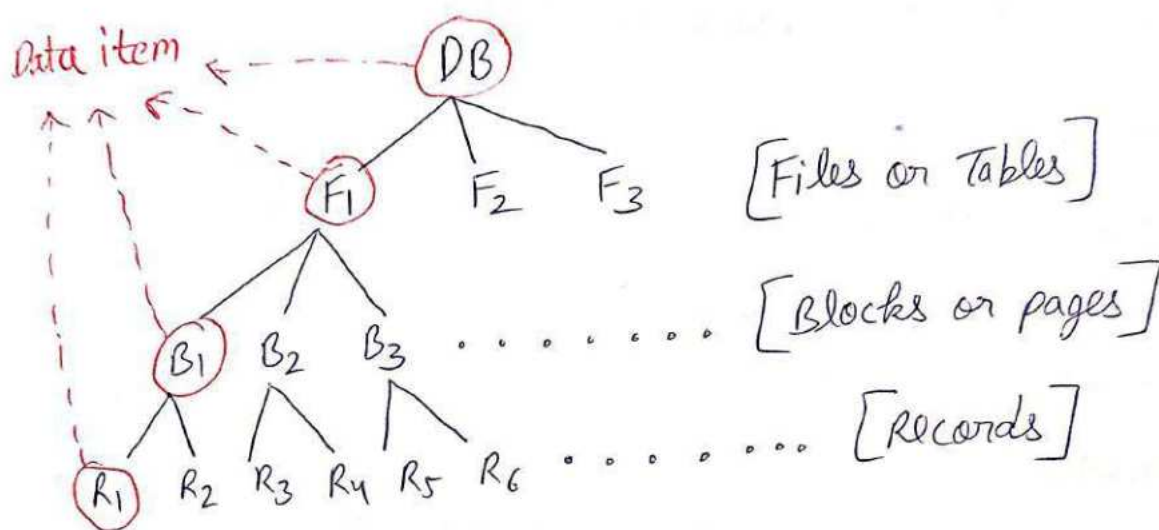
$$\frac{1}{\text{Result}} + \frac{1}{\text{Outer Rel}} + \frac{\min(X, Y)}{\text{Inner Rel}} \text{ :- MM Space}$$

* Transactions & Concurrency Control :-

⇒ Transaction :- Set of logically related operations to perform unit of task. (or) [Set of R, W op. to perform].

⇒ Data item :- [shared Resource]

- DB element which required to access by many transactions.



⇒ Concurrent Execution :- Simultaneous execution of two or more transactions over same DB.

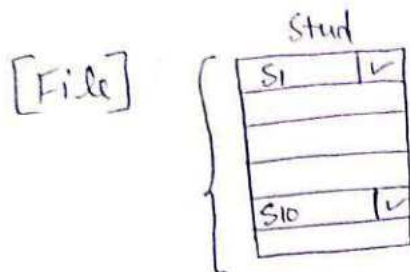
⇒ Degree of Concurrency :- Number of users (Transactions) who can execute simultaneously over same DB.

* Concurrent execution of two or more transaction over same DB may leads incorrect result.

- X : Data
- ① R - W
 - ② W - R
 - ③ W - W

⇒ Concurrency Control :- method (Protocol) to avoid incorrect result because of concurrent execution of many transactions.

* OS [Flat File] also maintain concurrency control at File Level.



[High Level Concurrency Control]

• Advantage :- Easy to maintain concurrency control (locks)

• Disadvantage :- Less degree of concurrency.

⇒ DBMS File's allowed low level concurrency control.

Record : resource

• Advantage :- more degree of concurrency

• Disadvantage :- Too complex to maintain concurrency control (locks).

⇒ RDBMS s/w supports multilevel concurrency control so that

• more degree of concurrency

• Easy to maintain lock table.

*** ⇒ Degree of concurrency if concurrency control only at

Record Level > Block Level > File Level > DB Level

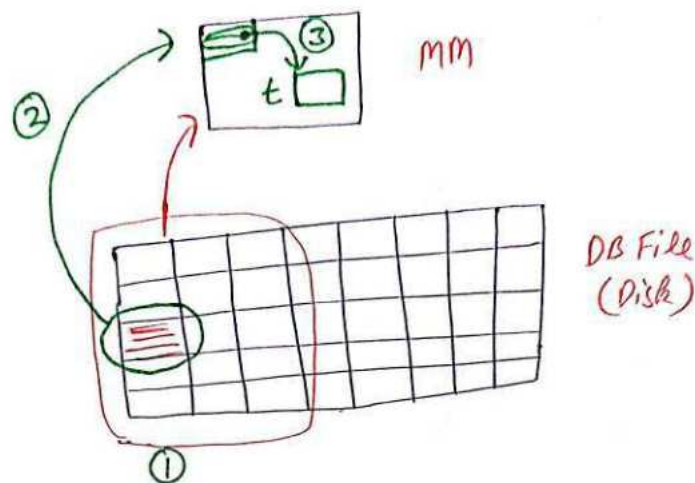
* main operations used for transaction :-

① Read(x) [x: data item] :-

⇒ Access value of data item x from DB File (Disk) in order to use the current value of x for transaction logic.

⇒ Read(x) :-

- 1) Find Block which contain data item x from DB file (Disk)
 - 2) Transfer block to main memory.
 - 3) Find address of data item x & copy value of x in programmed variable.
- Atomic



② Write(x) :-

⇒ updation of data item x in DB File.

* Write(x) :-

- 1) Find which contain data item x in DB File
 - 2) Transfer block to mm.
 - 3) Update Data item x in mm block
 - 4) Replace updated block into DB file
- Atomic

Eg:- Account (aid, bal)

Transaction :- Transfer 5000 amount from
aid 101 to 102

⇒ begin Trans T₁ :-

update Account bal = bal - 5000
where aid = 101;

USER

Update Account $bal = bal + 5000$
where $aid = 102$;

commit ;
end Trans ; // Trans executed successfully

⇒ begin Trans (T_1)

Read (bal of aid : 101)

$bal = bal - 5000$

write (bal of aid : 101)

Read (bal of aid : 102)

$bal = bal + 5000$

write (bal of aid : 102)

commit ;

end Trans (T_1) ;

⇒ T_2 : set bal of 101, 102 aid holder to 50000

begin Tran (T_2)

write (bal of aid : 101 as 50000)

write (bal of aid : 102 as 50000)

commit ;

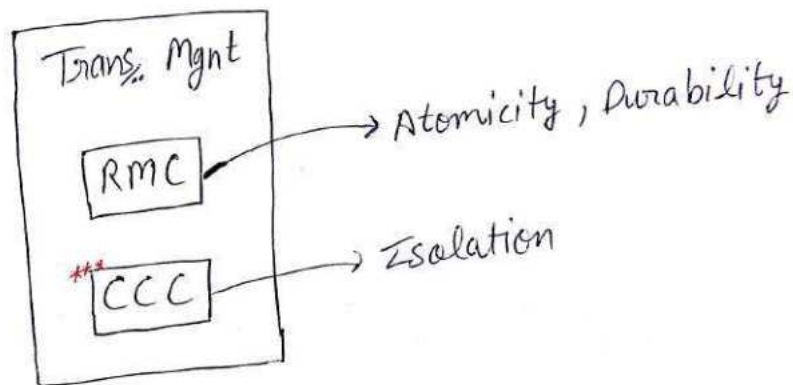
end Tran (T_2) ;

Blind
writes

* ACID Properties :-

⇒ To preserve integrity (correctness) transactions must obey ACID properties.

- | | | | | |
|------|---|--------------|--|-------|
| DBMS | { | • Atomicity | maintained by <u>recovery management component</u> of DBMS s/w | [RMC] |
| | | • Durability | | |
| | | • Isolation | maintained by <u>concurrency control component</u> of DBMS s/w | [CCC] |
| USER | { | Consistency | maintained by USER | |
| | | | → DB Administrator [DBA] | |
| | | | → DB Developer | |
| | | | → End USER | |



⇒ Atomicity :- [All/None]

* Execute all opⁿ. of transaction including commit or execute none of the opⁿ. of transaction.

Eg: T₁ : Transfer 500 from A to B

Trans (T₁)

Roll Back

failed →

✓ R(A)

✗ A = A - 500

✗ W(A)

R(B)

B = B + 500

W(B)

Commit

Atomicity Failed

A = 2000¹⁵⁰⁰

B = 3000

Incorrect State of DB

[All/None]

Commit Rollback

- Recovery Management Component :-

⇒ Rollback's Trans (T_1) if Trans (T) failed anywhere before Commit.

- Rollback [Abort] :-

⇒ Undo all updation of DB file which are done by failure Trans.

- Transaction Log :-

⇒ File maintained by RMC in Disk to record all activities of trans.

Eg:-

Trans (T_1)

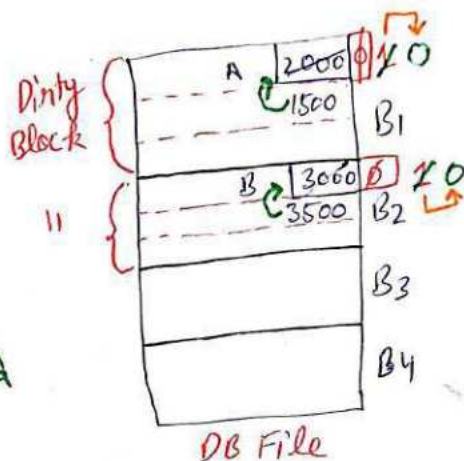
✓ R(A)

✓ W(A)

✓ R(B)

✓ W(B) failed

✓ Commit



Trans. Log file

1. T_1 begins

2. $T_1, R, A, 2000$

3. $T_1, W, A, 2000, 1500$

4. $T_1, R, B, 3000$

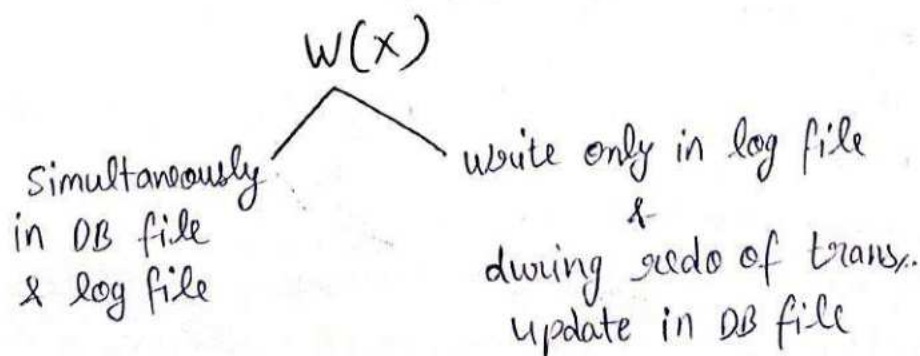
5. $T_1, W, B, 3000, 3500$

6. $T_1, \text{Rollback Commit}$

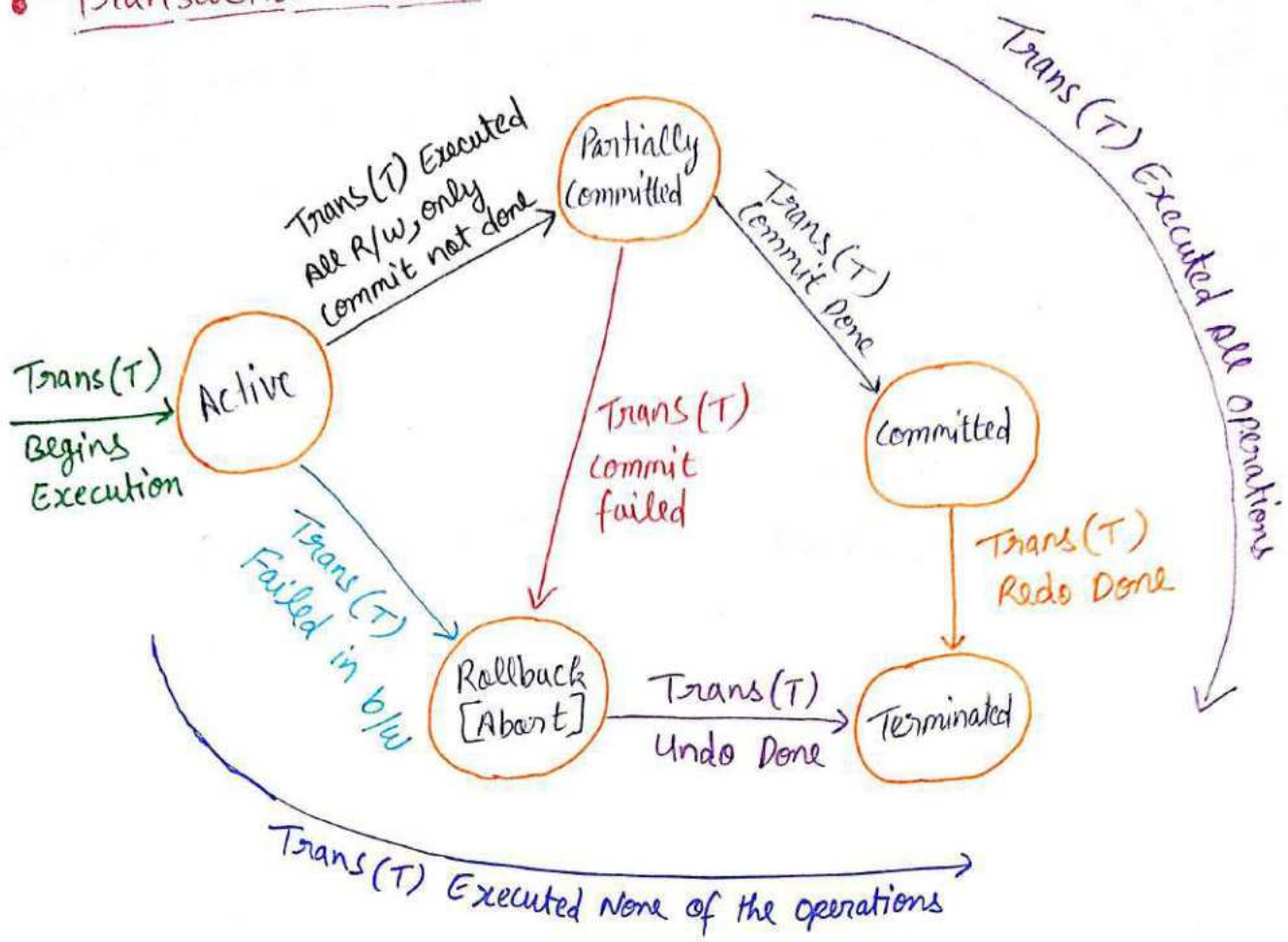
- Dirty Block :- DB block which is updated by uncommitted transaction.

- Undo of Trans (T) :- All updations of DB file because of transaction Rollback.

- Redo of Trans (T) :- All updations of DB file because of Trans (T) Commit.

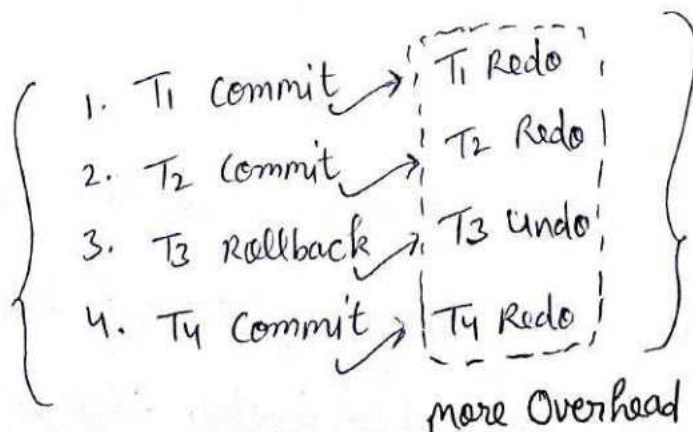


• Transaction States :-



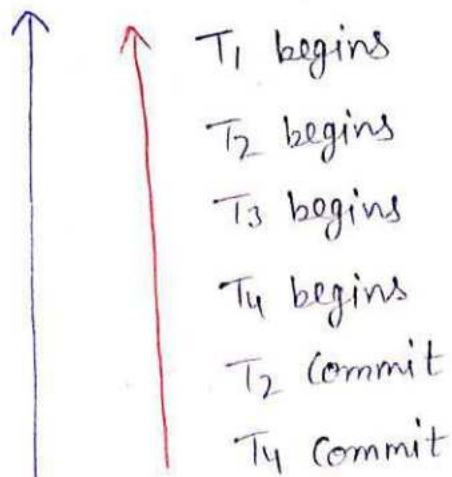
• Consistency Check Point :- [To Reduce Overhead]

- 1) All committed transaction until previous checkpoint is call for redo operation.
- 2) All transactions of Rollback state until previous checkpoint call for undo operation.
- 3) Clean log entries of terminated transactions.



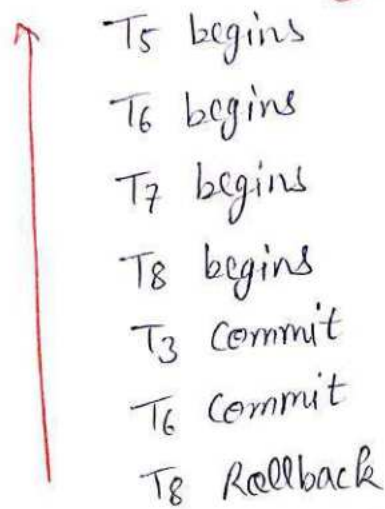
Eg:-

9:00 AM DB begins



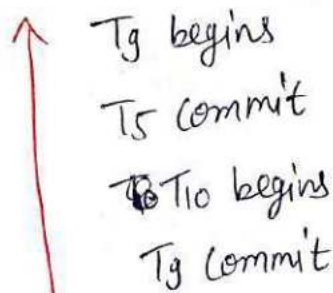
9:05 AM check Point

T₂ T₄ Redo



9:10 AM check Point

T₃ T₆ Redo
T₈ Undo



System
Crash

T₅ T₉ Redo
T₁ T₇ T₁₀ Undo

• System Crash Recovery :-

- 1) All committed trans. until previous checkpoint call for Redo.
- 2) All Rollback trans. until previous checkpoint call for Undo.
- 3) All uncommitted trans. in entire system change the state to Rollback & call for undo.
- 4) Clean log entries.

Q) consider given log file

① T_1 begins ② T_1 R, A, 10

③ T_2 begins ④ T_3 begins

⑤ T_3 W, B, 0, 20 ⑥ T_2 W, A, 10, 30

⑦ T_3 commit ⑧ checkpoint → T_3 Redo

⑨ T_4 begins ⑩ T_5 begins

⑪ T_4 commit ⑫ T_2 commit → T_2, T_4 Redo
 T_1, T_5 undo

If system crash occurs after log entry 12.
what is crash recovery?

a) T_2 T_3 T_4 Redo, T_1 T_5 Undo

~~b)~~ T_2 T_4 Redo, T_1 T_5 Undo ✓ **ans**

c) T_2 redo, T_1 T_3 T_4 T_5 undo

d) T_1 T_2 T_4 Redo, T_3 T_5 undo

⇒ Durability :-

* Transaction must be able to Rollback under every possible failures.

* DB updates done by committed transactions must be persistent even system crash occurred.

• Transaction Failure :-

1) Power Failure

2) S/W crash (OS Overloaded)

3) Concurrency control Protocol may kill Trans.

4) Disk Crash } Affects Durability

} Does not affect durability.

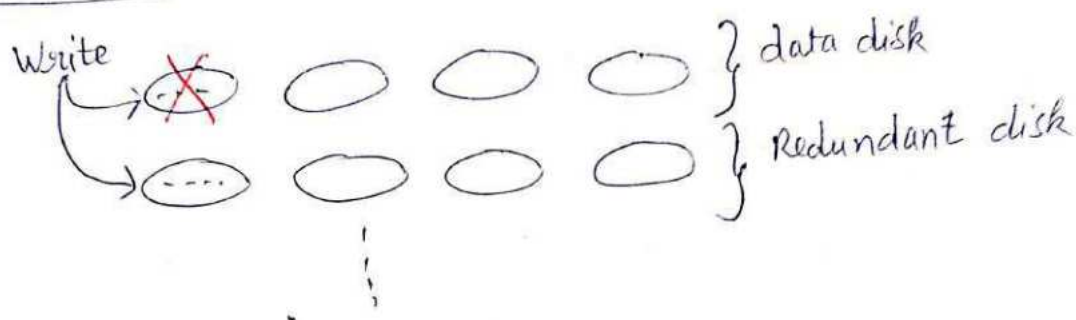
⇒ RAID Disk design must be used :-

[RAID: Redundant Array of Independent Disk's]

RAID : 0 No redundant disk



RAID : 1 mirror image of Independent Disk



⇒ Consistency :-

* Before execution & after execution of trans., DB must be in consistent state.

* Incorrect results because of faulty transactions not possible to recover by DBMS because it is not in scope of DBMS software.

* Isolation :- [Concurrency Control Component]

⇒ Concurrent execution of two or more than two transactions result must be equal to result of some serial execution of transactions.

• Schedule :- Time order execution sequence of two or more transactions.

Eg:-

S:

T ₁	T ₂	T ₃
R ₁ (A)		R ₃ (B)
	W ₂ (A)	
W ₁ (A)	W ₂ (B)	R ₃ (C)

Schedule (indicated by a red arrow pointing down)

① Serial Schedule :-

⇒ Transactions must execute one after other.

Eg:- T₁: Transfer 500 from A to B

T₂: Display to A+B

A = 2000
B = 3000
↓ consistency criteria

A: 1500
B: 3500
T₂: 5000

S₁:

T ₁	T ₂
R(A)	
W(A)	
R(B)	
W(B)	
Commit	

Serial (indicated by a red arrow pointing down)

T₁ → T₂ *correct*

S₂:

T ₁	T ₂
	R ₂ (A): 2000
	R ₃ (B): 3000
	Commit 5000
R ₁ (A)	
W ₁ (A)	
R ₁ (B)	
W ₁ (B)	
Commit	

correct (indicated by a red arrow pointing down)

T₂ → T₁ *Serial*

• Advantage :-

⇒ Every Serial Schedule always preserves integrity (correctness).

• Disadvantage :-

⇒ Less Degree of Concurrency.

② Concurrent Schedule :-

⇒ Transactions allowed to execute simultaneously / Concurrently / Interleaved.

Eg:-

S3:

T ₁	T ₂
r ₁ (A)	
w ₁ (A)	
	r ₂ (A) : 1500
	r ₂ (B) : 3000
	Commit 4500
r ₁ (B)	
w ₁ (B)	
Commit	

*A = 2000
B = 3000*

Incorrect Schedule

- S3 not equal S1
 - S3 " " S2
- Isolation failed
(Non-serializable Schedule)

S4:

T ₁	T ₂
r ₁ (A)	
w ₁ (A)	
	r ₂ (A) : 1500
r ₁ (B)	
w ₁ (B)	
Commit	
	r ₂ (B) : 3500
	Commit 5000

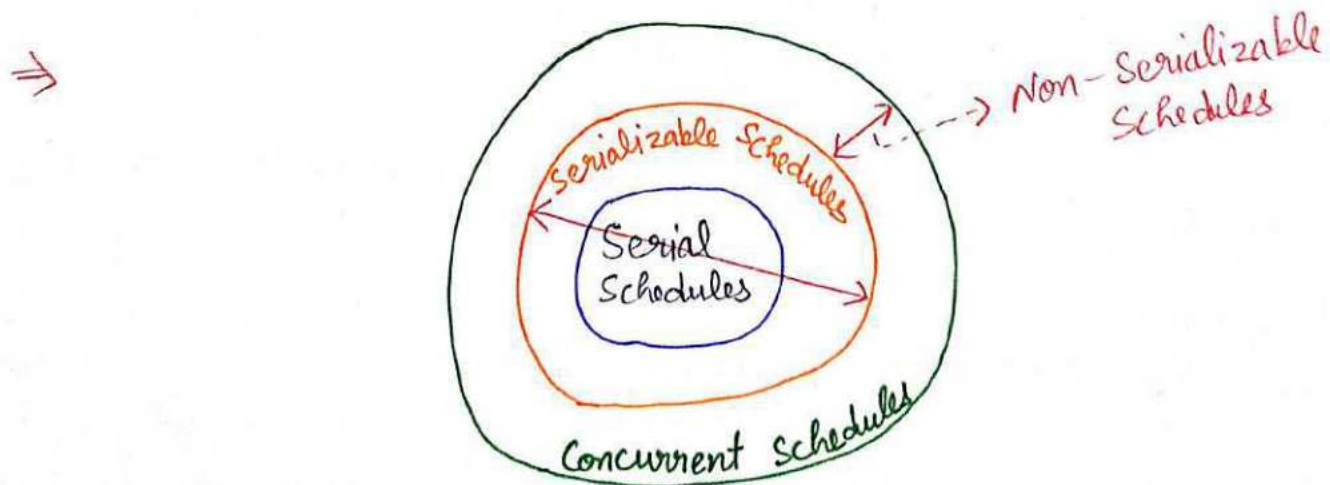
Correct Schedule

- S4 equal S1 } Isolation True
(Serializable Schedule)

⇒ Serializable Schedule :-

⇒ Given schedule S is serializable iff at least one serial schedule must be equal to the given schedule.

⇒ Every serial schedule is always serializable schedule.



$\Rightarrow S_1, S_2, S_3, S_4$ // Given

of concurrent : $S_1 S_2 S_3 S_4 = 15$

of serial : $S_1 S_2 = 2$

of non-serial : $S_3 S_4 = 13$

of Serializable : $S_1 S_2 S_4$

of non serializable : S_3

Q) If n trans. are there $T_1 T_2 \dots T_n$, How many serial schedules possible for given n trans.?

Solⁿ $n!$ serial schedules possible if n trans. given.

Q) $T_1: \overrightarrow{r_1(A) w_1(A) r_1(B) w_1(B)}$

$T_2: \overrightarrow{r_2(A) r_2(B)}$

How many concurrent schedules possible?

Solⁿ

S:

	T_1	T_2
6 operations	$r_1(A)$	
		$r_2(A)$
	$w_1(A)$	
		$r_2(B)$
	$r_1(B)$	
	$w_1(B)$	

$$= {}^6C_4 \cdot {}^2C_2 = 15 \text{ concurrent schedules}$$

$$= \frac{n!}{(n-k)! k!} = 15 //$$

Q) $T_1 T_2 T_3$ three transactions with 5, 4, 3 operations each respectively, how many concurrent schedules possible?

Solⁿ

$${}^{12}C_5 \cdot {}^7C_4 \cdot {}^3C_3 \Rightarrow$$

$$= \frac{n!}{s! \cdot 4! \cdot 3!} = 27,720$$

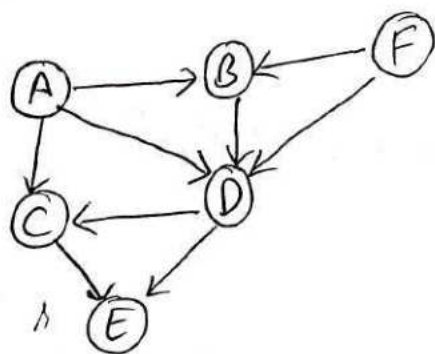
Serializable Schedule :-

- ① Conflict serializable schedule
- ② View serializable schedule

Topological Order :-

- Graph Traversal algorithm which is only applicable for Directed Acyclic Graph (DAG)
- 1) Visit vertex 'v' whose indegree 0 & delete visited vertex 'v' from graph.
 - 2) Repeat step ① for all vertices of the graph.

Eg:- ①

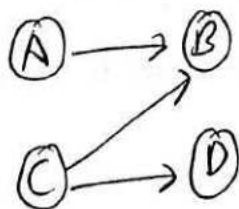


How many topological orders are possible?

Soln

$$\left\{ \begin{array}{l} A-F-B-D-C-E \\ F-A-B-D-C-E \end{array} \right\} \text{ Ans} = 2 //$$

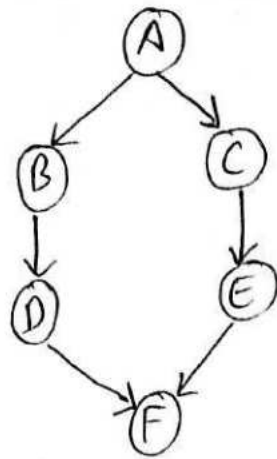
Q) Find the number of topological order in the given graph?



Soln

$$\left\{ \begin{array}{l} A-C-B-D, A-C-D-B \\ C-D-A-B, C-B-B \\ C-A-B-D \end{array} \right\} \text{ Ans} = 5 //$$

Q) Find the number of topological orders in the given graph?



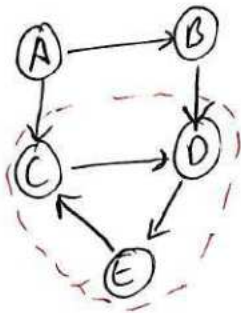
Soln

$$[A \text{ --- } F]$$

${}^4C_2 \times {}^2C_2$

$$= 6 // \text{Ans.}$$

Q) Number of topological orders possible?



Soln

Graph is cyclic so topological order not possible.
Ans = 0.

*) NULL Graph with n vertices # of Topological orders : $n!$

$$\left\{ \begin{matrix} \textcircled{A} & \textcircled{B} \\ \textcircled{C} & \textcircled{D} \end{matrix} \right\} \text{ NULL graph}$$

NOTE:- Topological ordering is also a type of Greedy algorithm.

⇒ Conflict Pair :-

* Pair of operations from schedule (S) with

- Atleast one write
- Over same data item
- From different transactions.

Eg:-

- $S: \dots r_i(A) \dots w_j(A)$
- $S: \dots w_i(A) \dots r_j(A)$
- $S: \dots w_i(A) \dots w_j(A)$

} Conflict Pairs

- $S: \dots r_i(A) \dots r_j(A)$
- $S: \dots r_i(A)/w_i(A) \quad r_j(B)/w_j(B)$

} Non-conflict Pairs

⇒ Pair of operations from schedule S which are belongs to same transaction are neither conflict pair nor non-conflict pair.

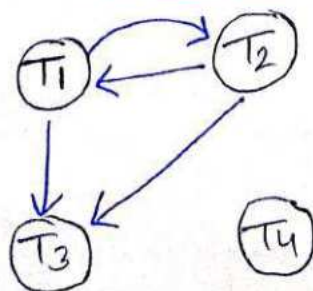
⇒ Precedence Graph :-

* Graph $G(V, E)$

Vertices (V): Transaction of schedule (S)

Edges (E): Conflict Pair Precedence of Schedule (S)

Eg:- $S: r_2(A) r_2(B) w_1(A) w_2(A) w_2(B) w_3(A) w_3(B) w_4(D)$

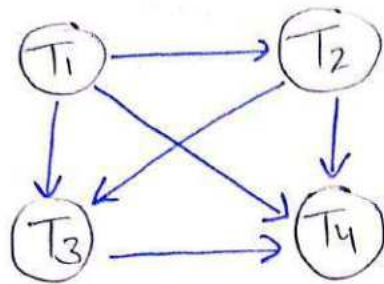


: Precedence graph of S

Q) Draw precedence graph of given transaction.

S: $g_1(A)$ $g_2(A)$ $g_3(A)$ $g_4(A)$ $w_1(B)$ $w_2(B)$ $w_3(B)$ $w_4(B)$

Solⁿ



**** $\Rightarrow$ Conflict Equal Schedules :-**

* S_1, S_2 conflict equal schedules iff S_2 derived by swapping adjacent non-conflict pairs of S_1 .

• $S_1 \xrightleftharpoons[\text{Adjacent non-conflict pairs}]{\text{Swap}} S_2 \quad \therefore S_1, S_2 \text{ conflict equal schedules}$

(OR)

* S_1, S_2 conflict equal iff

① S_1, S_2 must be with same set of transactions.

$S_1: \dots T_i \dots$

$\downarrow$
 $R(A)$
 $R(B)$
 $W(B)$

$S_2: \dots T_i \dots$

$\downarrow$
 $R(A)$
 $R(B)$
 $W(B)$

② Every conflict pair of S_1 must be same Precedence in S_2 .

Eg:-

S1	T1	T2
	$\pi_1(A)$	
	$w_1(A)$	
	$w_1(B)$	$\pi_2(A)$

S2	T1	T2
	$\pi_1(A)$	
	$w_1(A)$	$\pi_2(A)$
	$w_1(B)$	

Conflict
Equal

Q) Test S1 S2 conflict equal or not.

S1: $\pi_1(A) \pi_2(A) \pi_3(A) w_1(B) w_2(B) w_3(B)$

S2: $\pi_1(A) w_1(B) \pi_3(A) \pi_2(A) w_2(B) w_3(B)$

Soln
S1: $\pi_1(A) \pi_2(A) \pi_3(A) w_1(B) w_2(B) w_3(B)$

$\pi_1(A) \pi_2(A) w_1(B) \pi_3(A) w_2(B) w_3(B)$

$\pi_1(A) w_1(B) \pi_2(A) \pi_3(A) w_2(B) w_3(B)$

$\pi_1(A) w_1(B) \pi_3(A) \pi_2(A) w_2(B) w_3(B) \Rightarrow$ Conflict equal //

Q) S1: $\pi_1(A) w_1(A) \pi_2(A) \pi_2(B) \pi_1(B) w_1(B)$

S2: $\pi_1(A) w_1(A) \pi_1(B) w_1(B) \pi_2(A) \pi_2(B)$

Soln
S1: $\pi_1(A) w_1(A) \pi_2(A) \pi_2(B) \pi_1(B) w_1(B)$

$\pi_2(A) \pi_1(B) \pi_2(B)$

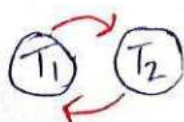
$\pi_1(B) \pi_2(A) \pi_2(B) w_1(B)$

Conflict not possible to swap pairs

$\Rightarrow$ S1, S2 are not conflict equal //

$\Rightarrow$ Precedence graph:-

S1:



S2:



NOTE:-

*) If S_1, S_2 conflict equal then S_1, S_2 precedence graph's equal. [S_1, S_2 with same set of trans.]

*) If S_1, S_2 with precedence graphs not equal, then S_1, S_2 not conflict equal.

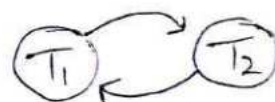
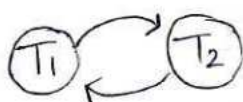
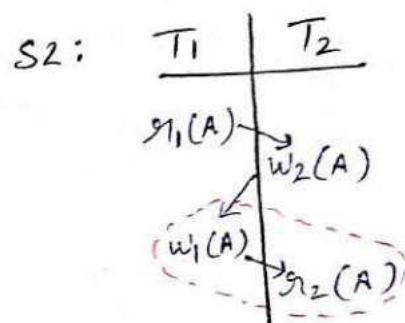
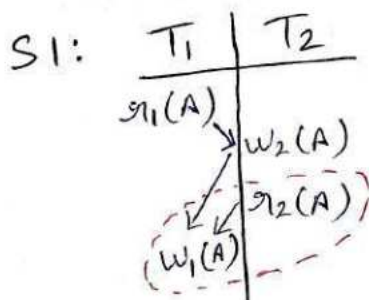
*) If S_1, S_2 precedence graphs equal.

(and)

Acyclic precedence graph, then S_1, S_2 conflict equal.

*) If S_1, S_2 precedence graphs equal (and) cyclic precedence graphs then S_1, S_2 may/may not conflict equal.

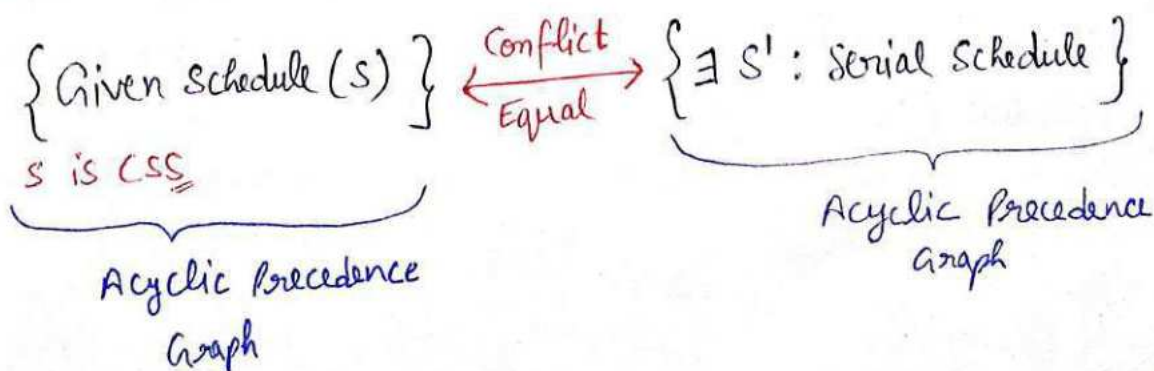
Eg:-



S_1, S_2 not conflict Equal //

* Conflict Serializable Schedule :-

$\Rightarrow$ Given schedule (S) is conflict serializable schedule iff some serial schedule (S') must be conflict equal to given schedule (S).



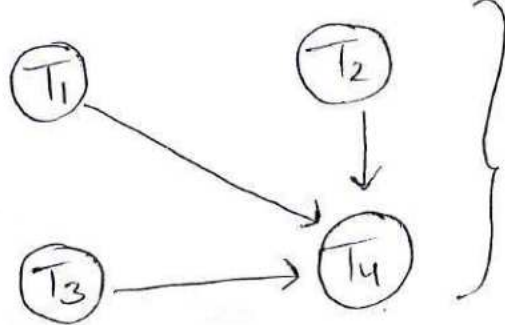
⇒ Testing Condition of Conflict Serializable Schedule :-

- * Given schedule S is CSS iff precedence graph of Schedule (S) must be Acyclic.
- * Serial schedule sequence which is conflict equal to given schedule are topological orders of schedule (S) precedence graph.

Q) $S: r_1(A) r_2(A) r_3(A) r_4(A) w_1(B) w_4(B) r_1(C) r_2(C) w_4(C)$
 $w_3(D) r_4(D)$

- S is CSS or not?
- # of serial schedules are conflict equal to schedule S ?

Soln



$$\left. \begin{array}{cccc} T_1 & T_2 & T_3 & T_4 \\ T_1 & T_3 & T_2 & T_4 \\ T_2 & T_1 & T_3 & T_4 \\ T_2 & T_3 & T_1 & T_4 \\ T_3 & T_2 & T_1 & T_4 \\ T_3 & T_1 & T_2 & T_4 \end{array} \right\} = 6$$

i) S is CSS //

ii) 6 //

NOTE :-

$$\left\{ \begin{array}{l} \text{\# of serial schedules} \\ \text{conflict equal to} \\ \text{schedule (S)} \end{array} \right\} = \left\{ \begin{array}{l} \text{\# of topological order} \\ \text{of schedule (S) precedence} \\ \text{graph} \end{array} \right\}$$

* View Serializable Schedule :-

⇒ Given schedule S is VSS iff some serial schedule S' must be View equal to given schedule S .

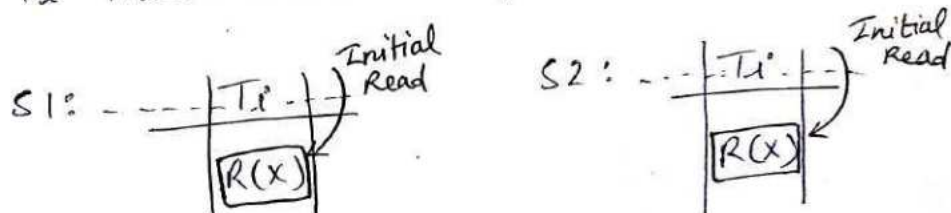
$$\left\{ \begin{array}{l} \text{Given schedule } S \\ S \text{ is VSS} \end{array} \right\} \xleftrightarrow[\text{Equal}]{\text{View}} \left\{ \exists S' : \text{Serial Schedule} \right\}$$

⇒ View Equal Testing Conditions :-

* S_1 S_2 schedule View equal iff

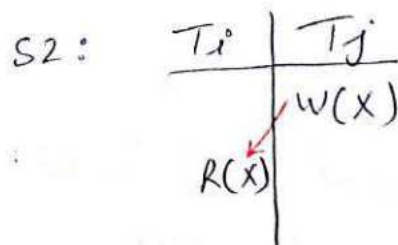
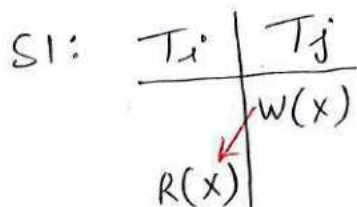
① Initial Read :-

* If T_i reads 'x' from initial database in S_1 then in S_2 also T_i must read 'x' from initial database.

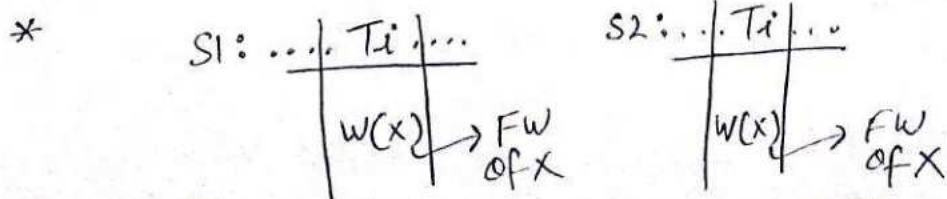


② Updated Read [write, Read] :-

* If T_i reads (x) which is updated by T_j in S_1 then in S_2 also T_i must read x which is updated by T_j .



③ Final Write :-



eg:- ① S1:

T ₁	T ₂	T ₃
R(A)	R(A) → IR	W(A)

S2:

T ₁	T ₂	T ₃
R(A)	W(A)	R(A) ←

S1 S2 not VE

② S1:

T ₁	T ₂	T ₃
W(A)	W(A)	R(A) ←

S2:

T ₁	T ₂	T ₃
W(A)	W(A)	R(A) ←

S1 S2 not VE

③ S1:

T ₁	T ₂	T ₃
W(A)	W(A)	W(A) (FW of A)

S2:

T ₁	T ₂	T ₃
W(A)	W(A)	W(A) (FW of A)

S1 S2 are VE

Q) Test S1 S2 VE?

1) S1: $\overset{IR}{\pi_1(A)} \overset{IR}{\pi_2(A)} \overset{IR}{\pi_3(A)} w_1(A) w_2(A) w_2(A)$
 S2: $\overset{IR}{\pi_1(A)} w_1(A) \pi_2(A) w_2(A) \pi_3(A) w_3(A)$
 (Arrows from $\pi_2(A)$ in S1 to $\pi_2(A)$ in S2 and from $w_2(A)$ in S1 to $w_3(A)$ in S2 are marked with X)

Soln

S1: FW of A: T₃

⇒ S1 S2 not VE

S2: FW of A: T₃

2) S1: $\overset{IR}{\pi_2(A)} \overset{IR}{\pi_2(B)} \overset{IR}{\pi_1(A)} w_2(A) w_1(A) \overset{IR}{\pi_2(C)} \overset{IR}{\pi_3(C)} \pi_3(A) w_1(B)$

S2: $\overset{IR}{\pi_1(A)} \overset{IR}{\pi_3(C)} \overset{IR}{\pi_2(A)} w_2(A) w_1(A) \overset{IR}{\pi_2(C)} \pi_3(A) w_2(B) w_1(B)$
 (Arrows from $w_1(A)$ in S1 to $w_1(A)$ in S2 and from $\pi_3(C)$ in S1 to $\pi_3(A)$ in S2 are shown)

Soln S1: FW of A: T₁, FW of B: T₃

S2: FW of A: T₁, FW of B: T₃

⇒ S1 S2 are VE.

⇒ S1 S2 not conflict equal.

NOTE:-

- $\Rightarrow$ If Schedules $S_1 S_2$ conflict equal then $S_1 S_2$ also view equal.
- $\Rightarrow$ If Schedules $S_1 S_2$ not conflict equal then $S_1 S_2$ may / may not view equal.
- $\Rightarrow$ If Schedules $S_1 S_2$ not view equal then $S_1 S_2$ also not conflict equal.

Q) Test S is VSS or not?

$$S: \underbrace{r_1(A)}_{IR} \underbrace{r_2(A)}_{IR} w_1(A) w_2(A)$$

Seqm
not preferred

$$S': r_1(A) w_1(A) \underbrace{r_2(A) w_2(A)}_{\text{red arrow}} \\ T_1 \rightarrow T_2 \text{ Serial}$$

not VE

$$S': r_2(A) w_2(A) \underbrace{r_1(A) w_1(A)}_{\text{red arrow}} \\ T_2 \rightarrow T_1 \text{ Serial}$$

not VE

$\Rightarrow$ Given S is not VSS //

$\Rightarrow$ Testing Conditions of VSS :-

* Given Schedule 'S'

Given Schedule(S)

View Equal Serial Schedule

① FW:- $X: T_i T_j \underbrace{(T_k)}_{FW}$

$Y: T_i \underbrace{(T_j)}_{FW}$

$Z: \underbrace{(T_i)}$

$\Rightarrow (T_i, T_j) \rightarrow T_k$

$\Rightarrow T_i \rightarrow T_j$

$\Rightarrow$ Every serial

② IR:-

Data item	Initial Read	Write	
X	T_i	T_j	$\Rightarrow T_i \rightarrow T_j$
Y	T_i T_j	$T_j T_k$	$\Rightarrow T_j \rightarrow T_k$ $\Rightarrow T_i \rightarrow (T_j, T_k)$
Z	$T_i T_j$	—	$\Rightarrow$ Every serial
X	T_i only	T_i only	$\Rightarrow$ Every serial

③ WR:- ① $W_i(X) \rightarrow R_j(X)$ and no other Trans writes X

$\Rightarrow T_i \rightarrow T_j$ $\left\{ \begin{array}{l} T_k T_i T_j \\ T_i T_k T_j \\ T_i T_j T_k \end{array} \right\}$

② $W_i(Y) \rightarrow R_j(Y)$ and some Trans T_k is also writes Y

$\Rightarrow T_i \rightarrow T_j$ and T_k should not b/w T_i & T_j $\left\{ \begin{array}{l} T_k T_i T_j \\ T_i T_j T_k \end{array} \right\}$

Q) Test given schedule VSS or not?

① $S: r_2(A) w_1(A) w_2(A) w_3(A)$

Soln

	Given Schedule (S)	view equal serial						
(FW)	A: $T_1 T_2 (T_3)_{FW}$	$\Rightarrow (T_1, T_2) \rightarrow T_3$						
(IR)	<table border="1"> <thead> <tr> <th>data item</th><th>IR</th><th>write</th></tr> </thead> <tbody> <tr> <td>A</td><td>T_2</td><td>$T_1 T_2 T_3$</td></tr> </tbody> </table>	data item	IR	write	A	T_2	$T_1 T_2 T_3$	$\Rightarrow T_2 \rightarrow (T_1, T_3)$
data item	IR	write						
A	T_2	$T_1 T_2 T_3$						

$S': T_2 T_1 T_3$ Serial view equal to S.

Ans $\Rightarrow$ S is VSS.

② S: $\overset{IR}{r_2}(A) \overset{IR}{r_2}(B) w_2(A) \overset{IR}{r_3}(C) w_1(A) \overset{IR}{r_2}(C) \overset{IR}{r_3}(A) w_1(B)$
 $w_2(B) w_3(B)$

Soln

Given Schedule S | view equal serial

(FW) A: $T_2 \text{ (T}_1\text{)}_{FW} \Rightarrow T_2 \rightarrow T_1$
 B: $T_1 T_2 \text{ (T}_3\text{)}_{FW} \Rightarrow (T_1, T_2) \rightarrow T_3$

IR	Data item	IR	write	
	A	T_2	$T_1 T_2$	$\Rightarrow T_2 \rightarrow T_1$
	B	T_2	$T_1 T_2 T_3$	$\Rightarrow T_2 \rightarrow (T_1, T_3)$
	C	$T_2 T_3$	—	$\Rightarrow$ —

(UR) $w_1(A) \rightarrow r_3(A)$ and
 other trans writes A: $\underline{T_2}$ $\Rightarrow T_1 \xrightarrow{\text{(T}_2\text{)}} T_3$

$S': T_2 \rightarrow T_1 \rightarrow T_3$ serial schedule view equal to S.

Ans $\Rightarrow$ S is VSS //

Q) S: $\overset{IR}{r_1}(A) \overset{IR}{r_2}(A) \overset{IR}{r_3}(A) \overset{IR}{r_4}(A) w_1(B) w_2(B) w_3(B) w_4(B) \overset{IR}{r_1}(C) \overset{IR}{r_3}(C)$
 $w_4(C)$

How many serial schedule view equal to S.

Soln

Given Schedule (S) | view equal serial

(FW) B: $T_1 T_2 T_3 \text{ (T}_4\text{)}_{FW} \Rightarrow (T_1, T_2, T_3) \rightarrow T_4$
 C: $\text{(T}_4\text{)}_{FW} \Rightarrow$ —

IR	Data item	IR	write	
	A	$T_1 T_2 T_3 T_4$	—	$\Rightarrow$ —
	C	$T_1 T_3$	T_4	$\Rightarrow (T_1, T_3) \rightarrow T_4$

$(T_1, T_2, T_3) T_4$
 3!

$\Rightarrow$ 6 serial schedules view equal to S.

Q) S: $\overset{IR}{r_1(A)} \overset{IR}{r_3(D)} \overset{w_1(B)}{w_1(B)} \overset{r_2(B)}{r_2(B)} \overset{w_3(B)}{w_3(B)} \overset{r_4(B)}{r_4(B)} \overset{w_2(C)}{w_2(C)} \overset{r_5(C)}{r_5(C)}$
 $\overset{w_4(E)}{w_4(E)} \overset{r_5(E)}{r_5(E)} \overset{w_5(B)}{w_5(B)}$

Soln

(FW)

B: T_1 T_3 $(T_5)_{FW}$

C: $(T_2)_{FW}$

E: $(T_4)_{FW}$

(IR)

Data item	IR	write
A	T_1	—
D	T_3	—

(UR)

① $w_1(B) \rightarrow r_2(B)$

other trans writes B : $\{T_3, T_5\}$

② $w_3(B) \rightarrow r_4(B)$

other trans writes B : $\{T_1, T_5\}$

③ $w_2(C) \rightarrow r_5(C)$

no other trans write C

④ $w_4(E) \rightarrow r_5(E)$

no other trans write E

view equal serial

$\Rightarrow (T_1, T_3) \rightarrow T_5$

$\Rightarrow$ —

$\Rightarrow$ —

$\Rightarrow$ —

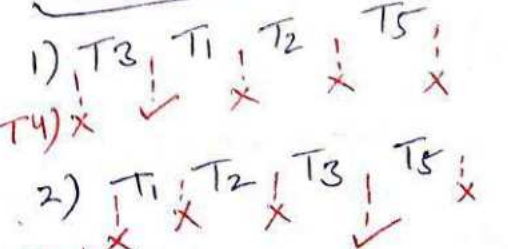
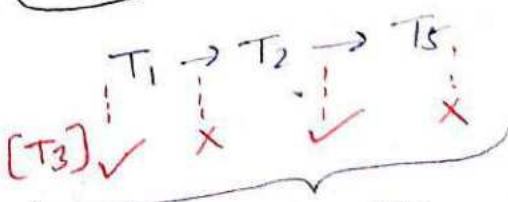
$\Rightarrow$ —

$\Rightarrow T_1 \xrightarrow{(T_3, T_5)} T_2$

$\Rightarrow T_3 \xrightarrow{(T_1, T_5)} T_4$

$\Rightarrow T_2 \rightarrow T_5$

$\Rightarrow T_4 \rightarrow T_5$



Ans = 2 //

$\checkmark T_3 T_4 T_1 T_2 T_5 \Leftarrow$

$\checkmark T_1 T_2 T_3 T_4 T_5 \Leftarrow$

Q) S: $w_1(A) \rightarrow R_2(A) \rightarrow w_3(A) \rightarrow R_4(A) \rightarrow w_5(A) \rightarrow R_6(A)$

Soln

Given Schedule (S)	view equal serial
(FW) A: $T_1 \ T_3 \ (\overline{T_5})_{FW}$	$\Rightarrow (T_1, T_3) \rightarrow T_5$
(UR) ① $w_1(A) \rightarrow R_2(A)$ other trans: $\{T_3, T_5\}$	$\Rightarrow T_1 \xrightarrow{(T_3, T_5)} T_2$
② $w_3(A) \rightarrow R_4(A)$ other trans: $\{T_1, T_5\}$	$\Rightarrow T_3 \xrightarrow{(T_1, T_5)} T_4$
③ $w_5(A) \rightarrow R_6(A)$ other trans: $\{T_1, T_3\}$	$\Rightarrow T_5 \xrightarrow{(T_1, T_3)} T_6$

Ans = 2 //

$\left\{ \begin{array}{l} T_1, T_2, T_3, T_4, T_5, T_6 \\ T_3, T_4, T_1, T_2, T_5, T_6 \end{array} \right\}$

Q) S: $w_1(A) R_2(A) \rightarrow w_3(A) R_4(A) \rightarrow w_5(A) R_6(A) \xrightarrow{FW \text{ Last}} w_7(A) R_8(A)$

Soln

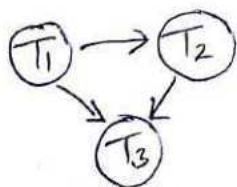
$3! = 6$

6 serial schedule view equal to S.

Q) S: $g_1(A) \rightarrow g_2(A) \rightarrow g_3(A) \rightarrow w_1(B) \rightarrow w_2(B) \rightarrow w_3(B)$ Given schedule

i) # of serial schedules conflict equal to S.

Soln



$S': T_1 \ T_2 \ T_3 \rightarrow$

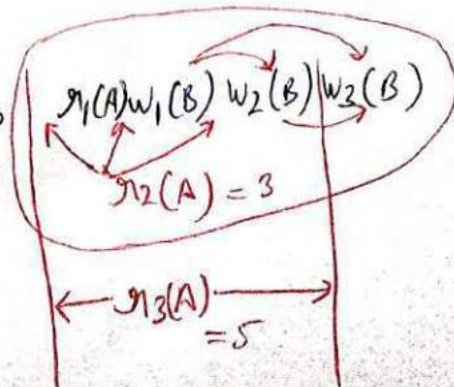
Serial

(# of topological orders)

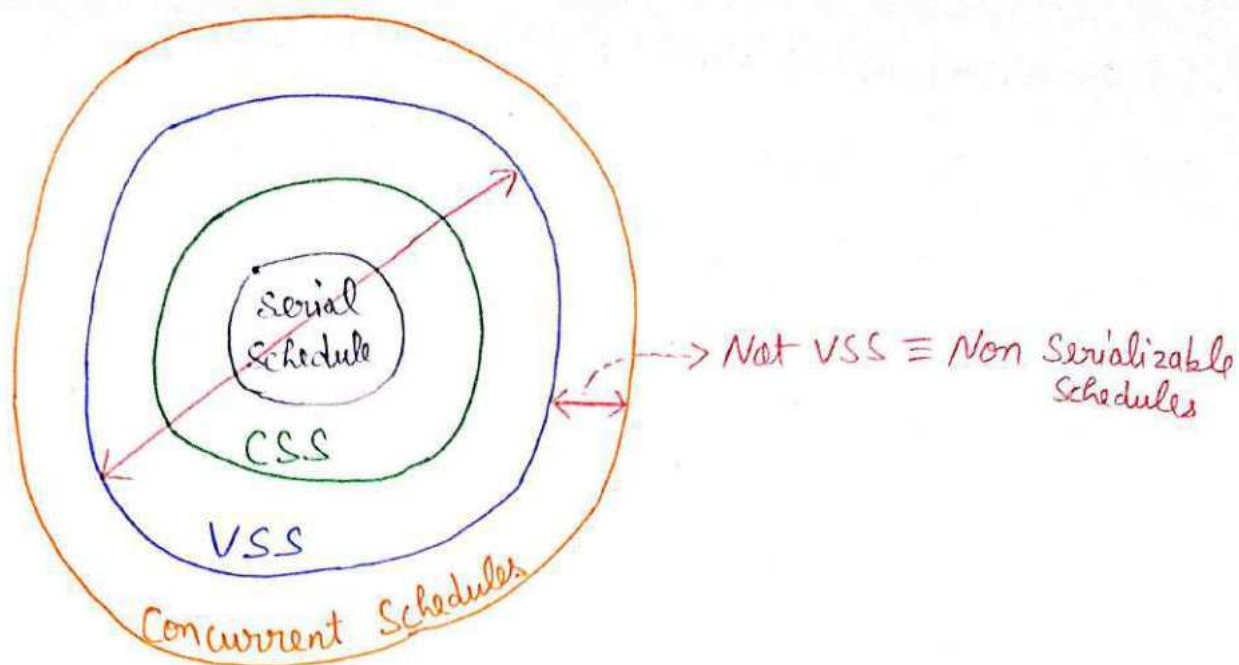
ii) # of schedules conflict equal to S.

Soln $\{S: g_1(A) g_2(A) g_3(A) w_1(B) w_2(B) w_3(B)\} \xleftrightarrow{\text{Conflict equal}}$

$\Rightarrow$ 15 concurrent schedule conflict equal to S.



⇒



NOTE:-

⇒ Serializable schedule $\equiv$ VSS.

⇒ Not CSS $\equiv$ may/may not serializable.

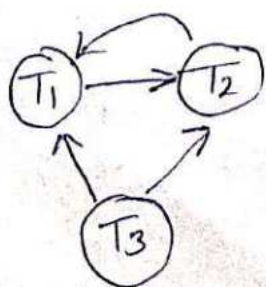
⇒ CSS testing condition is only sufficient but not necessary testing condition to test serializability. $P \rightarrow Q$

⇒ VSS testing condition is both sufficient and necessary condition to prove schedule is serializable. $P \leftrightarrow Q$

⇒ If ① S is not conflict serializable [precedence graph cyclic] (And)

② No blind writes in S. Then, S is also not view serializable.

Eg:- S: $g_1(A)$ $g_2(A)$ $g_3(A)$ $w_1(A)$ $w_2(A)$



{ Not CSS
+
No blind writes }

⇓

Not VSS //

* Classification of Schedules based on Recoverability :-

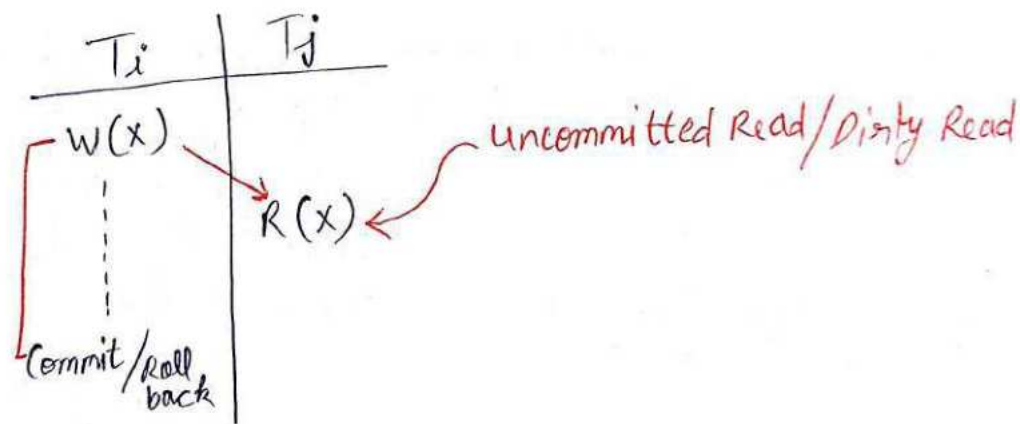
⇒ Concurrent execution may leads

- 1) Irrecoverable Problem
- 2) Cascading Rollbacks Problem
- 3) Lost Update Problem

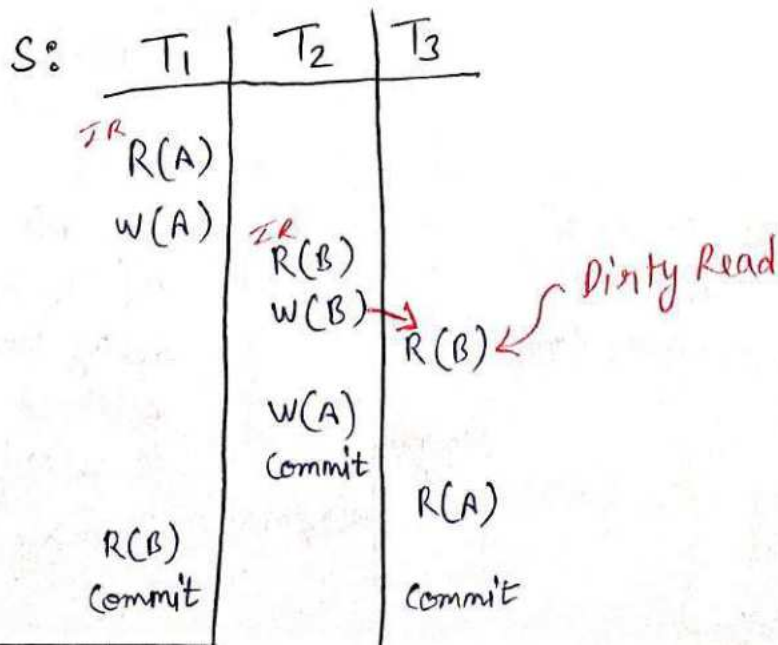
These problems also can cause incorrect result (failed for integrity) & these problems can occurs even schedule is serializable.

⇒ Uncommitted Read (Dirty Read) :-

* Transaction T_j reads data item x which is updated by uncommitted transaction T_i .



Eg:-



Problems

① Irrecoverable Problem:-

⇒ Schedule S is irrecoverable iff T_j reads data item x which is updated by T_i & commit of T_j is before commit/rollback of T_i .

T_i	T_j
$W(x)$	$R(x)$ ← Dirty Read
Commit/Rollback	Commit

} IRREC

Protocol Condition (Solution)

① Recoverable Schedule :-

⇒ Schedule S is recoverable iff
a) No uncommitted reads in Schedule S.
(OR)

b) If T_j reads x which is updated by T_i then commit of T_j must delay until commit or rollback of T_i .

T_i	T_j
$W(x)$	$R(x)$ ← Dirty Read
Commit/Rollback	Commit

② Cascade Rollbacks (Cascade Aborts):-

⇒ Because of failure of some transaction forced to Rollback some set of other transactions also.

⇒ Because of uncommitted Reads.

Disadvantage:- wastage of CPU time, I/O access cost.

② Cascadeless Rollbacks :- (Avoid cascade Aborts)

⇒ Schedule S with no cascade aborts iff T_j read of x which is reading data item x updated by T_i , then read of x of T_j must delay until commit/rollback of T_i .

T_i	T_j
$W(x)$	$R(x)$
Commit/Rollback	

③ Lost Update Problem:-

⇒ Lost update problem may occur if T_j writes data item x which is already written by uncommitted trans T_i .

T_i	T_j
$W(x)$	$W(x)$ ← may be lost
Commit/RB	

③ Strict Recoverable:-

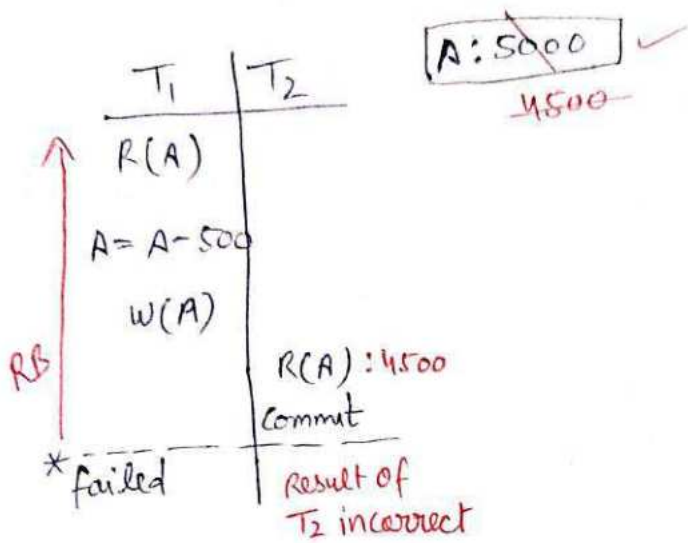
⇒ Schedule S Strict recoverable iff T_j R/W of x must delay until commit/rollback of T_i which is already written data item x .

T_i	T_j
$W(x)$	$R(x)/W(x)$
Commit/RB	

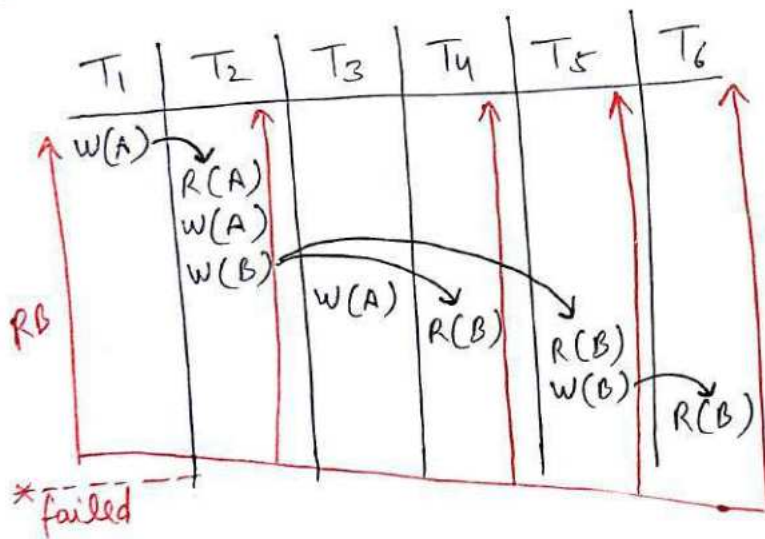
Eg:- ① IRREC Example

T_1 : Withdraw 500 from A

T_2 : check bal of A

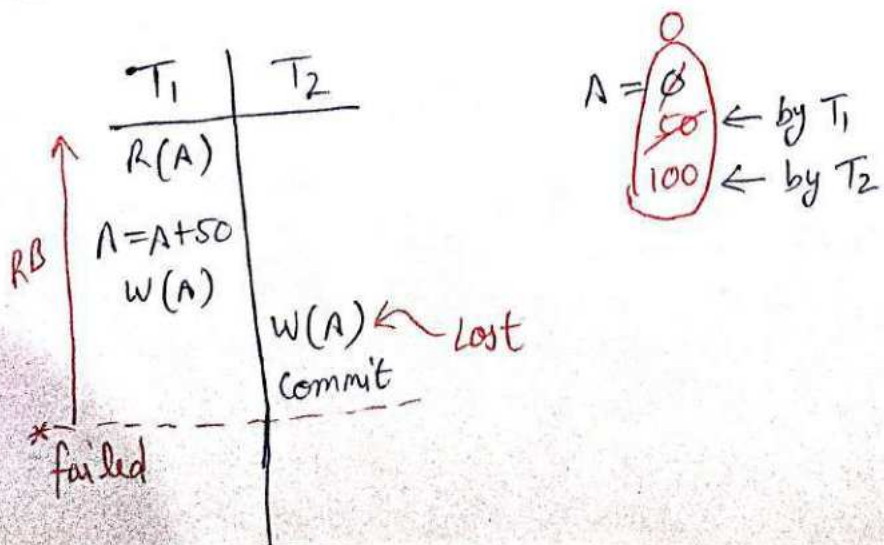


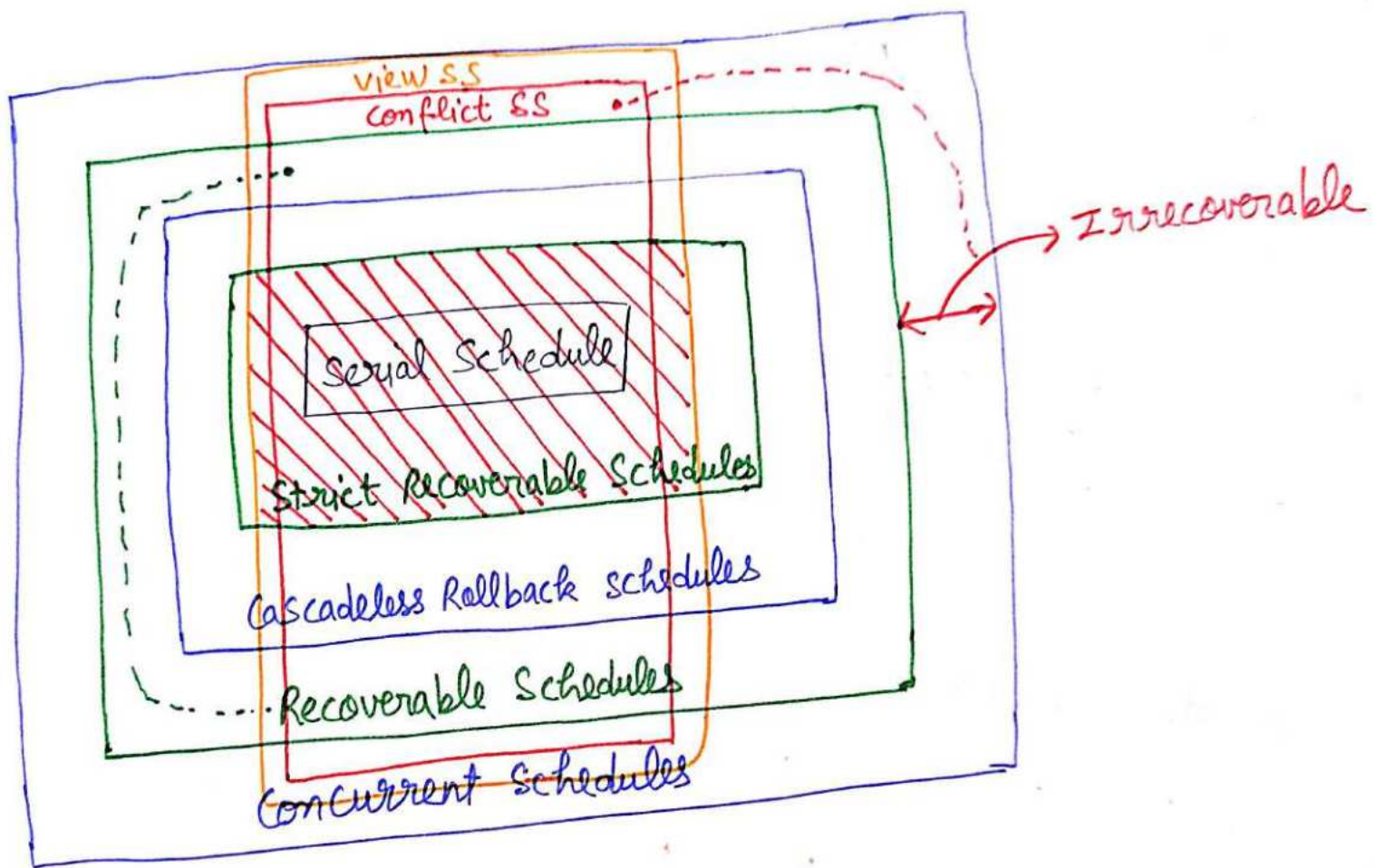
② Cascade Rollback's



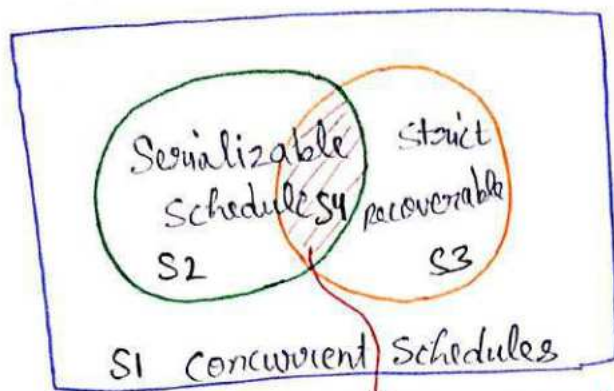
$T_2 \quad T_4 \quad T_5 \quad T_6$
Cascade Rollback's

③ Lost Update Problem





* Concurrency Control Protocols :- [CCP]



S4: Correct Schedules,
allowed by CCP

⇒ Concurrency Control Protocol should not allow non-serializable schedule or not strict recoverable schedule. in other words

⇒ The schedule which is exec. by CCP must be only serializable & Strict recoverable.

① Locking Protocol

② Time Stamp Ordering Protocol

① Locking Protocols :-

⇒ Lock :- Lock is variable which used to identify status of data item & also possible operations can perform over the data item.

• Binary Locking :-

1) Lock } two
2) Unlock } states

Eg:-

Trans(T₁)

Lock(A)

← Grants by CCP

R(A)

Lock(B)

← Denied by CCP

wait

Lock(B)

← Grants by CCP

W(B)

Unlock(A)

Unlock(B)

commit

Eg:-

Trans(T_1)

Lock(A) ← grants CPP

$\{ R(A) \}$
 $\{ W(A) \}$

T_1	T_2
lock(A)	
R(A)	

lock(A) ← denied
R(A)

Disadvantage :- Less degree of concurrency

⇒ Shared-Exclusive Locking :-

shared lock [S] :- Read only lock
Exclusive lock [X] :- Read-Write lock

} more than one lock
[modes of Lock]

Eg:-

Trans(T_1) ← S/X locking Trans + 2PL

S(A)
R(A)
X(B)
R(B)
W(B)
X(C)
W(C)
X(D)
R(D)
U(A)
U(B)
U(C)
U(D)
Commit

⇒

Request T_j	A	Hold T_i	
		S	X
S		Yes	No
X		No	No

[Lock compatible Table]

Eg:-

T_1	T_2
S(A)	
R(A)	
	S(A)
	R(A)
S(B)	
R(B)	
	X(B)

← Grants

← Denied

⇒ Two Phase Locking Protocol [2PL] :-

→ Guaranteed for Serializability

→ 2PL meant for Isolation

Def:-

①* Trans (T) can request lock on any data item in any mode until first unlock of trans (T).

Eg:- Trans (T)

X(A)

R(A)

W(A)

S(B)

R(B)

X(C)

W(C)

S(D)*

U(A)*

U(B)

U(C)

R(D)

U(D)

Commit

Locking Phase
[Growing Phase]

Lock Point of Trans (T) :- position of last lock on first unlock of Trans (T).

Unlocking Phase
[Shrinking Phase]

(OR)

②* Lock request of Trans (T) is not allowed in unlocking phase of transaction T.

① T_1 : Display A+B

Trans (T_1)

S(A)

R(A)

S(B)

U(A)

R(B)

U(B)

Commit

S/X + 2PL
Trans

T_2 : Transfer 50
A to B

T_3 : set A, B, C AS 1000

Trans(T_2)

X(A)
R(A)
W(A)
X(B)

U(A)
R(B)
W(B)
U(B)
Commit

S/X + 2PL Trans.

Trans(T_3)

X(A)
W(A)
X(B)
W(B)

X(C)
U(A)
U(B)
W(C)
U(C)
Commit

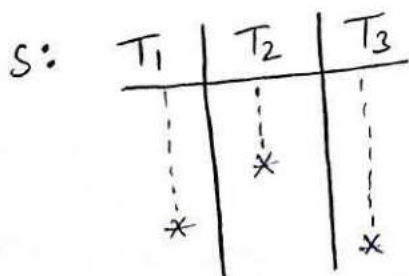
S/X + 2PL Trans.

⇒ Concurrent execution of T_1, T_2, T_3 Guaranteed for serializability (Isolation).

NOTE:-

- 1) If schedule S is not conflict serializable schedule then schedule not allowed to execute by 2PL.
- 2) If schedule allowed to execute by 2PL then schedule is also conflict serializable & conflict equal serial schedule is order of lock points of transactions.

Eg:-



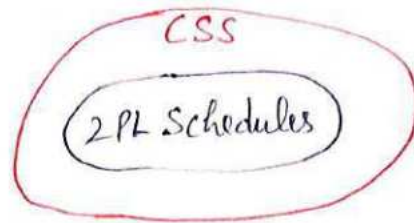
{ Lock points order
of trans } = { one of topological
order of schedules
precedence graph }

S: Executed by 2PL

S: CSS

Equal serial: $T_2 T_1 T_3$

3) Every 2PL schedule is also conflict serializable schedule but not every CSS is allowed to execute by 2PL.

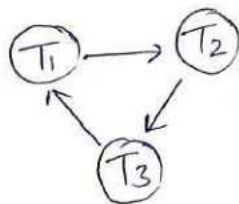


Q) $S: g_1(A) \rightarrow w_2(A) \rightarrow w_2(B) \rightarrow g_3(B) \rightarrow g_3(C) \rightarrow w_1(C)$, which is True?
Given schedule S is

- a) 2PL & CSS ~~X~~ b) 2PL & not CSS ~~X~~
c) CSS & not 2PL ~~X~~ ~~not~~ not CSS & not 2PL

Soln

CSS:



not CSS //

2PL Test :-

S:

T_1	T_2	T_3
$g_1(A)$		
$w_2(A)$		
$x(C)$		
	$x(A)$	
	$w_2(A)$	
	$x(B)$	
	$w_2(B)$	
	$u(A)u(B)$	
		$g_3(B)$
		$g_3(C)$
		$u(C)$
	$w_1(C)$	
	$u(C)$	

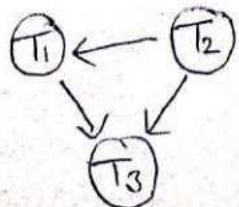
S not in 2PL //

Denied //

Q) $S: w_2(A) \rightarrow w_1(A) \rightarrow w_2(B) \rightarrow w_3(A) \rightarrow w_1(B) \rightarrow w_3(B)$

- ~~not~~ 2PL & CSS b) 2PL & not CSS ~~X~~
c) CSS & not 2PL ~~X~~ d) not CSS & not 2PL ~~X~~

CSS:



It is CSS //

2PL Test :-

T ₁	T ₂	T ₃
	X(A) w ₂ (A) X(B)U(A)	
X(A) w ₁ (A)		
	w ₂ (B)U(B)	X(A) ← grants w ₃ (A)
X(B)U(A)		
w ₁ (B) U(B)		X(B) w ₃ (B) U(A) U(B)

Equal Serial :-
T₂ → T₁ → T₃

2PL schedule //

Q) S: w₃(A) w₁(A) w₂(A) w₃(B) w₁(B) w₂(B)

a) 2PL & CSS X

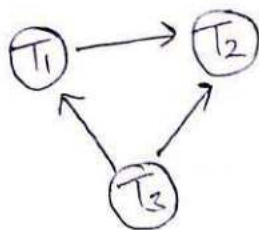
b) 2PL & not CSS X

c) CSS & not 2PL

d) not CSS & not 2PL X

Soln

CSS :-



It is CSS,

2PL Test :-

S:

T ₁	T ₂	T ₃
		X(A) w ₃ (A) X(B)U(A)
X(A) w ₁ (A)		U(B)X
X(B)U(A) X X	X(A) ← Denied w ₂ (A)	
		w ₃ (B) U(B)
w ₁ (B)		
	w ₂ (B)	

not 2PL

⇒ Limitations of 2PL :-

- 1) 2PL Restriction not ensures strict recoverability
- 2) 2PL restriction may leads to deadlock.
- 3) 2PL restriction may leads starvation.

Eg:-
 $T_1: \mathcal{R}_1(A) \mathcal{R}_1(B) [S(A) S(B) U(A) U(B)]$
 $T_2: \mathcal{W}_2(B) \mathcal{W}_2(A) [X(B) X(A) U(B) U(A)]$

T_1	T_2
$S(A)$ $\mathcal{R}_1(A)$	$X(B)$ $\mathcal{W}_2(B)$
$S(B)$ $\mathcal{R}_1(B)$	$X(A)$ $\mathcal{W}_2(A)$

Denied T_1 waiting for T_2 (pointing to $S(B)$)
 Denied T_2 waiting for T_1 (pointing to $X(A)$)
 S not 2PL & deadlock

Q) $T_1: \mathcal{R}_1(A) \mathcal{R}_1(B)$
 $T_2: \mathcal{W}_2(A) \mathcal{W}_2(B)$

non serial concurrent of $T_1 T_2$

a) deadlock possible if executed by 2PL

~~b) No deadlock if executed by 2PL~~

~~c) Serializable~~
 i.e some non serial is S

~~d) non-serializable, i.e some non serial is N-S~~

Soln b) $T_1: \mathcal{R}_1(A) \mathcal{R}_1(B) [S(A) S(B) U(A) U(B)]$
 $T_2: \mathcal{W}_2(A) \mathcal{W}_2(B) [X(A) X(B) U(A) U(B)]$

S not 2PL
 &
 not deadlock

T_1	T_2
$S(A)$ $\mathcal{R}_1(A)$ $S(B) U(A)$	$X(A)$ $\mathcal{W}_2(A)$ $X(B)$ $\mathcal{W}_2(B)$
$\mathcal{R}_1(B)$ $U(B)$	

Denied, T_2 waiting for T_1 (pointing to $X(B)$)

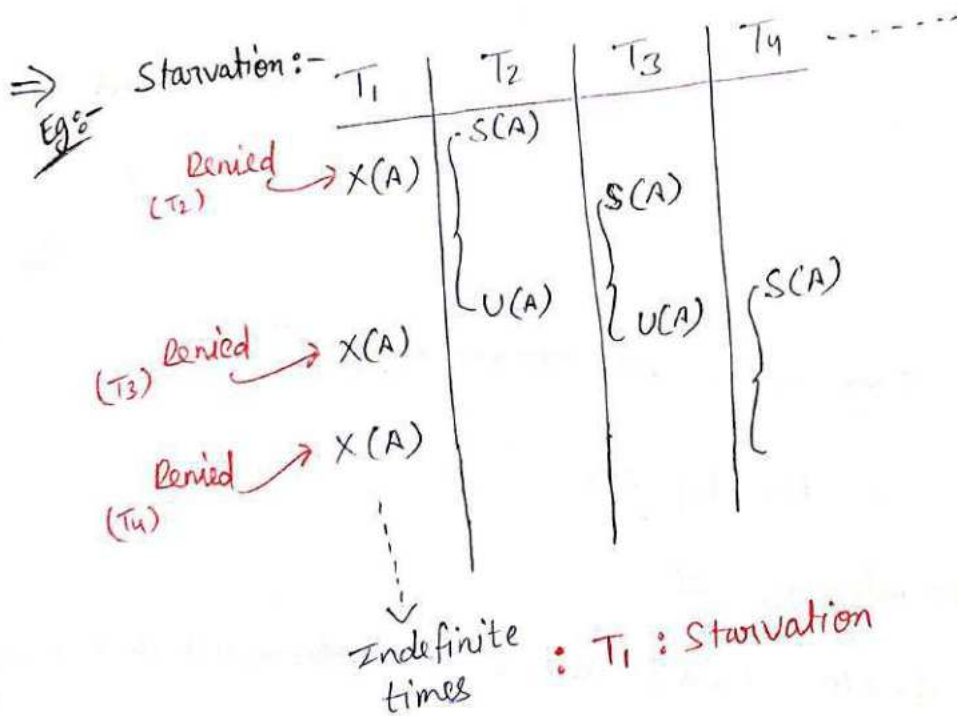
NOTE:-

$\Rightarrow$ If T_1 & T_2 locks their ^{possessed} data item in proper sequence of data item address by using 2PL restriction then not possible to form the deadlock even by using 2PL restriction.

$A_1 A_2 A_3 A_4 A_5 \dots A_n$
 sequence of data item base on their Address //

c) Serializable :- $r_1(A) \rightarrow w_2(A) \rightarrow r_1(B) \rightarrow w_2(B) \checkmark$

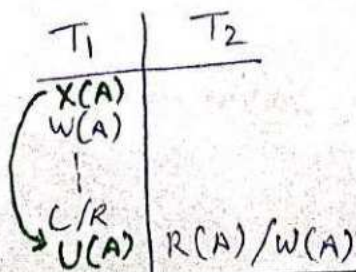
d) Non-Serializable :- $r_1(A) \rightarrow w_2(A) \rightarrow w_2(B) \rightarrow r_1(B) \checkmark$



$\Rightarrow$ Strict 2PL Protocol :- $\rightarrow$ CCP

① Basic 2PL Condition :- Lock request not allowed during unlocking phase. (Serializable)

② Strict Recoverable :- all exclusive locks [lock's which are allowed to update the data] of Trans (T) must hold until commit/RB of Trans (T). (Strict Recoverable)



Eg:-

Trans (T) $\Rightarrow$ Strict 2PL + S/X Trans //

✓ X(A)
R(A)
W(A)
S(B)
R(B)
S(C)
R(C)
✓ X(D)
W(D)
U(B)
U(C)
commit
U(A)
U(D)

$\Rightarrow$ Rigorous 2PL Protocol :-

① Basic 2PL condition
(and)

② all locks of Trans (T) must hold until commit/rollback
of Trans (T)

Eg:-

Trans (T)

X(A)
R(A)
W(A)
S(B)
R(B)
S(C)
R(C)
X(D)
W(D)

commit

U(A)
U(B)
U(C), U(D)

Lock point

⇒ Conservative 2PL Protocol :- (not efficient protocol)

- ① Basic 2PL conditions
(and)
- ② Trans must lock all required data items at initial stage.

Eg:-

Trans(T)

X(A)
S(B)
S(C)
X(D)

R(A)

W(A)

U(A)

R(B)

U(B)

R(C)

U(C)

W(D)

U(D)

Commit

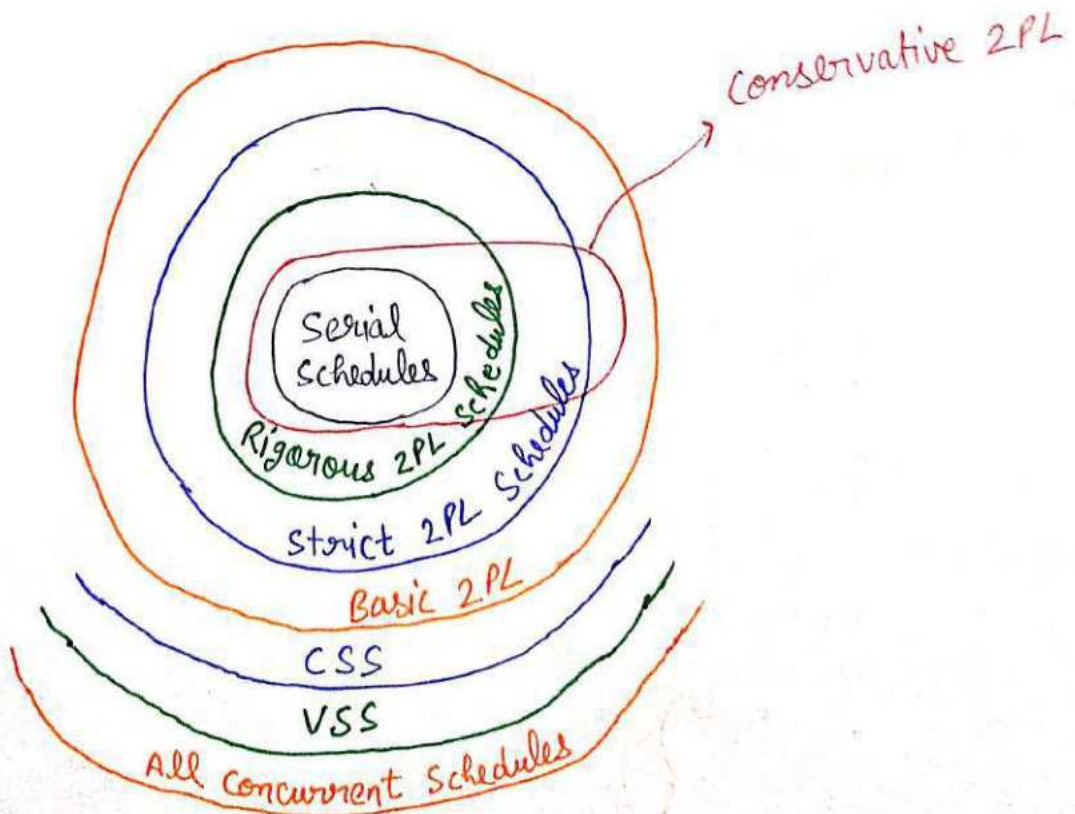
✓ serializable

✗ not strict Recoverable

✓ deadlock free

✓ starvation possible

⇒



② Time stamp Ordering Protocols [TSO] :-

⇒ Time stamp value of Transactions :-

* Unique value assign by DBMS for each transaction in sequence order

Eg:- ①

S:	10 <u>T₁</u>	15 T ₂	20 T ₃	25 T ₄	30 <u>T₅</u> <i>Younger</i>
	R(A) W(A)		R(A) R(B) W(B)	W(A)	

⇒ T.S value of data item (x)

1) Read TS(x)

2) Write TS(x)

1) Read TS(x) :-

* Highest Trans. TS value that has performed Read(x) is Read TS(x)

2) Write TS(x) :-

* Highest Trans. TS value that has performed Write(x) is Write TS(x)

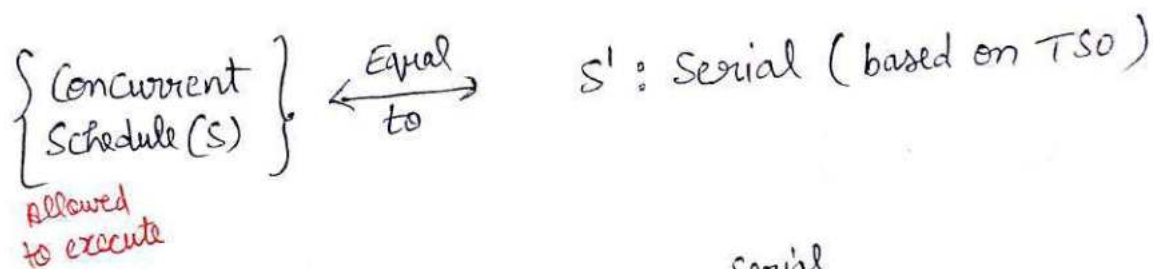
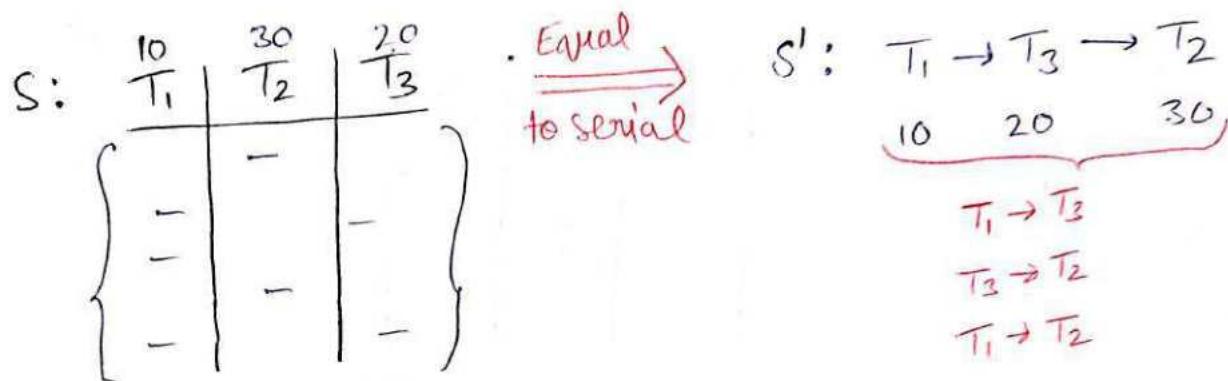
Eg:- from eg ① :-

	A	B
RTS	10 20	20
WTS	10 25	20

↙
Youngest Trans
executed W(A)

⇒ Basic Idea of TSO Protocol :-

- * TSO Protocols allowed to execute concurrent schedule (No Rollbacks) if concurrent execution equal to serial based on time stamp orders otherwise Rollback transaction.

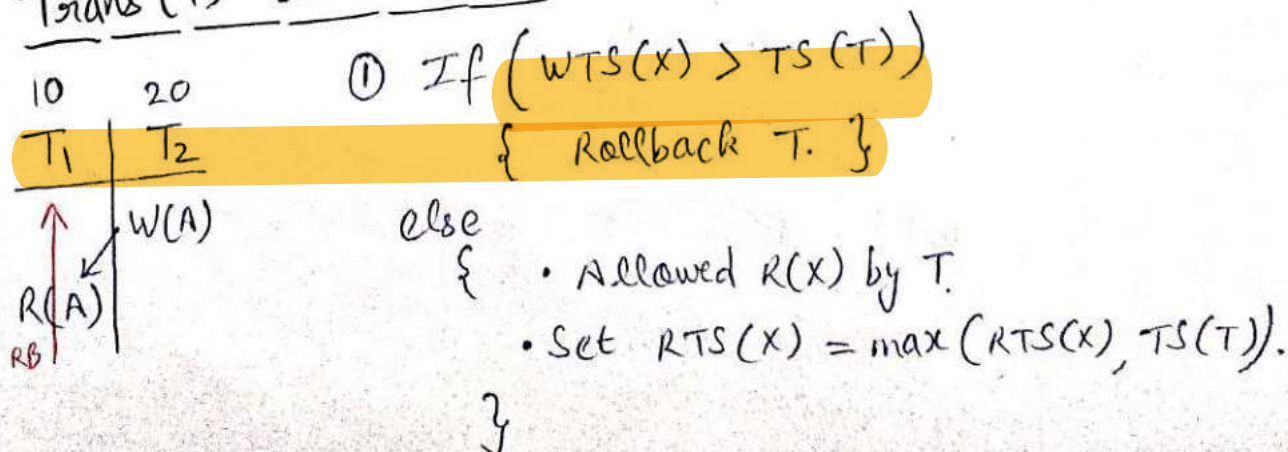


- ① Basic TSOP → Conflict equal^{Serial} based on TSO.
- ② Thomas write TSOP → View equal serial based on TSO.

① Basic TSO Protocol :-

[Allowed conflict pair if conflict pair precedence same as TSO]

i) Trans (T) Issues R(x) :-



ii) Trans (T) Issues W(X) :-

$\Rightarrow$ If $(RTS(x) > TS(T))$ ①
 { Rollback T }

else if $(WTS(x) > TS(T))$ ②
 { Rollback T. }

else

{ Allowed W(X) by T.

• Set $WTS(x) = TS(T)$.

3

Q) S: $r_2(A) \rightarrow r_1(A) \rightarrow w_3(A) \rightarrow w_2(B) \rightarrow r_3(B) \rightarrow w_2(C) \rightarrow w_1(C) \rightarrow r_1(C) \rightarrow r_2(D) \rightarrow r_3(E)$

i) If TSO of trans $(T_1 T_2 T_3) = (20, 10, 30)$, what are trans rollbacks by using Basic TSOP?

Soln

$(T_1 T_2 T_3) = (20, 10, 30)$

Ans: No Rollbacks //

TSO $\Rightarrow T_2 \rightarrow T_1 \rightarrow T_3$
 10 20 30

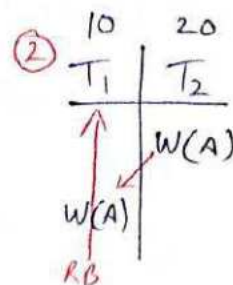
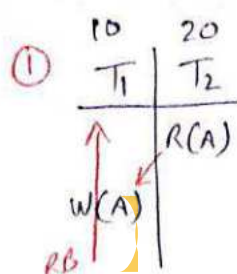
ii) If TSO of trans $(T_1 T_2 T_3) = (30, 10, 20)$ what are rollbacks by using Basic TSOP?

Soln

TSO $\Rightarrow T_2 \rightarrow T_3 \rightarrow T_1$
 10 20 30

Ans = T_3 Rollback //

S: $r_2(A) \rightarrow r_1(A) \rightarrow \cancel{w_3(A)} \rightarrow w_2(B) \rightarrow \cancel{r_3(B)} \rightarrow w_2(C) \rightarrow w_1(C) \rightarrow r_1(C) \rightarrow r_2(D) \rightarrow \cancel{r_3(E)}$



⇒ Basic TSO Protocol allowed to execute the schedule if schedule is conflict serializable & conflict equal serial order is same as TSO.

$$\text{If } \left\{ \begin{array}{l} \text{TSO of} \\ \text{Trans} \end{array} \right\} = \left\{ \begin{array}{l} \text{One of the topological order} \\ \text{of schedule precedence graph} \end{array} \right\}$$

Then Schedule allowed by BTSP (No Rollbacks)
Otherwise some Rollbacks.

② Thomas Write TSO Protocol [TWTSP] :-

1) Trans(T) Issues R(x) :-

* If $(WTS(x) > TS(T))$
{ Rollback T }

else

{ • Allowed R(x) by T.

• Set $RTS(x) = \max(RTS(x), TS(T))$

}

2) Trans(T) Issues W(x) :-

* If $(RTS(x) > TS(T))$ ①
{ Rollback T. }

else if $(WTS(x) > TS(T))$ ②

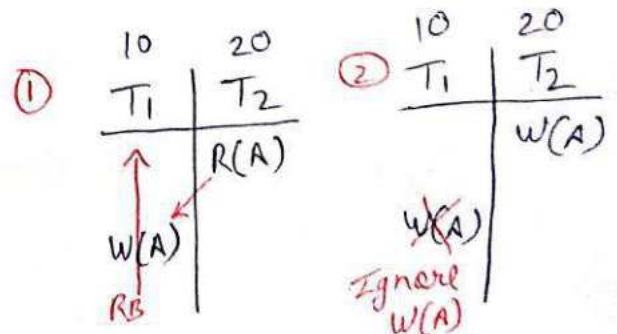
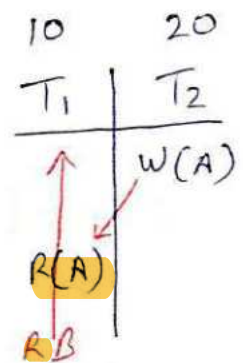
{ Ignore W(x) of T continue Trans T }

else

{ • Allowed W(x) by T

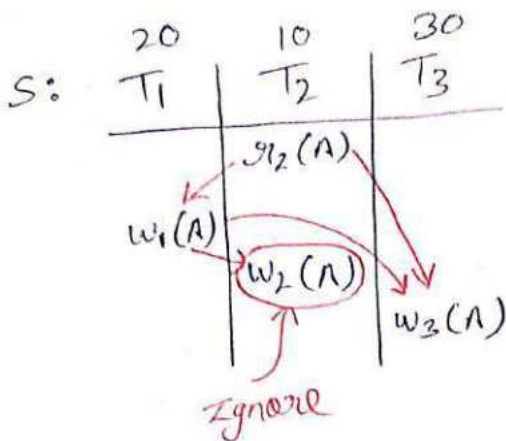
• Set $WTS(x) = TS(T)$.

}



Q) S: $g_2(A)$ $w_1(A)$ $w_2(A)$ $w_3(A)$
 $TSO(T_1 T_2 T_3) = (20, 10, 30)$

Soln



• BTSS :- Rollback T_2 //

• TWTSS :- No Rollback //

View equal to serial based on TSO

$[T_2 T_1 T_3]$ Serial

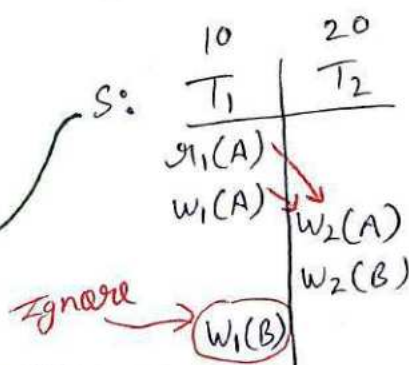
$S': g_2(A) w_2(A) w_1(A) w_3(A)$

Q) S: $g_1(A)$ $w_1(A)$ $w_2(A)$ $w_2(B)$ $w_1(B)$
 $(T_1 T_2) = (10, 20)$ RB's using BTSS & TWTSS?

Soln

TSO $\Rightarrow T_1 \rightarrow T_2$
 10 20

$[T_1 \rightarrow T_2]$ view equal serial



$S': g_1(A) w_1(A) w_1(B) w_2(A) w_2(B)$

• BTSS :- T_1 Rollback //

• TWTSS :- No Rollbacks //

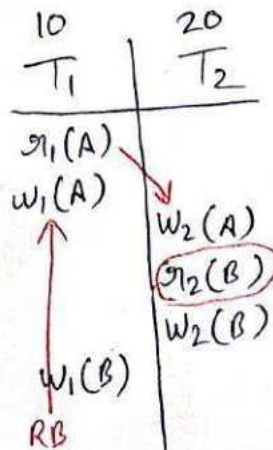
Q) S: $g_1(A)$ $w_1(A)$ $w_2(A)$ $g_2(B)$ $w_2(B)$ $w_1(B)$ $(T_1 T_2) = (10, 20)$
 RB's using BTSS & TWTSS?

Soln

TSO $\Rightarrow T_1 \rightarrow T_2$

Not View Equal serial

$S': g_1(A) w_1(A) w_1(B) w_2(A) g_2(B) w_2(B)$



• BTSS :- T_1 Rollback

• TWTSS :- T_1 Rollback

Basic TS ordering Protocol

- ⇒ Ensures conflict serialization
(Executed schedule conflict equal to serial based on TS ordering)
- ⇒ Free From Deadlocks
- ⇒ Not Free From Starvation
- ⇒ Not guaranteed strict recoverable

Thomas Write TS ordering Protocol

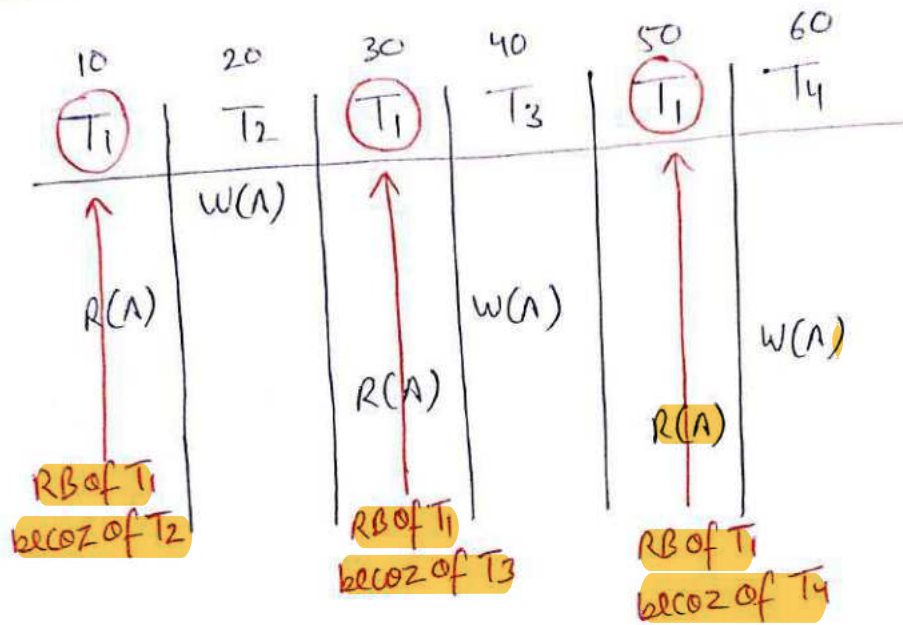
- ⇒ Ensures View serialization
(Executed schedule view equal to serial based on TS ordering)
- ⇒ Free From Deadlocks
- ⇒ Not Free From Starvation
- ⇒ Not guaranteed strict recoverable

Strict TS ordering Protocol (Basic TSO + strict Rec.)

- ⇒ Ensures conflict serialization
(Executed schedule conflict equal to serial based on TS ordering)
- ⇒ Free From Deadlocks
- ⇒ Not Free From Starvation
- ⇒ Guaranteed strict recoverable

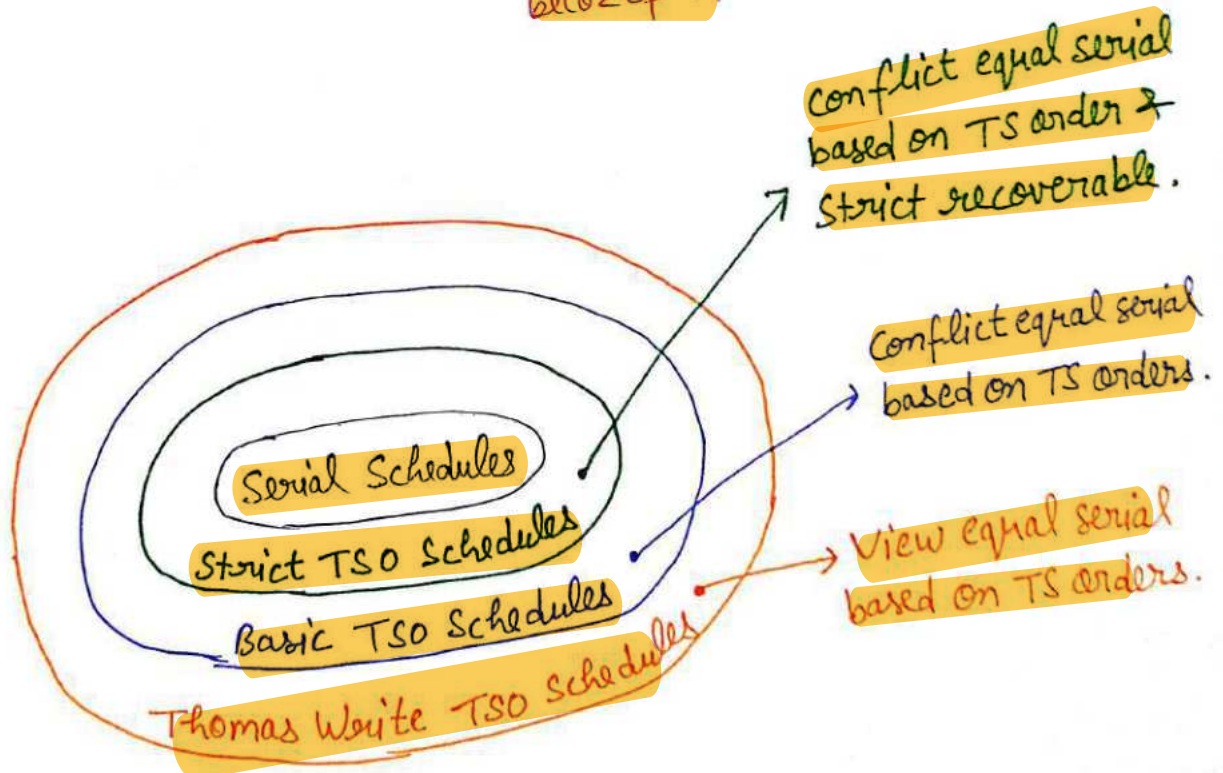
⇒ In TSO Protocols no concept of holding data item and waiting for data item. (Deadlocks not possible).

⇒ BTSO Protocol :-



(Starvation of T_1)

⇒



	Basic 2PL	Strict 2PL	Rigorous 2PL	Conservative 2PL
① Guaranteed Serializability	Yes	Yes	Yes	Yes
② Guaranteed Strict Recoverable	No	Yes	Yes	No
③ Free From Deadlocks	No	No	No	Yes
④ Free From Starvation	No	No	No	No
⑤ Equal Serial Schedule	based on lock grant order	based on lock grant order	based on commit order	based on lock grant order